

TTT와 Titans의 효율 — Scaling 병목과 해법, 그리고 memory device 집중 조명

neural-memory-study · 집중 조명 study paper

2026-07-16

Contents

0. 들어가며 — 목적 · 범위 · 읽는법	2
Part I — 집중 조명	2
1. TTT 집중 조명 — test-time 에 학습되는메모리	2
2. Titans 집중 조명과 그 효율전반	8
Part II — Scaling 병목 3 종	14
3. 병목 ① — model-size scaling	14
4. 병목 ② — training parallel/batch scaling	19
5. 병목 ③ — serving batch inference + Prefill/Decode(P/D)	24
Part III — 해법 3 종 (무엇을 · 어떻게 · 왜 working)	30
6. 해법 ① — model-size scaling 해결	30
7. 해법 ② — training parallel/batch scaling 해결	35
8. 해법 ③ —serving/P/D batch inference 해결	42
Part IV — memory device 집중조명	50
9. memory device 집중 조명 ① —한계	50
10. memory device 집중 조명 ② — 개선으로 해결되는해법	55
11. 종합 — 세 병목 × 해법 × memory device 지도	61
12. 열린 질문	62
References	62

0. 들어가며 — 목적 · 범위 · 읽는법

이 글은 TTT(test-time training)와 Titans 계열의 ‘효율과 scaling’을 논문의 그림·표·수치를 근거로 깊이파고드는 집중 조명이다. 세 가지 scaling 병목 — ① model-size, ② training의 parallel/batch, ③ serving의 batch inference(Prefill/Decode 포함) — 각각에 대해 (1) 병목이 왜생기는지, (2) 그것을 푸는 논문들이무엇을 · 어떻게 · 왜 working 하는지, (3) 이 모든것이 만나는 **memory device**의 한계와 개선 해법을 다룬다.

세 가지 caveat.

1. 이 글은 pre-training 레시피나 아키텍처 세부가 아니라 ‘**효율 · scaling 병목과 해법**’의 시스템 관점이다. 같은 논문도 여기서는 효율 측면만 조명한다.
2. 다루는 논문 상당수가 2025~2026년 것이라 수치 · 재현성이 미확정인 경우가 많다. 문장마다 근거등급을 달았고 모든 수치는 best-effort 검증이다 (다중 에이전트 심층집필 → 원문 · 그림 대조 사실검증).
3. 단일 논문으로는 그림이 불완전하다. 병목과 해법, 이론과시스템을 함께 봐야 이해된다. 서로 다른갈래 (distillation · 병렬화 · 서빙)는 억지로하나로 봉합하지 않았다.

읽는 법. 근거등급은 [a] 논문 명시 · [b] 메커니즘상 추론 · [c] 미명시 · 불확실. 인용은 대괄호 번호 [N](끝의 References). 그림은 각 논문의 원문 Figure를 그대로 인용했으며 캡션에출처 · Figure 번호 · [N]을 밝혔다 (제3자 그림 — 저작권은 원저자, 재사용 시 확인 필요). LaCT의 “~70% GPU utilization”은 원문 보고 수치이나 실측 하드웨어 (A100/H100) 표기가 소스마다 엇갈려 하드웨어 세부는 [b]로 둔다.

구조. Part I(1-2장) TTT·Titans 집중 조명 → Part II(3-5장) 세 병목 → Part III(6-8장) 세 해법 → Part IV(9-10장) memory device 한계 · 개선 → 11장 종합 지도 → 12장 열린 질문 → References.

Part I — 집중 조명

1. TTT 집중 조명 — test-time에 학습되는메모리

TTT의 정의: 은닉상태가 곧 ‘지금 학습 중인모델’

Test-Time Training(TTT)의 출발점은 하나의 대담한 재해석이다. RNN의 은닉상태 h_t 를 단순한 벡터가 아니라 **하나의 작은 모델의 weight**로 보고, 시퀀스를 따라 토큰이 들어올 때마다 그 weight를 self-supervised loss로 한 스텝 학습시킨다는 것이다. Sun 등의 「Learning to (Learn at Test Time): RNNs with Expressive Hidden States」 [1]은이를 다음과 같이 정식화한다. 은닉상태 W_t 는 어떤 함수 $f_W(\cdot)$ 의 파라미터이고, 갱신 규칙은

$$W_t = W_{t-1} - \eta \nabla_W \ell(W_{t-1}; x_t)$$

즉 **매 토큰마다 self-supervised gradient 1스텝**이다. 여기서 self-supervised loss는 입력 토큰을 저장된 뷰로 손상시킨 뒤 복원하는형태로, 논문은 학습 가능한 projection $\theta_K, \theta_V, \theta_Q$ 를두어 “무엇을 기억할지” 조차 outer loop에서 메타학습하도록 한다. 출력은학습된 모델을 query에 적용한 $z_t = f_{W_t}(\theta_Q x_t)$ 이다.

이 그림이 중요한 이유는, TTT가 기존 시퀀스 층 (attention, SSM) 과근본적으로 다른 대상은 은닉상태에 담는다는 점을 한눈에 보여주기때문이다. Transformer는 과거 전체를 KV 캐시에 그대로쌓고 (비압축 · 선형 증가), Mamba류 SSM은 고정 크기 상태에 선형갱신으로 압축한다. TTT는 그 고정 크기 상태를 **하나의 학습 문제의해로** 만든다 — 상태의 표현력은 곧 내부 모델 f_W 의 표현력이다. f_W 가 선형 모델이면 **TTT-Linear**, 2-layer MLP이면 **TTT-MLP**이다. 후자는 은닉상태가 비선형 함수 전체이므로이론적으로 더 풍부한 장기 의존성을 압축할 수 있다.

핵심 실증 결과는 “문맥을 더 넣을수록 계속 나아지는가”이다. 논문은 125M~1.3B 규모에서 강한 Transformer, Mamba와 비교했을 때 TTT-Linear와 TTT-MLP는 토큰을 더 조건화할수

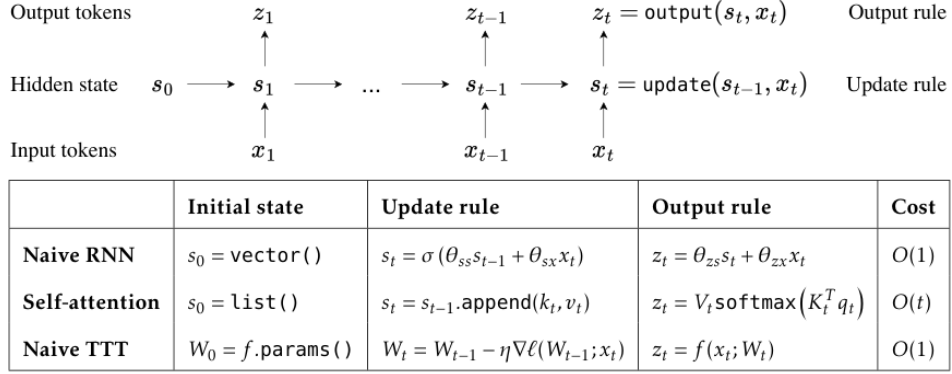


그림 1.1. 시퀀스 층을 '은닉상태를 규칙으로 변환하는 학습기'로 보는 TTT의 관점 — inner loop가 매 토큰 은닉 weight를 갱신한다 (출처: TTT, Figure 3, [1])

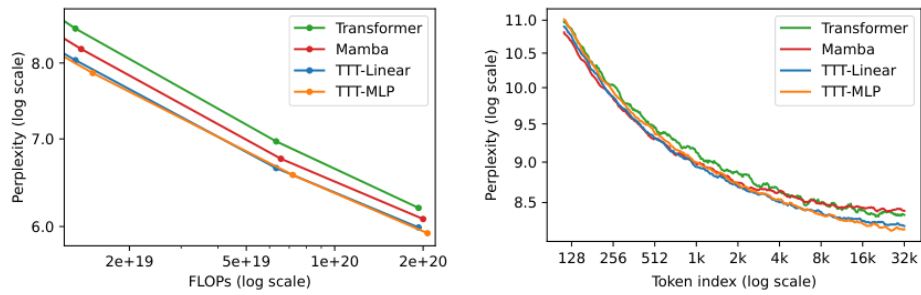


그림 1.2. 8k context에서 TTT-Linear/TTT-MLP가 Mamba와 유사한 perplexity에 도달 — 은닉상태를 '학습 문제의 해'로 만든 접근이 고정 크기 SSM 상태에 필적함을 보인다 (출처: TTT, Figure 2, [1])

록 perplexity 를 계속 낮추는반면, **Mamba 는 16k context 이후 perplexity 를 더 낮추지 못한다**고초록에서 명시적으로 보고한다 [1][a]. 이는 고정 크기 선형상태 (Mamba) 의 압축 한계를 “상태를 학습 문제로 만든” TTT 가 넘어설 여지를보여주는 대목이다.

표: 「Learning to (Learn at Test Time)」 주요 관찰 (요지 재현, [1])

항목	TTT-Linear	TTT-MLP	Mamba	Transformer
은닉상태	선형 모델 weight	2-layer MLP weight	고정 선형 상태	KV 캐시 (비압축)
문맥 확장 시 perplexity	계속 하락	계속 하락 (장문에서 잠재력 큼)	~16k 에서 정체	계속 하락하나 $O(N^2)$
8k wall-clock	Transformer 보다 빠름 · Mamba 와 유사	메모리 I/O 병목	빠름	느림 (장문)

skeptic 관점에서 이 표의 “계속 하락” 은 1.3B 이하 · 특정데이터에서의 관찰이며, TTT-MLP 는 latency/memory I/O 병목으로 실사용 이점이 제한적임을 논문 스스로명시한다는 점을 강조해 둔다.

fast-weight 계보: linear attention = fast weight, delta rule = 오차교정 gradient

TTT 는 하늘에서 떨어진 아이디어가 아니라 1990 년대 Schmidhuber 의 **fast weight** 계보의 현대적 부활이다. Schlag·Irie·Schmidhuber 의 「Linear Transformers Are Secretly Fast Weight Programmers」 [2] 는 linearised self-attention 이 fast weight programmer 와 **형식적으로 동등함**을보였다. “느린” 네트워크가 key·value 의 외적 (outer product) 합으로 “빠른” weight 행렬을 프로그래밍하고, query 로 그 행렬에서 값을읽는다:

$$W_t = W_{t-1} + v_t k_t^T, \quad y_t = W_t q_t$$

이는 정확히 linear attention 의 누적 상태이며, Katharopoulos 등의 「Transformers are RNNs」 [3] 가 softmax 를 커널 특성사상으로 대체해얻은 재귀식과 같다. 여기서 Schlag 의 결정적 통찰은 **순수 additive outer product 는 유한 메모리를 금세 포화시킨다**는 것이다. key 가 서로겹치면 값끼리 간섭하기 때문이다. 이를 고치기 위해 그는 additive 규칙을 **delta rule** 로 교체한다:

$$W_t = W_{t-1} + \beta_t (v_t - W_{t-1} k_t) k_t^T$$

여기서 $(v_t - W_{t-1} k_t)$ 는 “현재 메모리가 k_t 에 대해 내놓는예측” 과 목표 v_t 의 **오차**이고, 이 오차를 줄이는 방향은 정확히제공손실 $\frac{1}{2} \|W k_t - v_t\|^2$ 의 gradient 이다. 즉 delta rule = 오차교정 = **online gradient descent 1 스텝**이다. 이 등식이 TTT 와 fast weight 를 하나로 묶는 다리다. 이후의 gated linear attention 계열, 예컨대 GLA[4] 는 이 계보에서 데이터 의존적 게이팅으로 메모리감쇠 (망각) 를 추가한 변형으로, DeltaNet 계열은 위 delta rule 을 병렬 학습가능한 형태로 되살린 변형으로 읽을 수 있다.

test-time regression: 세 개의 설계 선택으로통일

Wang·Shi·Fox 의 「Test-time regression: a unifying framework for designing sequence models with associative memory」 [5] 은 이 흩어진 관점들을 하나의 우산 아래 모은다. 핵심명제는 “**associative memory 에 토큰을 기억시키는 것 = test-time 에회귀 (regression) 를 푸는 것**” 이다. 연상기억은 memorization(저장) 과 retrieval(인출) 의 2 단계인데, memorization 을 key → value 회귀 문제로캐스팅하면 linear attention, SSM, fast-weight programmer, online learner, 심지어 softmax attention 까지 모두 **세 가지 설계 선택**의특수 경우로 떨어진다:

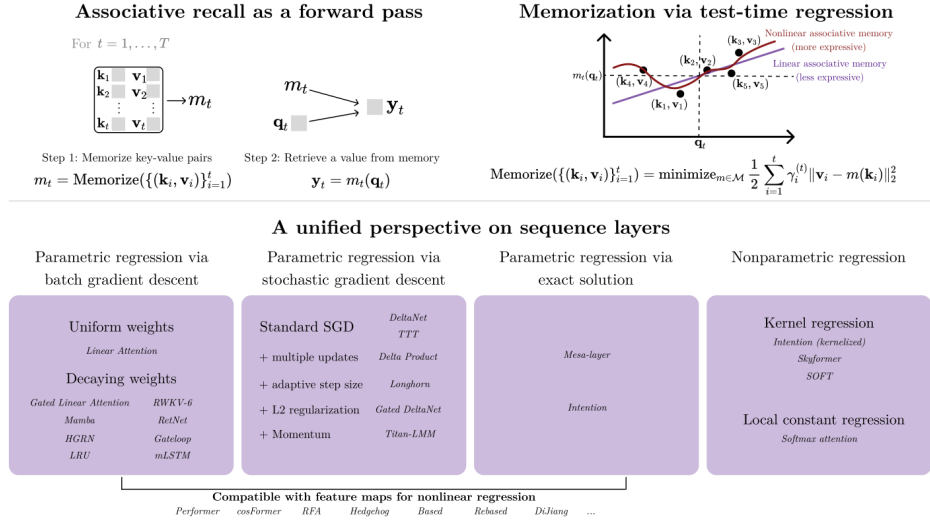


Figure 1: Our framework provides a systematic way to derive sequence models that can perform associative recall, following a two step process of memorization and retrieval. The memorization step can be formalized as a solving a regression problem at test-time. Our perspective results in a “recipe” for deriving sequence layers by making three choices: the importance of each association $\{\gamma_i^{(t)}\}_{i=1}^t$, the regressor

그림 1.3. memorization=regression 으로 보는 통일 프레임워크: 회귀 weights·함수 클래스·최적화 알고리즘 세축으로 기존 시퀀스 층을 특수 경우로 유도 (출처: Test-time Regression, Figure 1, [5])

1. **회귀 weights** — 각 (key,value) 쌍을 얼마나 중요하게볼지 (=recency 게이팅/감쇠).
2. **regressor 함수 클래스** — 선형이면 linear attention/TTT-Linear, 비선형 (MLP) 이면 TTT-MLP, 무한폭커널이면 softmax attention.
3. **test-time 최적화 알고리즘** — 단일 gradient step(delta rule), 여러 step, 혹은 closed-form least-squares 해.

이 그림이 논지에서 중요한 이유는, TTT의 inner loop가 임의적 트릭이 아니라 **잘 정의된 회귀 문제를 test-time에 온라인으로 푸는절차**임을 못박기 때문이다. 예컨대 delta rule은 “step size 하나로 recursive least squares를 근사하는 것”이고, closed-form을 쓰면 학습가능한 파라미터 없이도 non-stationary online 회귀가 성립한다.

이 프레임워크의 실천적 가치는 “무엇을 바꾸면 무엇이 좋아지는가”를 축별로분해해준다는 점이다. recency 게이팅을 추가하면 Mamba/GLA 의망각이 되고, 함수 클래스를 키우면 TTT-MLP의 표현력이 되며, 최적화 step 수를 늘리면 test-time 계산을 정확도로 환전한다. skeptic 노트: 이 통일은**설명적**이지 새 SOTA를 보장하지 않는다 — 세 축 어디에 자원을쓸지는 여전히 경험적 문제다.

계산 이중성: parallel / recurrent / chunkwise 와 dual form

TTT의 아름다움은 개념에 있지만, 병목도 바로 여기서 시작된다. inner loop는 정의상 **순차적**이다: W_t 는 W_{t-1} 에 의존한다. GPU/TPU는 큰 행렬곱 (matmul)을 TensorCore로 처리할 때만빠르는데, 토큰당 gradient 1스텝은 작고 순차적인 연산들의 사슬이라 TensorCore를 텅 빈 채로 돌린다. [1]은 이를 두 가지로 완화한다.

첫째, **mini-batch TTT**: 매 토큰이 아니라 크기 b 의 청크 단위로갱신한다. $b = 1$ 이면 순수 online GD, $b = T$ (전체 시퀀스)면 배치 GD 예가깝다. b 는 병렬성과 갱신 빈도 (=적응 속도)의 트레이드오프를 직접 왼손잡이다.

둘째, **dual form**: 청크 내부의 gradient 계산을 하나의 큰 matmul로재배열한다. primal form은 각 토큰의 gradient와 중간 weight 행렬을명시적으로 materialize하지만, dual form은 청크 시작 weight W_0 기준으로 청크 전체의 기여를 한 번의 행렬-행렬 곱으로 묶어낸다. 두 형태

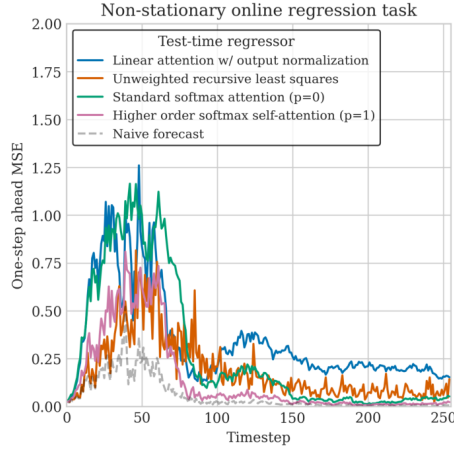


Figure 2: Solving a non-stationary online regression task *without any learnable parameters*. The regression-memory correspondence enables us to perform regression via a single forward pass of a test-time regression layer. The inputs change more rapidly in initial $T/4$ timesteps, making it harder for regressors to learn. The latter $3T/4$ timesteps are more stable; however, without decaying weights, linear attention and unweighted RLS are unable to adjust to the resulting non-stationary data.

그림 1.4. learnable parameter 없이 closed-form 회귀만으로 non-stationary 온라인 신호를 추적 — 최적화 알고리즘 축의 선택이 곧메모리 동역학 (출처: Test-time Regression, Figure 2, [5])

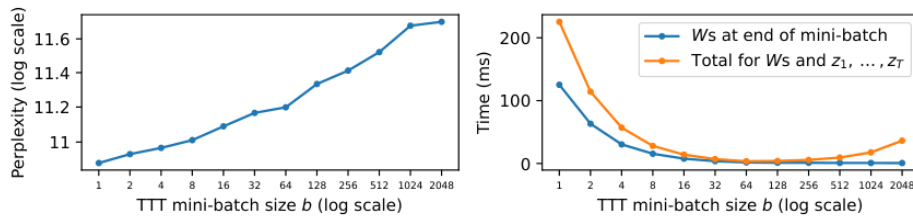


그림 1.5. mini-batch size b ablation — $b=1$ (online GD) 부터 $b=T$ 까지, b 가 병렬성과 적응 신호 선도 사이의 트레이드오프를 결정 (출처: TTT, Figure 7, [1])

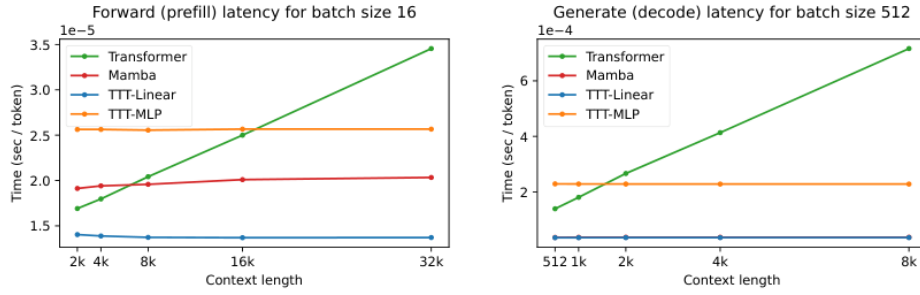


그림 1.6. A100 에서 TTT layer latency — dual form 과 mini-batching 으로 8k 에서 Transformer 보다 빠른 벽시계 시간, Mamba 와 유사한 latency 달성 (출처: TTT, Figure 12, [1])

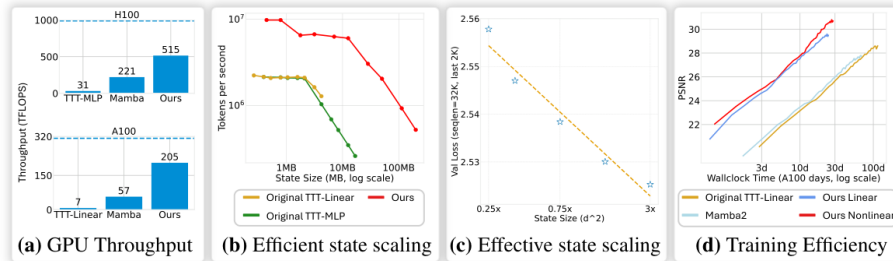


Figure 1: Using larger chunk sizes significantly improves GPU utilization compared to the original test-time training (TTT) method that even uses customized kernels (a). This enhanced utilization enables efficient and effective scaling to larger state sizes (b), (c), leading to better overall performance in less wall-clock time (d). The dotted line in (a) is the theoretical peak BF16 throughput of the GPU. Panel (c) measure average validation loss of the last 2K tokens in sequences processed by a LaCT language model across varying state sizes, demonstrating benefits of larger state size. Panel (d) compares performance versus training time across different baselines on the novel view synthesis benchmark. Further experimental details can be found in Sec. C.4.

그림 1.7. 작은 청크는 memory-bound 라 peak FLOPs 의 5% 미만, 큰청크 (2K~1M 토큰) 로 가면 compute-bound 가 되어 H100 에서 70% 이상으로 도약 (출처: LaCT, Figure 1, [6])

는출력이 정확히 동일하지만, dual form 은 TPU 에서 naive 구현보다 5 배 이상 빠르다 [1][a]. 이것이 chunkwise 병렬화의정수다: 청크 내부는 matmul(병렬), 청크 사이는 recurrent(순차).

이 세 얼굴 — 순차 갱신을 정의하는 **recurrent form**, 학습 · 이해를 위한 **parallel form**, 하드웨어를 위한 **chunkwise+dual form** — 이 등가라는 사실이 TTT(그리고 linear attention 전체)를 실용 가능하게 만든다. 그러나 등가성이 병목을 없애지는못한다.

순차 갱신이 모든 병목의 뿌리: LaCT 의 FLOPs util < 5%

여기서 이 장의 결론적 관찰이 나온다. 모든 병목의 뿌리는 “상태를순차적으로 갱신해야 한다” 는 것 자체다. 청크 크기 b 를 키우면 matmul 이 커져 하드웨어는 좋아지지만, 갱신이 뜸해져 상태가 낡는다. b 를 줄이면 상태는 신선하지만 TensorCore 가 굶는다. 「Test-Time Training Done Right」 (LaCT)[6] 는 이 긴장을 정량으로 못박았다: 기존 TTT 는 16~64 토큰마다 작은 갱신을 하도록 설계 되어 FLOPs utilization 이 흔히 5% 미만으로 떨어진다는 것이다 [a]. 즉 값비싼 A100/H100 이 이론 성능의 5%도 못 쓴다.

LaCT 의 처방은 개념적으로 단순하다 — 청크를 극단적으로크게 (2K 에서 1M 토큰까지) 잡아 fast-weight 갱신을 큰 matmul 로 만든다. 그러면 연산이 memory-bound 에서 compute-bound 로 넘어가 H100 이용률이 <5% → >70%로 뛴다 [6][a]. 게다가커스텀 커널 없이 수십 줄의 순수 PyTorch 로 구현되며, 비선형 상태 크기를모델 파라미터의 최대 40%까지 키워 상태 용량 자체를 확장할 수 있다. 이그림이 이 장 전체의 논지를 봉인한다: TTT 의 실용성 문제는 알고리즘의우아함이 아니라 순차 의존이 만든 하드웨어 굶주림이며, 해법은언제나 “얼마나 큰 덩어리로 묶어 순

차성을 감출 수 있는가”로 귀결된다. Titans·Atlas 같은 후속 라인이 체크·모멘텀·깊은 메모리로 씨름하는 것도 결국이 한 축 위의 다른 좌표들이다.

표: 체크 크기와 하드웨어 이용률 (LaCT, [6])

갱신 체크 크기	연산 특성	H100 FLOPs 이용률	상태 신선도
16~64 토큰 (기존 TTT)	memory-bound	< 5%	매우 높음
2K~1M 토큰 (LaCT)	compute-bound	> 70%	낮음 (체크 내 지연)

TTT $\approx$ linear attention 등가: 비판적시선

마지막으로 이 장은 반드시 회의적 각주 하나를 남겨야 한다. Liu 등의 「Test-Time Training with KV Binding Is Secretly Linear Attention」 [7] 은 TTT 를 “test-time 에 진짜로 학습하는 메타러너” 로 낭만화하는 서사에 제동을 건다. 저자들은 **multi-layer MLP fast weight 와 momentum 을 포함한 폭넓은 TTT 변형조차 하나의 ‘학습된 linear attention 연산자’ 로 등가 재작성될 수 있음**을 해석적으로 보인다 [7][a]. 이 관점에서 inner loop 는 통상적 의미의 meta-learning 을 하는 것이 아니라, query·key·value 벡터들의 구조화된, 히스토리 의존적 혼합 (mixing) 을 유도할 뿐이다.

실천적 함의는 이중적이다. 긍정적으로는, 이 등가성이 **완전 병렬형태**를 허용해 성능을 유지하면서 순차 병목을 줄이는 아키텍처 단순화를 준다 (ICML 2026 채택). 회의적으로는, “test-time learning” 이라는 프레이밍이 주는 신비감의 상당 부분이 실은 linear attention 의 재표기라는 뜻이다 — TTT 의 이득이 정말 “학습” 에서 오는지, 아니면 더 표현력 있는 KV 혼합에서 오는지는 열린 논쟁이다. 이 두 해석 ([1] 의 학습 서사 vs [7] 의 linear attention 환원) 을 억지로 봉합하지 말고, **같은 계산의 두 렌즈**로 병치해 두는 것이 정직하다. 어느 렌즈로 보든, 다음장들이 다룰 Titans·Miras·Atlas 가 싸우는 전장은 동일하다: 순차 갱신이 강제하는 하드웨어 굶주림을, 상태의 표현력을 희생하지 않고 어떻게 감출 것인가.

Sources: - arXiv:2407.04620 — Learning to (Learn at Test Time): RNNs with Expressive Hidden States - arXiv:2102.11174 — Linear Transformers Are Secretly Fast Weight Programmers - arXiv:2501.12352 — Test-time regression: a unifying framework - arXiv:2505.23884 — Test-Time Training Done Right (LaCT) - arXiv:2602.21204 — TTT with KV Binding Is Secretly Linear Attention - arXiv:2006.16236 — Transformers are RNNs - arXiv:2312.06635 — Gated Linear Attention Transformers with Hardware-Efficient Training

2. Titans 집중 조명과 그 효율전반

Titans 가 풀려던 병목: attention 의 quadratic 과 RNN 의 압축손실사이

Titans 의 출발점은 두 진영의 실패를 동시에 직시하는 데 있다. Transformer 의 self-attention 은 문맥 전체를 손실 없이 보존하지만 그대가로 시퀀스 길이 N 에 대해 $O(N^2)$ 의 연산·메모리를 지불한다. 반대로 선형 RNN·state-space 계열은 문맥을 고정크기 상태로 압축해 $O(N)$ 을 얻지만, 그 상태가 벡터/행렬 하나라는 점에서 표현 용량이 근본적으로 제한된다. Titans 의 제안은 이 둘 사이에 “test time 에 스스로 학습하는 신경망 메모리 (neural long-term memory)” 를 끼워 넣는 것이다 [8]. 핵심 전환은 메모리를 데이터로 담는 자료구조가 아니라 데이터를 파라미터에 새겨 넣도록 test time 에 갱신되는 하나의 모델로 보는 관점이다. 즉 추론 시점에도 내부 미니신경망이 gradient step 을 밟으며 문맥을 흡수한다 — 이것이 TTT 계열과 공유하는 “learning to memorize at test time” 의 골격이다.

long-term memory = deep MLP 와 ‘surprise’ 라는 갱신신호

Titans 의 장기 메모리 $\mathcal{M}$ 은 스칼라/행렬 상태가 아니라 여러 층의 MLP 다. 이 선택이 표현력의 원천이다. 입력 토큰 x_t 로부터 key $k_t = x_t W_K$, value $v_t = x_t W_V$ 를 만들고, 메모리가 “이 key 를 넣

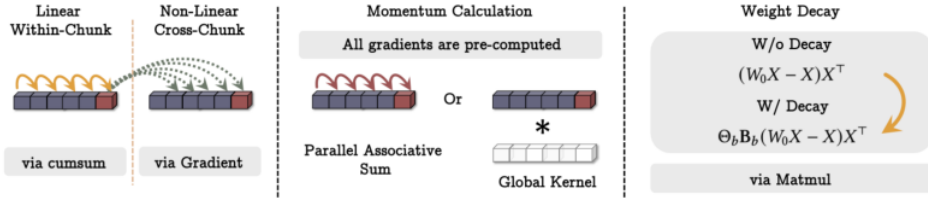


그림 2.1. neural memory 를 chunk 단위로 병렬 학습시키는 구조 — 순차 recurrence 를 청크 내부에서 행렬 연산으로 펼쳐 GPU 활용도를 끌어올린다. (출처: Titans, Figure 1, [8])

으면 value 가 나와야 한다” 는 associative memory 목표를 갖게 한다. 손실은 단순한 회귀 손실

$$\ell(\mathcal{M}_{t-1}; x_t) = \|\mathcal{M}_{t-1}(k_t) - v_t\|_2^2$$

이다. 여기서 결정적 아이디어는 이 손실의 메모리 파라미터에 대한 gradient $\nabla \ell$ 를 ‘surprise(놀람)’ 로 해석하는 것이다. 인간 기억이 “기대를 위반한 사건” 을 더 잘 각인하듯, 예측이 크게 빗나가 gradient 가 큰 토큰일수록 메모리를 크게 갱신한다 [8]. 즉 갱신량 자체가 데이터 내용에 따라 조절 되는 것이다. 이는 associative loss 를 test time 에 online 으로 최소화하는 것과 정확히 등가이며, deep MLP 를 메모리로 쓰는 순간 이 online 목표가 비선형 key-value 매핑까지 답을 수 있게 된다.

momentum = 과거 surprise 의 누적, weight-decay = adaptive forget

순간 gradient 만으로 갱신하면 두 가지 약점이 생긴다. 첫째, 놀라운 사건 직후의 이어지는 (그러나 중요한) 토큰을 놓친다. 둘째, 무한히 흘러드는 문맥에서 메모리가 포화한다. Titans 는 이를 최적화 언어로 정확히 대응시킨다.

$$S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(\mathcal{M}_{t-1}; x_t), \quad \mathcal{M}_t = (1 - \alpha_t) \mathcal{M}_{t-1} + S_t$$

여기서 S_t 는 surprise 의 momentum 향으로, η_t 가 과거 surprise 를 얼마나 이어받는지 조절한다 — “놀란 뒤에도 잠시 memorize 상태를 유지” 하는 past surprise 누적이다. α_t 는 weight-decay, 곧 adaptive forget gate 로, 메모리 크기와 들어오는 데이터 양에 비례해 오래된 내용을 지워 용량을 확보한다 [a]. $\eta_t, \theta_t, \alpha_t$ 가 모두 입력에 따라 달라지는 data-dependent 스칼라라는 점이 중요하다. 결과적으로 test time 메모리 갱신은 “momentum 과 weight decay 를 가진 mini-batch gradient descent 로 meta 신경망을 최적화하는 것” 과 등가가 된다 [8]. 이 등가성이 이후 Miras·Nested Learning 이 확장할 이론적 발판이 된다: forget gate = retention regularization, optimizer = memory 라는 재해석이 여기서 싹튼다 [9].

MAC / MAG / MAL: 메모리를 회로에 꽂는 세 방식

Titans 는 이 neural memory 를 backbone 에 통합하는 세 가지 아키텍처를 제시한다.

- **MAC (Memory as Context):** 장기 메모리에서 retrieve 한 내용을 현재 문맥에 토큰처럼 이어 붙여 attention 의 입력 문맥으로 쓴다. attention 이 “지금 메모리를 참조할지” 를 결정하고, 그 출력이 다시 메모리를 갱신한다. 세 변형 중 긴 문맥 추론에서 가장 강하다.
- **MAG (Memory as Gate):** sliding-window attention(단기) 과 neural memory(장기) 를 두 갈래로 두고 gating 으로 결합한다.
- **MAL (Memory as Layer):** 메모리를 attention 이전의 한 층으로 쌓아 압축을 먼저 수행한다.

이 분해가 중요한 이유는, “장기 기억을 어디에 꽂느냐” 가 성능을 좌우함을 명시적으로 보여주기 때문이다. 특히 MAC 은 attention 이 메모리 접근을 게이트링하므로, 관련 없는 과거를 무시하고

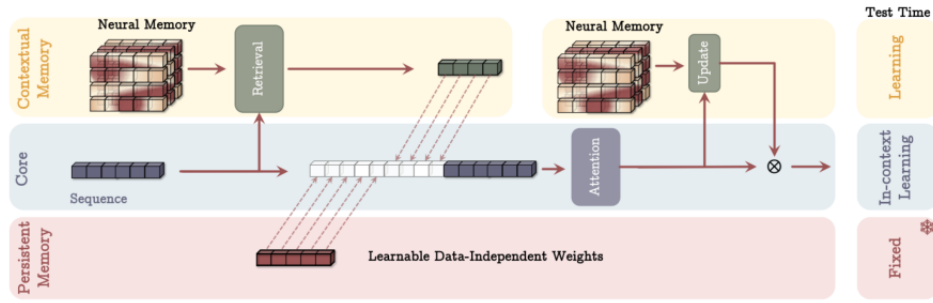


그림 2.2. Memory-as-Context(MAC) 구조 — persistent memory, 현재 segment, long-term memory 에서 retrieve 한 내용을 하나의 문맥으로 결합해 attention 에 넣는다. (출처: Titans, Figure 2, [8])

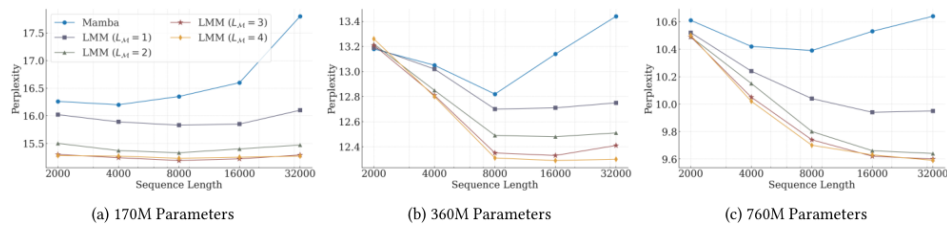


그림 2.3. memory depth 가 깊어질수록 (층수 $\uparrow$) perplexity 가 낮아지고 특히 긴 문맥에서 이득이 커진다 — deep MLP 메모리가 선형 상태의 용량 한계를 넘어서는 증거. (출처: Titans, Figure 7, [8])

필요한 순간에만 장기 기억을 끌어오는 행동을 학습한다 [b]. 다만 원논문의 소규모 언어모델링표 (Table 1) 에서는 변형 간 우열이 규모에 따라 뒤바뀐다 — 340M·760M에서는 오히려 MAG/MAL이 낮은 perplexity를 보이고, MAC의 우위는 주로 매우 긴 문맥·장문추론 과제에서 두드러진다 [a]. 즉 “MAC이 항상 최강”이 아니라 “긴문맥에서 MAC이 강하다”가 정확한 진술이다.

parallel-chunk 학습과 memory depth 의효과

test time 에 토큰마다 gradient step 을 순차로 밟는다면 학습은 느려 터진다. Titans 는 시퀀스를 chunk 로 나누고, chunk 내부의 순차 갱신을 행렬 연산으로 펼쳐 병렬화한다 (위 fig1). 이로써 recurrence 의 표현력을 유지하면서도 GPU 에서 batch matmul 로 처리 가능한 형태를 얻는다 — 다만 이 chunking 이뒤에서 볼 skeptic 논점의 진원지이기도 하다.

메모리를 깊게 쌓는 것이 실제로 이득인지도 이 논문이 실증한다. MLP 층수 L_M 을 $1 \rightarrow 2 \rightarrow 3$ 으로 늘리면 perplexity 가 단조 감소하며, 특히 긴 시퀀스에서 격차가 벌어진다. 이는 “선형 RNN = 1 층 선형메모리” 라는 통상적 구조가 왜 용량 한계를 갖는지, 그리고 깊은 메모리가 그벽을 어떻게 넘는지를 직접 보여준다.

표: Titans 언어모델링 · 긴 문맥 대표 수치 (원논문보고값)

항목	값	비교 baseline
WikiText perplexity (340M, LMM 변형)	26.18	Transformer++ 31.52
Needle-in-Haystack (S-NIAH, 16K) 정확도	98.4% (MAC)	길이 확장 시 attention 급락
유효 문맥 길이	2M+ tokens 까지 확장	attention 은 $O(N^2)$ 로 불가
BABILong 장문 추론 (fine-tune, MAC)	~93% > GPT-4 ~87% · Llama3.1-70B ~85%	파라미터 수십 배 큰 모델

Model	S-NIAH-PK				S-NIAH-N				S-NIAH-W			
	2K	4K	8K	16K	2K	4K	8K	16K	2K	4K	8K	16K
TTT	98.4	98.8	98.0	88.4	60.2	36.6	10.2	4.4	78.8	28.0	4.4	0.0
Mamba2	98.6	61.4	31.0	5.4	98.4	55.8	14.2	0.0	42.2	4.2	0.0	0.0
DeltaNet	96.8	98.8	98.6	71.4	47.2	15.4	12.8	5.4	46.2	20.0	1.6	0.0
Titans (LMM)	99.8	98.4	98.2	96.2	100.0	99.8	93.4	80.2	90.4	89.4	85.8	80.6
Titans (MAC)	99.2	98.8	99.0	98.4	99.6	98.2	97.6	97.4	98.2	98.2	95.6	95.2
Titans (MAG)	99.4	98.0	97.4	97.4	99.2	98.8	97.2	98.6	98.0	98.0	90.2	88.2
Titans (MAL)	98.8	98.6	98.8	97.8	99.8	98.1	96.8	96.4	98.0	97.4	92.0	90.4

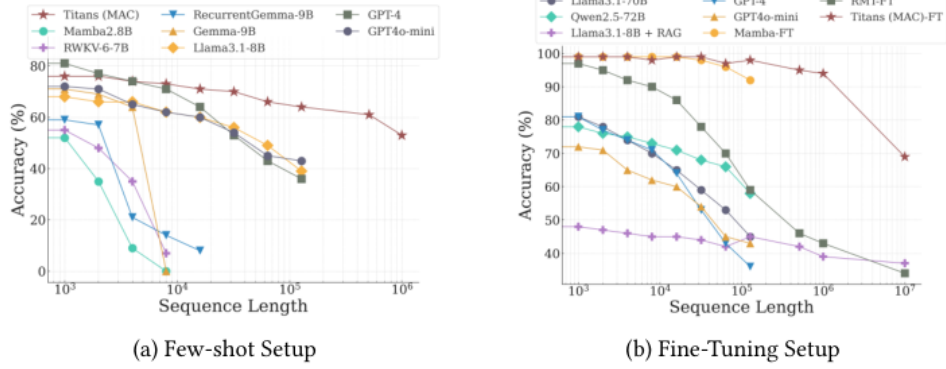


그림 2.4. BABILong 에서 Titans(MAC) 가 매우 큰 파라미터 모델을포함한 baseline 을 문맥 길이가 늘어날수록 앞서는 곡선 — 압축형 장기메모리가 초장문 추론에서 갖는 이점. (출처: Titans, Figure 6, [8])

주의: 표의 perplexity 는 340M 규모의 memory-only 변형 (LMM) 값이며, 같은규모에서 MAC=25.43, MAG=25.07, MAL=24.69 로 변형에 따라 더 낮아진다 — 즉 절대치는변형·규모에 민감하다. 또한 이들 수치는 원논문 보고값이고공식 코드가 오래 부재했다 — 재현성은 아래 skeptic 절에서 별도로 다룬다 [a].

효율·표현력 라인 (1): Atlas — Omega rule + Muon + 다항 feature

Titans 의 online 갱신은 여전히 “마지막 토큰 하나” 의 gradient 에 기반한근시안적 규칙이다. Atlas 는 이를 세 방향으로 밀어붙인다 [10]. 첫째, **Omega rule**: 매 step 에서 마지막 토큰이 아니라 sliding context window 전체에 대한 손실을 최소화하도록 메모리를 갱신한다 — 즉 “지금까지의 문맥을 최적으로 memorize” 하는 국소적 문맥 최적화로 전환한다. 둘째, **Muon optimizer**: 1 차 gradient 대신 Newton-Schulz 반복으로 2 차 정보를 근사해 메모리를 갱신, 수렴을 개선한다 — Atlas 는 저자들표현으로 “2 차 정보로 메모리를 최적화하는 최초의 병렬화 가능 recurrent 아키텍처” 다. 셋째, **polynomial feature mapping**: key 를 다항특징으로 사상해 메모리 용량 (선형 분리 가능한 연상 쌍의 수) 을 끌어올린다. 이 세 요소를 Transformer 일반화로 확장한 것이 **DeepTransformers** 와 **Dot** 이다. 결과적으로 Atlas 는 짧은문맥으로 학습하고도 BABILong 의 10M context 길이에서 견고하게 동작하며, 그 지점에서 Titans 대비 +80% 정확도를 보고한다.

효율·표현력 라인 (2): TNT —chunkwise training 의 딜레마를 정면돌파

Titans/TTT 계열의 아킬레스건은 학습 속도다. chunk 를 크게잡으면 병렬성이 좋아 빠르지만 정확도가 떨어지고, 작게 잡으면 정확하지만순차 의존이 길어져 느리다 — 속도와 정확도가 정면충돌한다. TNT 는 이를 2 단계로 분리한다 [11]. **1 단계 (pre-training)**: 계층적 메모리를쓴다 — 큰 하드웨어 친화적 chunk 를 처리하는 global 모듈이 장거리문맥을, 여러 개의 병렬 local 모듈이 세부를 담당하되, local 상태를주기적으로 리셋해 순차 의존을 끊어 대규모 context 병렬화를 가능케 한다. **2 단계 (fine-tuning)**: local 모듈만 작은 고해상도 chunk 로 짧게재적응시켜 정확도를 회복

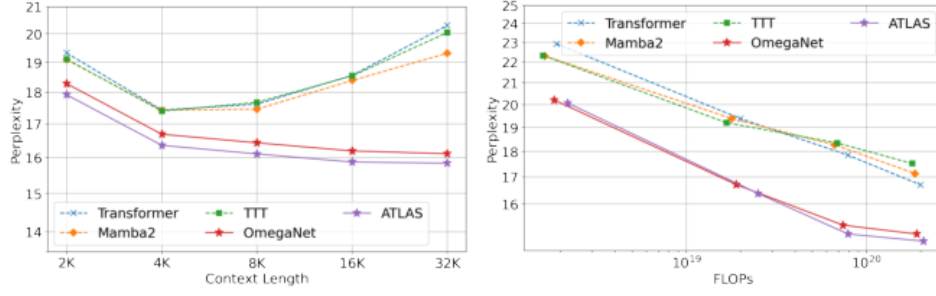


그림 2.5. Atlas 의 scaling pattern — 모델 · 문맥규모에 따른 성능 곡선이 recurrent/Transformer baseline 위에 위치, Omega rule 과 deep memory 가 유효 문맥 길이를 늘림을 보여준다. (출처: Atlas, Figure 8, [10])

TNT	JAX	{8}	2.54	7.68×
TNT	JAX	{16}	1.65	11.78×
TNT	JAX	{32}	1.22	15.92×
TNT	JAX	{64}	1.12	17.37×
TNT	JAX	{128}	1.16	16.75×

ences. As sequence length increases, TNT's superior scalability creates a crossover point where it becomes faster. This efficiency is most pronounced at very long contexts; for instance, at a sequence length of 32K, a implementation of TNT ($C_L = 128$) outperforms even the highly optimized FlashAttention kernel, confirming it as a viable alternative for demanding long-context training scenarios.

Quality Comparison. We next translate the speedup gains into a practical time-to-solve. As shown in Table 1, our TNT framework accelerates the total training time required to reach target model quality. Our best configuration is up to **17.4× faster** than the original Titans. This efficiency gain is fundamental to the success of TNT; for instance, using an identical local memory budget of 8, TNT is already **7.7× faster** than its baseline counterpart. While competitive with standard Transformers in JAX, our implementation does not perform highly optimized baselines like the Transformer with FlashAttention (Dao, Fu, et al., 2022) as an expected result, as TNT currently lacks a kernel, which we leave for future work. Nonetheless, these results establish TNT as an efficient foundation.

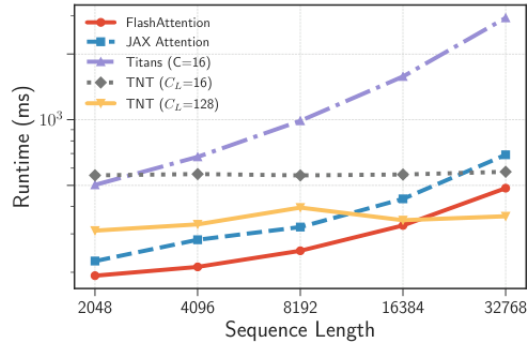


그림 2.6. TNT 의 runtime 비교 — 계층적 chunk 전략이 동일 정확도지점에서 순수 소형-chunk 대비 학습 시간을 큰 폭으로 줄임을 보여주는 곡선. (출처: TNT, Figure 4, [11])

한다. 보고된 결과는 가장 정확한 baseline 설정 대비 최대 **17× 학습 가속**이면서 정확도까지 개선되었다.

표현력 라인 (3): HOPE / Nested Learning — optimizer 가 곧 memory

Nested Learning 은 Titans 라인의 이론적 정점에 해당한다 [12]. 세 가지 재해석이 축이다. (i) **Deep/Expressive Optimizers**: SGD-momentum-Adam 같은 optimizer 자체가 gradient descent 로 압축하는 associative memory 모듈이라는 관찰 — 앞서 Titans 의 momentum 항이 사실 하나의 메모리였다는 사실의 일반화다. (ii) **Continuum Memory System (CMS)**: 단기/장기의 이분법을 버리고, 서로 다른 주파수로 갱신되는 메모리 모듈의 스펙트럼으로 대체한다. 어떤 파라미터는 매 예제마다 (fast), 어떤것은 아주 드물게 (slow) 갱신된다. (iii) **Self-Modifying**: 자기 자신의 갱신 규칙 (update algorithm) 을 학습하는 시퀀스 모델. 이 셋을 결합한 구현이 HOPE 로, Titans 의 장기 메모리를 무한중첩 (unbounded nested) 학습 레벨과 CMS 로 확장한 것이며, 언어모델링 · 상식추론에서 현대 recurrent 모델과 표준 Transformer 대비 더 낮은 perplexity 와 더 높은 정확도를 보고한다.

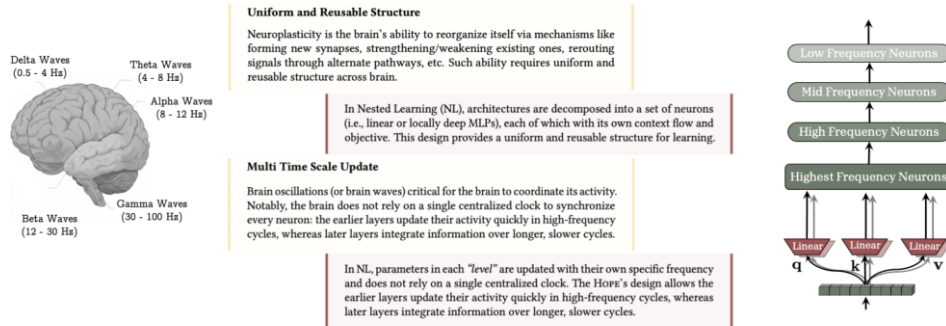


그림 2.7. HOPE의 multi-time-scale 갱신 — 파라미터 블록마다 서로다른 주파수로 메모리를 갱신하는 continuum memory 구조가 in-context 와장기 지식을 분리해 담는다. (출처: HOPE, Figure 1, [12])

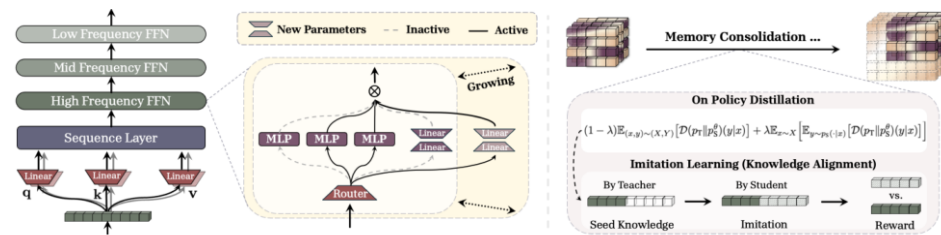


그림 2.8. Sleep의 consolidation — 작은 자기의 메모리를 더 큰 네트워크로 상향 distill 해 모델이 스스로 크기를 늘리며 지식을 보존·확장하는 과정. (출처: Sleep, Figure 2, [13])

표현력 라인 (4): Sleep — 모델이 스스로 커지는 offline consolidation

Sleep은 “test time에 쌓인 불안정한 단기 메모리를 어떻게 안정적 장기지식으로 굳힐 것인가”를 다룬다 [13]. 생물학의 수면 중 memory consolidation에 착안해, 온라인 추론과 별개의 offline 단계를 둔다. 두축이 핵심이다. **Memory Consolidation via Knowledge Seeding**: 작은 자기 자신 (smaller-self)의 메모리를 더 큰 네트워크로 상향 distillation하고 replay로 재생하여, 불안정한 단기 기억을 보존하면서 용량을 늘린다 — 즉 모델이 필요에 따라 스스로 크기를 키운다. **Dreaming**: RL로 합성 데이터 curriculum을 생성해 새 지식을 rehearse하고 기존 능력을 재정립하는 자기개선 단계로, 사람 감독 없이 진행된다. 이 offline 시간축의 다단계 갱신은 CMS의 다 주파수 계층 사상을 온라인이 아닌 수면 단계에서 구동하는 셈이다.

skeptic: Titans Revisited — chunking이 baseline을 못넘기게 할때

이 라인 전체를 냉정하게 붙드는 것이 Titans Revisited다 [14]. 이 논문은 공식 코드의 오랜 부재와 원문의 모호한 설계 서술이 재현을 가로막았음을 지적하고, 명시되지 않은 설계 선택을 드러낸 경량 재구현을 제시하며 Masked Language Modeling·Time Series Forecasting·Recommendation 과제에서 평가한다. 핵심 발견은 두 갈래다. 첫째, **chunking 탓에 Titans가 항상 baseline을 이기지 못한다** — 병렬화를 위한 chunk 근사가 online 갱신의 이점을 깎아먹는 조건이 존재한다. 둘째, 그럼에도 **neural memory 컴포넌트 자체는 attention-only 모델 대비 일관되게 성능을 올린다**. 즉 “test time에 학습하는 메모리”라는 아이디어의 핵심 가치는 살아남되, 원논문이 보고한 화려한 절대 수치는 구현·chunk 설정에 민감하다는 것이 정직한 결론이다 [a]. 이는 앞서 표의 수치들을 “메커니즘의 방향성 증거”로 신뢰하되 “재현 가능한 절대 SOTA”로 과신하지 말라는 경고로 읽어야 한다. 다만 이 재구현의 평가도메인 (MLM·시계열·추천)이 원논문의 자가회귀언어모델링·초장문 추론과 다르다는 점은 감안해야 한다 [b]. TNT가 chunk 크기와 정확도의 trade-off를 정면으로 공학화한 것 역시, 바로 이 chunking 병목이 실재함을 우회적으로 확인해 준다 [11].

종합하면 Titans는 (deep MLP 메모리 × surprise-gradient × momentum·decay)라는 하나의 등가식을 세우고, Miras가 이를 4축 (연상 메모리 구조·attentional bias·retention gate·

학습 알고리즘) 설계공간으로 [9], Atlas 가 갱신 규칙의 고도화로, TNT 가 학습 효율로, Nested Learning·Sleep 이 시간척도와 자기수정으로 확장한계보를 이룬다. 병목은 언제나 “표현력을 유지한 online 갱신을 어떻게병렬로 싸게 돌리느냐” 였고, 각 논문은 그 축의 한 변을 민 셈이다.

Sources: - Titans (2501.00663) - Atlas (2505.23735) - TNT (2511.07343) - Nested Learning / HOPE (2512.24695) - Sleep (2606.03979) - Miras (2504.13173) - Titans Revisited (2510.09551)

Part I 정리. TTT/Titans 의 표현력은 ‘추론 중 self-supervised gradient 로 fast weight 를 갱신’ 하는 데서 오고, 바로 **그순차 갱신**이 이후 세 병목의 공통 뿌리다. LaCT 가 정량화한 FLOPs util<5%가 그 증거다.

Part II — Scaling 병목 3 종

3. 병목 ① — model-size scaling

문맥은 파라미터가 아니라 ‘메모리 용량’ 예산다

TTT/Titans 계열을 하나의 관점으로 압축하면 이렇게 된다. Transformer 는 과거 문맥을 KV 캐시라는 **외부 저장소**에 통째로 쌓아두고 (길이에 비례해 자라는 associative memory), 질의 시점에 query-key 유사도로 그 저장소를 통째로 뒤진다. 반면 TTT/Titans/ATLAS 는 과거 문맥을 **고정크기의 내부 상태** (가중치 행렬 혹은 작은 MLP) 에 압축해 써 넣는다. Titans 는 이 내부 상태를 “test-time 에 학습되는 neural long-term memory” 로부르고, 무엇을 쓸지는 associative memory loss 의 gradient 가 만드는 ‘surprise’ 로, 무엇을 지울지는 decay(forget) 게이트로 결정한다 [8]. 즉 이들에게 “긴 문맥을 기억한다” 는 것은 파라미터를 더 얹는 문제가 아니라, **정해진 용량의 메모리에 얼마나 많은 연상 (association) 을 간섭 없이 새겨 넣느냐**의 문제다.

이 재정의가 이 장의 첫 번째 병목을 낳는다. 모델 크기 (d , 파라미터 수) 를 키우면 표현력은 늘지만, 문맥을 담는 ‘메모리 용량’ 은 파라미터 수에 비례해늘지 않는다. 왜 그런지를 associative memory 의 용량 이론에서 출발해정리한다.

용량 이론: d^2 파라미터에 왜 $\sim d$ 규모의정보만 안정저장되는가

가장 단순한 선형 연상 메모리는 key-value 외적의 합 $M = \sum_i v_i k_i^\top$ (correlation matrix memory) 이다. 파라미터자유도는 $d \times d = d^2$ 개다. 그런데 이 d^2 개의 슬롯에 실제로**간섭 없이** 넣을 수 있는 연상의 수는 d^2 이 아니라 d 규모다. 이유는 crosstalk 다. 질의 k_j 를 넣어 $Mk_j = v_j (k_j^\top k_j) + \sum_{i \neq j} v_i (k_i^\top k_j)$ 를 읽을 때, 두 번째 항이 잡음이 된다. 이 잡음이 사라지려면 key 들이 서로 직교해야하는데, d 차원 공간에는 직교 벡터가 최대 d 개뿐이다. 그 이상을넣으면 검색 결과가 여러 value 의 선형결합으로 뭉개진다. 이것이 “ d^2 파라미터 메모리에 $\sim d$ 규모 정보만 안정 저장” 이라는 경험칙의뿌리다 — 다만 이 $\sim d$ 는 간섭을 완전히 0 으로 요구하는직교-key 극단의 하한임을 유념하자.

여기서 반드시 붙여야 할 skeptic 주석: 용량의 정확한 상한은 **검색기준 (retrieval criterion) 에 따라 달라진다**. 최근 연구는 이를 날카로운상전이 (sharp phase transition) 로 정리한다 — 모든 신호가 자기 최대 distractor 를 이겨야 하는 top-1(winner-take-all) 기준에서는 isotropic Gaussian 모델 가정 하에 $d^2 \asymp n \log n$ 이 필요하고 (즉 저장 가능한연상 수 $n \sim d^2 / \log d$), 이 log 인자는 극단값통계 (extreme-value) 가 부과하는 winner-take-all 디코딩의 본질적비용이라는 것이 핵심이다 [15][a]. 반면 정답이 유일한 최고점일필요 없이 상위 후보군에만 들면 되는 listwise 기준 (저자들이 제안한 Tail-Average Margin, TAM 으로 형식화) 에서는 $d^2 \approx n$, 즉 $n \sim d^2$ 까지 늘어난다 [15][a]. 그러므로 “용량 = d ” 는간섭 0 을 요구하는 최엄격 하한이고, 완료된 기준에서는 d^2 에근접할 수도 있다. 요점은 스케일이 아니라 **선형 용량이 파라미터수보다 느리게 (정확 검색에 선 log 인자를 낀 채) 자란다**는 점, 그리고정확 검색을 요구할수록 이 벽이 낮아진다는 점이다.

표: 선형 연상 메모리의 저장 가능 연상 수 (파라미터 자유도 d^2 , 이론적 스케일)

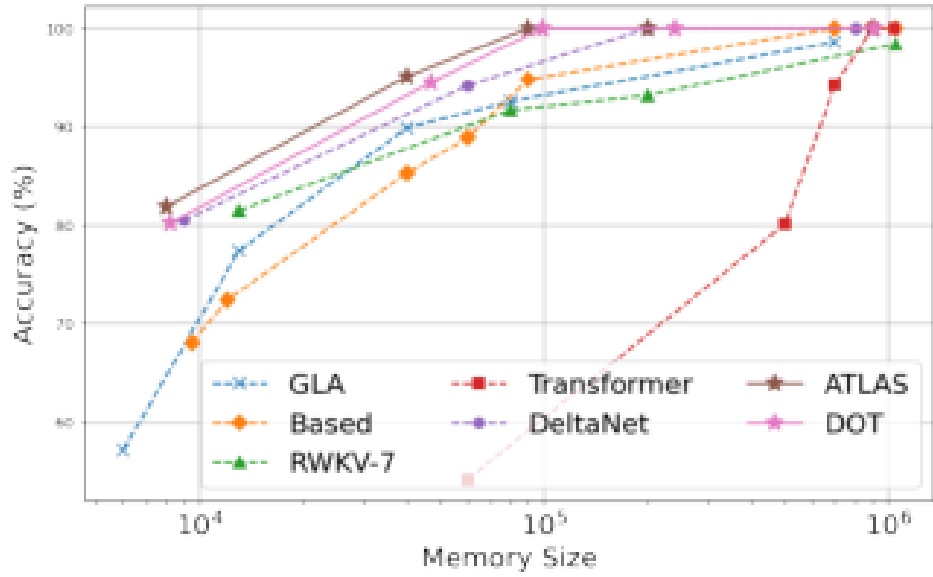


그림 3.1. ATLAS의 associative memory recall 실험: 저장한 key-value 쌍의 수가 상태 용량을 넘어서면 recall 정확도가 급락하는 모습 —고정 용량 메모리의 선형 벽을 실측으로 보여준다 (출처: Atlas, Figure 7, [10])

검색 기준	필요 파라미터 스케일	저장 가능 연상 수 n	출처
직교 key(간섭 0)	—	$\leq d$	crosstalk 논증
top-1 (winner-take-all)	$d^2 \asymp n \log n$	$\sim d^2 / \log d$	[15]
listwise (Tail-Average Margin)	$d^2 \approx n$	$\sim d^2$	[15]
고전 Hopfield(이진, Hebbian)	—	$\approx 0.14 N$	고전 결과

고전 Hopfield 가 $\approx 0.14N$ 에서 spurious attractor 에 무너지는 결과는 이 선형 벽의 원형이다. 그리고 이 벽을 **비선형으로 부수**는 길이 존재한다는 것이 modern Hopfield 의 핵심 기여다: log-sum-exp(=softmax attention) 에너지를 쓰면 저장 용량이 차원 (패턴 차원) 에 대해 **지수적으로** 커지고, 한 번의 업데이트로 지수적으로 작은 오차로 검색된다 [16][a]. 이 사실이 이 장 전체의 긴장을 만든다 — softmax attention 은 용량이 지수적이지만 비용도 $O(L^2)$ 로 폭발하고, TTT/ Titans 의 고정 상태는 비용이 $O(L)$ 로 싸지만 용량이다시 선형 벽에 갇힌다. **용량과 비용은 같은 축의 양 끝**이다.

위 그림 (ATLAS) 이 이 이론을 실험으로 확인해 준다. 저장해야 할 연상의 수가 상태 용량 안에 있을 때는 recall 이 거의 완벽하다가, 그 수가 상태가 담을 수 있는 한계를 넘어서는 순간 정확도가 무너진다. 이것이 중요한 이유는, 용량 한계가 ‘점진적 열화’ 가 아니라 **문턱을 넘으면 급락하는 상전이**에 가깝다는 것을 보여주기 때문이다 — 즉 문턱이 길어지면 recurrent state 는 “조금씩 잊는” 게 아니라 어느 지점에서 “무너진다” .

고정 용량 recurrent state 의 정보이론적벽

이 상전이는 정보이론으로도 자명하다. 상태 S 가 s 비트를 담을 수 있다면, s 비트를 넘는 과거 정보를 무손실로 실어 나를 수 없다 (비둘기집원리). 길이 L 의 문맥에서 임의의 위치를 정확히 회상해

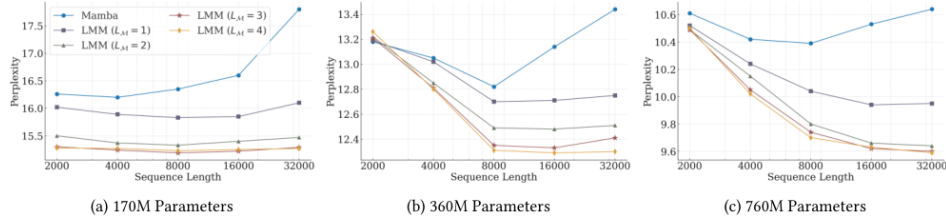


그림 3.2. memory module의 깊이 (1 → 4층 MLP)를 늘릴수록 더 긴 시퀀스에서 perplexity가 더 잘 낮아지는 (스케일이 개선되는) 관계 — 원문캡션 “Deeper long-term memory results in better scaling in longer sequences.” 선형 벱을 비선형 깊이로 밀어 올리는 경로 (출처: Titans, Figure 7, [8])

야 하는 과제 (예: needle-in-a-haystack, 정밀 copy/associative recall)의 요구정보량은 L 과 함께 자라는데, 고정 크기 S 는 자라지 않는다. 그래서 L 이 상태 용량을 넘어서면 recurrent 모델은 원리적으로 정확 회상을 보장할 수 없다. Titans가 attention window(단기 기억)와 neural memory(장기 기억)를 함께 두는 이유가 여기 있다 — 순수 recurrent state 하나로는 이 벱을 못 넘기 때문에, 최근 토큰은 무손실 window로 정확히 붙잡고 먼 과거만 압축 상태에 위임한다 [8].

chunking 자체가 병목: 규모에서 드러나는 열화 (Titans Revisited)

이 병목이 실무에서 조용히 새어 나오는 통로가 chunk 크기다. Titans 류는 test-time gradient 갱신을 토큰마다 하면 느리므로, 문맥을 chunk로 잘라 chunk 단위로 병렬 갱신한다. 여기서 근본 충돌이 생긴다 — 큰 chunk는 병렬성·속도를 얻지만 chunk 내부는 사실상 attention 처럼 처리되어 **메모리 갱신이 성겨지고**, 작은 chunk는 cross-segment 의존을 메모리에 더 의지하게 만들어 표현이 정밀해지지만 비용이 커진다.

독립 재현 연구 Titans Revisited는 공개 코드 부재와 서술 모호성 때문에 재현이 어려웠음을 지적하며, 세 가지를 실측으로 확인했다: (1) Neural Memory 요소 자체는 attention-only 대비 **일 관되게** 성능을 올린다, (2) 그러나 **chunking 때문에 Titans가 항상 기존 baseline을 이기지**는 않는다 — 저자들은 chunking을 성능 열화의 주된 원인으로 지목한다, (3) 큰 chunk가 성능은 좋지만 계산비가 함께 오른다 [14][a]. 즉 chunking은 병렬화를 위해 지불하는 정보 손실이고, Neural Memory는 그 손실을 일부 보전 (mitigate) 하는 장치다.

이것이 model-size scaling 병목의 실무 얼굴이다 — 표현력 (파라미터)을 키운다고 해서 chunk 병렬화가 강제하는 정보 손실이 자동으로 사라지지 않으며, 문맥 기억이 파라미터에 비례해 좋아지지도 않는다. 나아가 작은 규모에서 튜닝한 chunk 설정이 더 긴 문맥·더 큰 모델로 그대로 최적일 것이라는 보장도 없다 — 다만 이 마지막 명제는 Titans Revisited가 직접 실험으로 보인 바는 아니고 (연구가 소규모에 국한됨) 위 (1) – (3)에서 끌어낸 추론이다 [b].

skeptic 주석: Titans Revisited는 chunk 효과를 **표가 아니라 Figure 1(b)의 곡선 (가로축 chunk size, 세로축 F1-Score)으로만** 제시했고, 실험 규모가 작다 (시퀀스 길이 128–512, chunk 크기 32–128, 과제는 MLM·시계열 예측·추천). 정밀 수치 표는 주지 않았다. 또한 Mamba/DeltaNet 같은 대표 linear-RNN과 직접 비교하지 않고 BERT 류·BERT4Rec·iTransformer·LSTM과 비교했다. 따라서 “chunking이 열화를 부른다”는 방향성은 신뢰하되, 그 크기를 정량으로 확정하거나 대규모 LLM으로 일반화하기엔 근거가 얇다 [b].

depth·expressivity로 용량을 밀어올리기 — 그리고 그대가

선형 벱을 우회하는 성공법은 메모리를 **비선형·깊게** 만드는 것이다. Titans는 memory module을 단순 행렬이 아니라 다층 MLP로 두면 긴 시퀀스에서 더 잘 스케일한다는 것을 depth 실험으로 보인다.

이 그림이 중요한 이유는, 문맥 기억을 늘리는 지렛대가 ‘**파라미터 총량**’이 아니라 ‘**메모리의 표현력 (깊이/비선형성)**’임을 명시하기 때문이다. Titans는 170M·360M·760M 세 규모에서 메모리 깊이 1–4를 비교하는데, 같은 파라미터라도 선형 (깊이 1) 상태로 쓰면 $\sim d$ 벱에 갇히지만,

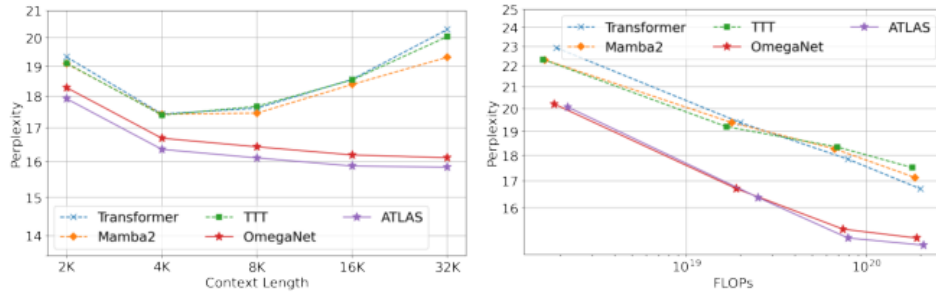


그림 3.3. ATLAS의 scaling 패턴: 모델 크기 · 문맥길이를 키울 때 baseline 대비 성능이 어떻게 갈라지는지 — 표현력을 키운메모리가 긴 문맥에서 이득을 키우는 구간을 보여준다 (출처: Atlas, Figure 8, [10])

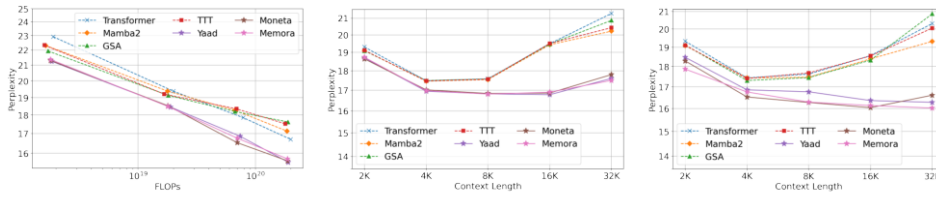


그림 3.4. Miras의 scaling 패턴: model size와 sequence length 두축을 함께 변화시키며 측정 — 용량 이득이 ‘문맥 길이’ 축에서 특히두드러짐을 보인다 (출처: Miras, Figure 3, [9])

비선형 깊은 상태로 쓰면 더 많은연상을 분리해 담아 긴 시퀀스일수록 perplexity 이득이 커진다 (perplexity 는유효 용량의 대리 지표다). ATLAS는 여기서 한 걸음 더 나가 (1) 마지막입력만 최적화하는 online 갱신의 근시안, (2) 고정 크기 메모리의 미숙한관리를 지목하고, **sliding window** 대신 **Omega rule**로 문맥 (윈도우 내모든 과거 토큰)을 통째로 최적 기억하도록 상태 관리를 개선한다 [10].

Miras는 이 흐름을 하나의 프레임으로 통합한다 — 어떤 sequence 모델이든 (i) associative memory 구조, (ii) attentional bias(내부 손실 목적), (iii) retention gate(과거 상태에 붙잡아 두는 정규화, 즉 forget 의재해석), (iv) 메모리 학습 알고리즘의 네 선택으로 기술되고, 그 조합으로 Moneta/Yaad/Memora라는 세 pure-recurrent(attention-free) 변형을 만든다 [9]. 핵심 관찰은문맥 길이를 늘릴 때 표현력 높은 메모리 구조를 가진 변형이 Transformer++·linear-RNN·hybrid baseline과의격차를 벌린다는 것이다.

이 그림들이 하나로 말하는 바: 문맥 기억의 개선은 **model-size** 축보다 ‘메모리 표현력 × 문맥 길이’ 축에서 나온다. 단, 대가가 있다. 비선형 · 깊은 메모리는 test-time 갱신 비용을 키우고, chunk 병렬화를 어렵게 하며 (앞의 chunking 문제와 직결), Muon 같은 옵티마이저가 heavy-tailed 코퍼스의 tail-end 연상 학습에서 Adam을 앞선다는 최근결과처럼 **어떻게 채우느냐 (optimizer/갱신 규칙)**까지성능을 좌우한다 — 그 이유가 Muon의 갱신 규칙이 선형 연상 메모리의 외적구조와 정렬되기 때문이라는 분석이다 [17][a]. 표현력을 키운 만큼효율이 깎이는 것이 이 병목의 본질이다.

hybrid attention 비율의존

그래서 실전 해법은 대개 **순수 recurrent가 아니라 hybrid**다. 소수의 full-attention 층으로 정확 회상 (용량 지수적)을 보장하고, 다수의층을 선형/상태 기반으로 바꿔 비용을 $O(L)$ 로 낮춘다. Nemotron-H는 self-attention 대부분을 per-token 상수 비용의 Mamba 층으로대체한 8B·56B(+56B를 MiniPuzzle로 압축한 47B) hybrid로, 유사 규모 Transformer(Qwen-2.5, Llama-3.1) 대비 정확도는 동등 이상이면서추론이 **최대 3배** 빠르다고 보고한다 [18]. 특히 긴 문맥 (입력 65536 토큰)에서 47B는 Qwen-2.5-72B·Llama-3.1-70B 대비 약 $2.9\times$ 빠르다.

표: hybrid(Mamba-Transformer)의 정확도-효율 tradeoff (Nemotron-H, [18])

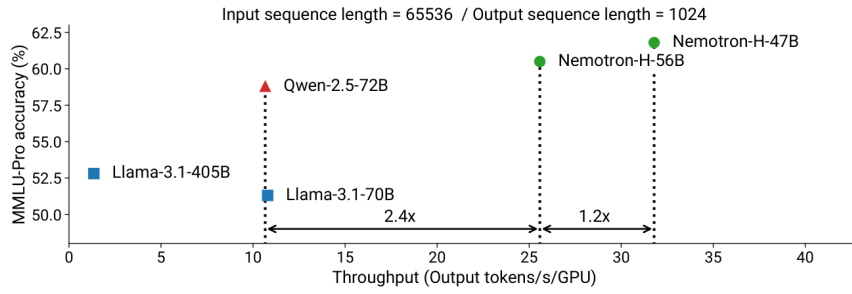


Figure 1 | MMLU-Pro accuracy (Wang et al., 2024a) versus per-GPU inference throughput for Nemotron-H-56B/47B-Base compared to existing similarly-sized Transformer models. Nemotron-H models provide state-of-the-art accuracy and inference time speedups.

그림 3.5. Nemotron-H: MMLU-Pro 정확도 대 per-GPU 추론 throughput — hybrid가 Transformer baseline 대비 같은 정확도에서 더 높은 throughput(우상단)으로 이동함을 보여준다 (출처: Nemotron-H, Figure 1, [18])

모델	비교 대상	정확도 (MMLU-Pro 등)	추론 속도
Nemotron-H 8B/56B	Qwen-2.5 / Llama-3.1 유사규모	동등 이상	최대 3×
Nemotron-H-47B (긴 문맥, 65536 입력 토큰)	Qwen-2.5-72B / Llama-3.1-70B	동등 이상	약 2.9×
Nemotron-H-47B (MiniPuzzle 압축)	Nemotron-H-56B	유사 (comparable)	20% 빠름

여기서 병목이 다시 얼굴을 바꾼다: **정확 회상 성능은 남겨 둔 attention 비율에 의존한다.** attention 층을 너무 줄이면 지수적 용량을 잃어 recall이 무너지고, 너무 남기면 $O(L^2)$ 비용이 돌아온다. hybrid 비율은 이 장 전체의 tradeoff를 하나의 하이퍼파라미터로 응축한 knob이다 — 그리고 그 최적값이 model size·문맥 길이에 따라 달라진다는 점이 바로 “단순 파라미터 확대가 통하지 않는” 이유의 실무적 표현이다.

종합: 왜 파라미터 확대가 문맥 기억을 비례해 늘리지 못하는가

세 갈래를 봉합하지 말고 나란히 두자. (1) **용량 이론**: 선형 상태는 d^2 파라미터에도 $\sim d$ (간섭 0) $\sim d^2/\log d$ (top-1 정확 검색) $\sim d^2$ (listwise 완화 검색) 규모의 연상만 안정 저장한다 [15]. 지수적 용량은 softmax(비선형) 에너지에서만 나오고, 그 대가가 attention의 $O(L^2)$ 다 [16]. (2) **정보 이론 벽**: 고정 상태는 L 이용량을 넘으면 원리적으로 정확 회상을 보장 못 하며, 실측에서 상전이형 급락으로 나타난다 [10]. (3) **규모에서의 열화**: chunking 이부과하는 정보 손실 탓에 Titans 류가 항상 baseline을 이기지 못하고 [14], 개선은 model-size가 아니라 ‘메모리 표현력 × 문맥 길이’ 축에서 온다 [9].

결론적으로 파라미터 d 를 키우면 선형 용량은 기껏해야 d (엄격) $\sim d^2/\log d$ (top-1) 규모로 자랄 뿐, 실제로 필요한 문맥 정보량 $\sim L$ 을 따라잡지 못한다. 문맥 기억을 늘리는 진짜 지렛대는 세 가지 — 메모리의 **비선형 깊이** (용량 벽 우회), **hybrid attention 비율** (지수 용량의 국소 보강), **chunk/갱신 설계** (표현력을 효율로 깎아먹지 않기) — 이고, 이들은 모두 표현력 ↔ 효율 tradeoff의 서로 다른 절단면이다. TTT/Titans 계열의 model-size scaling 병목은 “메모리를 더 키우면 되지 않나”가 아니라 “같은 용량을 어떤 표현력 · 어떤 비율 · 어떤 갱신으로 채우느냐” 의문제이며, 그 답이 규모에 따라 이동한다는 데 있다.

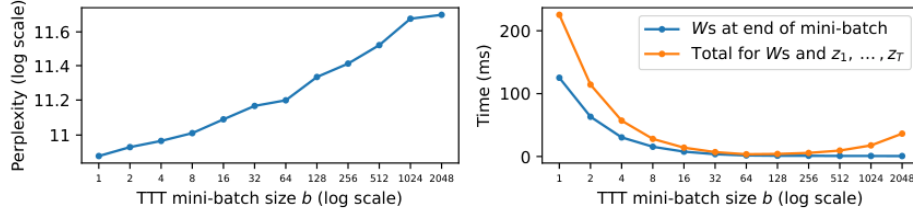


그림 4.1. mini-batch 크기 b 에 따른 TTT 성능: b 가 작을수록 갱신이 세밀해 표현력 (perplexity)은 좋아지지만 하드웨어 효율은 급락한다. (출처: TTT, Figure 7, [1])

4. 병목 ② — training parallel/batch scaling

inner-loop의 순차 의존성: 왜 sequence-parallel 이무너지는가

TTT·Titans 계열의 정의적 특징은 hidden state가 더 이상 고정된 벡터가 아니라 시퀀스를 따라 학습되는 fast weight W_t 라는 점이다. 각 토큰에서 fast weight는 self-supervised loss $\ell(W_{t-1}; x_t)$ 의 gradient로 갱신된다 $-W_t = W_{t-1} - \eta \nabla \ell(W_{t-1}; x_t)$ [1]. 이 한 줄이 학습 스케일링의 병목을 전부 결정한다. 표준 Transformer의 attention은 QK^T 한 번의 거대한 matmul로 전 토큰을 동시에 처리할 수 있다 (시퀀스 축이 완전 병렬). 반면 fast-weight 갱신은 W_t 가 W_{t-1} 에 함수적으로 의존하는 **first-order nonlinear recurrence**다. W_{t-1} 을 계산하기 전에는 W_t 를 계산할 수 없다. 즉 시퀀스 축이 데이터-의존 사슬로 묶여, GPU가 가장 잘 하는 일 — 축을 펼쳐 하나의 큰 tensor-core matmul로 던지는 것 — 이 원천적으로 막힌다.

이것이 “sequence-parallel 난점”의 본질이다. Attention이 $O(T^2)$ FLOPs를 쓰면서도 빠른 이유는 그 T^2 이 전부 병렬 matmul이기 때문이고, TTT가 $O(T)$ FLOPs를 쓰면서도 느린 이유는 그 T 가 순차 사슬이기 때문이다. FLOPs 수가 아니라 FLOPs의 배치 가능성이 벽이다. TTT 원논문이 명확히 인정하듯, naive online update(토큰마다 갱신)는 tensor core가 요구하는 matmul 형태를 거의 만들지 못해, 갱신을 dual form으로 matmul화하기 전에는 하드웨어를 놀리는 “few-matmul” 연산만 남고, 이는 wall-clock 효율에 치명적이다 [a].

chunk 크기 트레이드오프: 세밀함 대 활용률

순차 사슬을 우회하는 표준 처방은 **chunk(mini-batch) 병렬화**다. 시퀀스를 크기 b 의 청크로 자르고, 청크 내부에서는 fast weight를 상수 W 로 고정한 채 gradient를 계산해 한 번에 반영한다. $b = 1$ 이면 순수 online GD(가장 세밀하나 순차), $b = T$ 이면 batch GD(완전 병렬하나 청크 내 적응 없음)로, b 는 표현력과 효율 사이의 노브가 된다 [1]. TTT-Linear는 mini-batch $b = 16$ 을 기본값으로 택했는데 (청크당 fast weight를 한 번 갱신), 이는 타협의 산물이지 최적점이 아니다.

문제는 이 노브의 스윙이 얼마나 가혹한가이다. LaCT는 이를 정량화한다. TTT 계열이 통상 쓰는 작은 청크 ($b = 16 \sim 64$)에서는 GPU의 peak FLOPs 대비 실측 활용률이 **5% 미만**에 머문다 [6] [a]. 이유는 이중적이다. (1) 각 청크의 갱신 연산이 작아 memory-bound가 되어 tensor core가 놀고, (2) 청크마다 fast weight를 읽고 쓰는 상태 왕복이 산술강도 (arithmetic intensity)를 떨어뜨린다. LaCT의 핵심 통찰은 방향을 정반대로 트는 것 — 청크를 2,000~1,000,000 토큰까지 키워 갱신을 compute-bound로 만들면, NVIDIA A100에서 활용률이 **70% 이상**으로 뛴다 [6]. (덤으로, 이렇게 커진 청크는 nonlinear state 크기를 모델 파라미터의 40% 규모까지 키울 여지를 열어 상태 용량도 함께 늘린다 [6].)

이 그림이 장 전체의 논지를 압축한다. 알고리즘 (무엇을 학습하는가)은 그대로인데, 청크 크기라는 순수 스케줄링 선택만으로 처리량이 한자릿수 배 달라진다. 그러나 공짜가 아니다. 청크가 커지면 그 안의 모든 토큰이 같은 fast weight를 보므로, 청크 내부의 토큰별 세밀 적응이 사라진다. 긴 청크는 곧 “굵은 입자”의 test-time 학습이며, 세밀한 in-context 적응이 필요한 과제 (정밀 retrieval, 국소적 상태 전환)에서 품질 손실로 나타날 수 있다 [b]. 이것이 tradeoff의 양면이다.

표: chunk 크기와 하드웨어 활용률 (A100 기준, LaCT 정량)

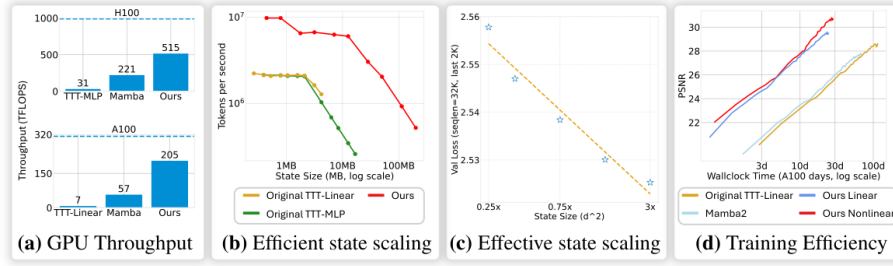


Figure 1: Using larger chunk sizes significantly improves GPU utilization compared to the original test-time training (TTT) method that even uses customized kernels (a). This enhanced utilization enables efficient and effective scaling to larger state sizes (b), (c), leading to better overall performance in less wall-clock time (d). The dotted line in (a) is the theoretical peak BF16 throughput of the GPU. Panel (c) measure average validation loss of the last 2K tokens in sequences processed by a LaCT language model across varying state sizes, demonstrating benefits of larger state size. Panel (d) compares performance versus training time across different baselines on the novel view synthesis benchmark. Further experimental details can be found in Sec. C.4.

그림 4.2. 청크를 크게 잡을수록 갱신 연산이 큰 matmul로 뭉쳐 GPU 활용률이 5% 미만에서 70%대로 개선된다. (출처: LaCT, Figure 1, [6])

갱신 방식	청크 크기 b	peak FLOPs 활용률	청크 내 적응 세밀도
online / small mini-batch (전통 TTT)	16 ~ 64	< 5%	높음 (토큰별)
TTT-Linear 기본	16	낮음 (memory-bound)	높음
LaCT large chunk	2K ~ 1M	> 70%	낮음 (청크 전체 공유)

수치 출처: [6], [1]. 활용률은 아키텍처·구현에 민감하므로 절대값보다 “5% → 70%대”의 추세로 읽어야 한다 (skeptical: LaCT의 70%는 저자 구현·특정 커널·A100 하에서의 상한이며 재현은 커널 최적화에 의존).

backprop-through-the-inner-loop의 비용

학습 스케일링을 두 배로 무겁게 만드는 두 번째 축은, TTT를 학습할 때 outer loop가 inner-loop 갱신을 관통해 미분해야 한다는 점이다. Fast weight 궤적 $W_1 \rightarrow W_2 \rightarrow \dots \rightarrow W_T$ 은 그 자체가 계산 그래프이고, slow weight (주 파라미터)에 대한 gradient는 이 사슬을 역방향으로 통과해야 한다. 이는 메타러닝의 backprop-through-optimization과 동형이며, 두 가지 비용을 부른다.

첫째, **메모리**. inner-loop의 중간 상태 W_t 들을 backward에서 되쓰려면 저장하거나 recompute해야 한다. 청크 수 T/b 에 비례해 activation footprint가 늘고, fast weight가 행렬 (예: TTT-MLP)이면 청크당 상태 크기가 d^2 규모라 저장 비용이 배가된다. 둘째, **계산**. dual form은 gradient를 명시적으로 materialize하지 않고 matmul로 재구성해 비용이 상당히 흡수하지만 [1], MesaNet처럼 청크마다 선형계를 풀어야 하는 경우엔 forward의 solve와 그 solve를 통과하는 backward가 겹쳐 train-time 비용이 더 커진다 [19]. 요컨대 TTT의 학습은 “모델을 학습하는 옵티마이저를 학습”하는 이중 루프이고, 그 이중성이 batch·시퀀스 스케일링에서 메모리와 커널 효율모두를 압박한다.

batch × state footprint

세 번째 압력은 상태의 절대 크기다. 표준 RNN의 hidden state가 $O(d)$ 벡터라면, fast-weight 모델의 상태는 $O(d^2)$ 행렬 (혹은 MLP 파라미터 전체)이다. 학습은 배치 B 개 시퀀스를 동시에 돌리므로, 상주 상태는 $B \times (\text{state size})$ 로 곱해진다. Attention의 KV cache가 배치 × 시퀀스에 선

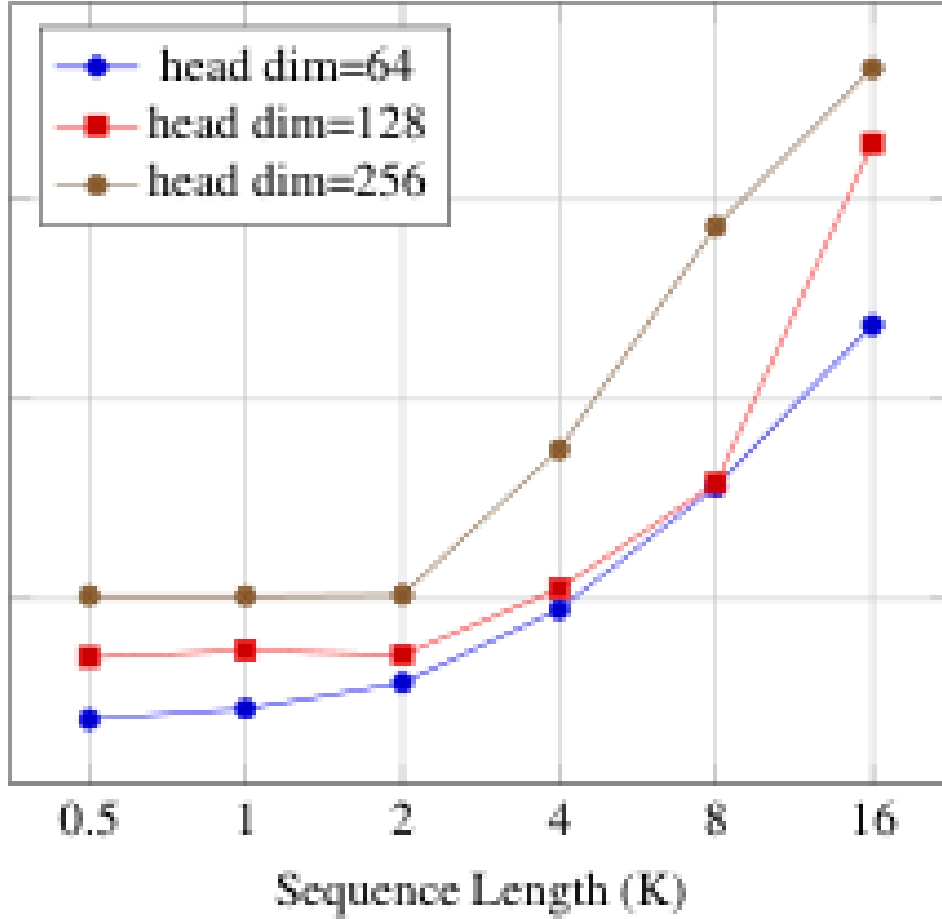


그림 4.3. delta rule 의 chunkwise-parallel 형식이 순수 recurrent 대비 얻는 속도 향상 — 청크 내부를 matmul 로 뭉친 효과. (출처: DeltaNet, Figure 1, [20])

형인 것과 달리, fast-weight 상태는 배치마다 독립적인 $d \times d$ 행렬을 요구해 시퀀스 길이엔 무관하지만 배치엔정면으로 비례한다. 이는 긴 문맥에서 attention 대비 유리하지만 (시퀀스길이 독립), 큰 배치 학습에서는 상태 행렬이 HBM 대역폭을 잡아먹어 다시 memory-bound 로 끌고 가는 요인이 된다 [b]. 청크 병렬화가 활용률을올리는 순간에도 이 배치 $\times$ 상태 footprint 는 배치 크기의 상한을 결정한다.

비선형 recurrence 의 조건부병렬화

그렇다면 순차 사슬은 절대 병렬화 불가인가? 아니다 — 조건부로가능하다. 여기서 두 갈래를 구분해야 한다 (억지 봉합 금지).

(1) 선형/준선형 recurrence 의 청크 병렬 (DeltaNet). Delta rule 갱신 $W_t = (I - \beta_t k_t k_t^T) W_{t-1} + \beta_t k_t v_t^T$ 는 각스텝이 rank-1 갱신을 가진 구조화된 선형 recurrence 다. Yang 등은이를 generalized Householder 변환을 가진 matrix-valued RNN 으로재매개변수화하고, memory-efficient WY 표현을 이용해 청크 내부를 하나의조밀 행렬 연산으로 재구성한다 [20]. 핵심은 청크 사이만 순차로 남기고청크 내부를 matmul 로 뭉쳐, 순차 스텝 수를 T 에서 T/b 로 줄이는것이다. 결과적으로 DeltaNet 의 chunkwise 커널은 recurrent 형식 대비 큰속도 향상을 얻어, 1.3B 모델을 H100 에서 약 45 Kt/s 의 실전학습 처리량으로 돌릴 수 있게 만들었다 [20] [a].

여기서 얻는 교훈은 미묘하다. DeltaNet 의 병렬화가 가능한 이유는 갱신이선형이고 결합적 (associative) 이기 때문이지, 갱신이 값싸서가아니다. 실제로 chunkwise 형식은 순수 recurrent 보다 이론 FLOPs 가 더많지만, 그 FLOPs 가 tensor core 용 matmul 로 배치되므로 wall-clock

<i>340M params / 15B tokens</i>													
Transformer++	28.39	42.69	31.0	63.3	34.0	50.4	44.5	24.2	41.2	42.2	22.1	21.4	N/A
RetNet (w/o. conv)	32.33	49.19	28.6	63.5	33.5	52.5	44.5	23.4	41.0	13.3	27.6	2.9	512x
Mamba (w. conv)	28.39	39.66	30.6	65.0	35.4	50.1	46.3	23.6	41.8	12.4	23.0	2.1	64x
GLA (w/o. conv)	28.65	43.35	30.3	64.8	34.5	51.4	45.1	22.7	41.5	18.6	27.2	8.1	128x
(w. conv)	29.47	45.53	31.3	65.1	33.8	51.6	44.4	24.6	41.8	24.0	24.7	7.3	128x
DeltaNet (w/o. conv)	29.08	50.87	30.0	63.6	33.6	51.7	46.0	23.0	41.3	24.6	26.9	4.5	128x
DeltaNet (w. conv)	28.24	37.37	32.1	64.8	34.3	52.2	45.8	23.5	42.1	26.4	28.9	12.8	128x
+ Sliding Attn	27.06	38.17	33.4	64.0	35.3	50.9	45.9	23.2	42.1	39.3	32.5	18.8	N/A
+ Global Attn (2 layers)	27.51	35.04	33.5	64.0	34.5	51.7	46.0	23.3	42.1	42.9	32.1	23.1	N/A
<i>1.3B params / 100B tokens</i>													
Transformer++	16.85	13.44	48.9	70.8	49.6	53.6	56.0	26.5	50.9	66.6	31.5	27.4	N/A
RetNet (w/o. conv)	18.64	17.27	43.3	70.0	47.3	52.5	54.8	25.6	48.9	42.8	34.7	14.3	512x
Mamba (w. conv)	17.06	13.89	46.2	72.2	40.1	54.1	59.0	28.2	50.0	41.4	35.2	6.2	64x
GLA (w/o. conv)	17.22	14.47	46.9	71.8	49.8	53.9	57.2	26.6	51.0	50.6	42.6	19.9	256x
(w. conv)	17.25	14.92	46.2	70.6	49.9	53.0	55.3	27.0	50.4	52.4	37.4	22.3	256x
DeltaNet (w. conv)	16.87	12.21	48.9	71.2	50.2	53.6	57.2	28.3	51.6	49.5	37.4	17.2	128x
+ Sliding Attn	16.56	11.74	49.2	71.8	51.1	52.8	58.9	28.8	52.1	53.3	43.3	22.3	N/A
+ Global Attn (2 layers)	16.55	12.40	48.8	70.8	50.7	54.2	58.4	28.1	51.8	71.0	43.0	29.8	N/A
<i>DeltaNet Ablations (340M)</i>													
w. L_1 -norm & 1+ELU	31.12	55.96	26.3	63.9	33.0	50.9	44.3	21.8	40.1	14.5	23.9	6.2	128x
w. L_2 -norm & 1+ELU	28.03	37.62	32.2	65.7	34.7	51.8	45.4	22.5	42.1	23.8	28.6	13.1	128x
w. L_2 -norm & ReLU	28.75	43.53	30.2	64.0	33.9	48.9	45.6	22.8	40.9	27.2	26.7	9.0	128x

Table 1: Main language modeling results against Transformer++, RetNet [108], Mamba [31], and GLA [124]. All models are trained on the same subset of the SlimPajama dataset with the Mistral tokenizer. The Transformer++, RetNet, Mamba, GLA (w/o. conv) results are taken from Yang et al. [124]. For hybrid models, “Sliding Attn” interleaves a sliding window attention every other layer, and “Global Attn” uses full global attention on two layers. The 340M/1.3B models are trained for 15B/100B tokens respectively. All results are obtained through `lm-evaluation-harness` [27]. The last column denotes the expansion ratio of the recurrent state size relative to the product of the number of layers and model dimension (see Zhang et al. [131, App. C]).

Model	ARC	HellaSwag	OBQA	PIQA	WinoGrande	MMLU	Average
Llama-3.2-3B [111]	59.1	73.6	43.4	77.5	69.2	54.1	62.8
PowerLM-3B [104]	60.5	74.6	43.6	79.9	70.0	45.0	62.3
DeltaNet-3B	60.4	72.8	41.0	78.5	65.7	40.7	59.8
RecurrentGemma-2B [30]	57.0	71.1	42.0	78.2	67.6	31.8	57.9
RWKV-6-3B [79]	49.5	68.6	40.6	76.8	65.4	28.4	54.9
Mamba-2.7B [31]	50.3	65.3	39.4	75.8	63.1	26.1	53.3

Figure 5: Zero-shot model performance across selected benchmarks for 3B models. Llama-3.2-3B and PowerLM-3B are Transformer models, while the others are recurrent models. ARC results are

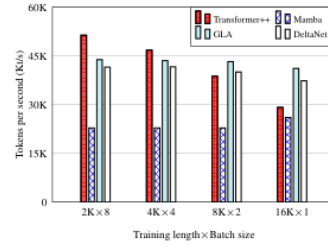


그림 4.4. 1.3B DeltaNet 의 H100 학습 처리량을 다양한 시퀀스길이 · 배치 설정에서 비교: chunk-wise 커널이 실전 규모 학습을가능케 한다. (출처: DeltaNet, Figure 6, [20])

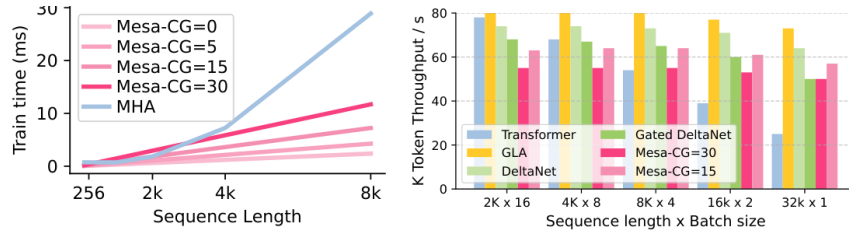


Figure 2: Train and inference time of a Mesa layer using different number of CG steps. *Left:* Train time of a single Mesa layer on a TPUv5: output the entire sequence, compute the cross entropy loss, and gradients w.r.t. layer parameters. We use batch size of 4, key size of 128 and 8 heads. *Right:* Token throughput (in thousands) when training 1B parameter models on a H100 GPU. We compare a Flash-Attention-2 (Dao, 2023) transformer implementation with a triton-based chunkwise parallel implementation of RNN models, including the MesaNet which uses 30 or 15 CG steps across all layers. All models use a key size of 128 and share the same backbone, see Appendix G. We observe competitive token throughput on H100s of the MesaNet despite using substantially more flops.

그림 4.5. MesaNet 의 CG step 수에 따른 학습 · 추론시간 (TPUv5, batch 4·key size 128·8 heads): 최적성을 위한 솔버 반복이 그대로 시간 비용으로 환산된다. (출처: MesaNet, Figure 2, [19])

이빨라진다 [b] — 다시, 알고리즘이 아니라 하드웨어 활용률이 지배한다.

(2) **최적화-기반 recurrence 의 조건부 수렴 (MesaNet).** MesaNet 은매 토큰에서 지금까지 본 전체 시퀀스에 대한 누적 정규화 최소제곱을최적으로 푸는 Mesa layer 를 쓴다 [19]. 이는 first-order online step(DeltaNet·Mamba·TTT) 보다 강한 “locally optimal” 갱신이지만, 청크마다 선형계 $Ax = b$ 를 풀어야 한다. MesaNet 은이를 **conjugate gradient(CG)** 로 풀고, 학습은 30 CG steps 로수행한다 — 저자들은 완전 수렴한 Mesa layer 를 먼저 보기 위해 train FLOPs 를 최적화하지 않았고, 400M 모델 실험에서 30 step 이후 개선이미미했기 때문이다 [19]. 여기서 병렬화는 반복 솔버의수렴이라는 조건에 묶인다 — CG step 수가 곧 순차 깊이이자 계산량이고, 이 값을 잘못 잡으면 수치 불안정이나 과도한 train/inference time 을 부른다.

이 그림이 보여주는 것은 “더 좋은 갱신 (최적해)” 이 곧 “더 병렬 친화적” 은아니라는 사실이다. MesaNet 은 품질에서 first-order RNN 을 앞서지만, 그대가로 토큰마다 동적 test-time 연산 (솔버 반복) 을 지불한다 [19]. 비선형 · 최적화 recurrence 의 병렬화는 항상수렴 · 안정성 조건부이며, 그 조건이 학습 스케일의새로운 병목으로 자리를 옮길 뿐이다.

표: 비선형 recurrence 병렬화 전략과 조건

방법	병렬화 기제	남는 순차 깊이	조건
Mamba (선택적 SSM)	hardware-aware associative scan	$O(\log T)$ (scan)	선형 · 게이팅 구조
DeltaNet	chunkwise WY / Householder 곱	T/b (청크간)	갱신의 결합성
TTT-Linear	mini-batch dual form	T/b	청크 내 상수 W
MesaNet	청크별 CG 솔버	CG step 수 (학습 시 30)	솔버수렴 · 수치 안정

출처: [21], [20], [1], [19].

결론: 병목은 알고리즘이 아니라 하드웨어활용률

이 장을 관통하는 하나의 명제는 Mamba 가 이미 극적으로 보여준 것이다. Mamba 의 selective SSM 은 데이터 의존 게이팅 때문에 LTI convolution 트릭을못 쓰지만, 저자들은 fetch·discretization·scan·projection 을하나의 커널로 융합하고 recompute 로 상태 materialization 을 피하는 hardware-aware 병렬 scan 을 짜서, 순진한 표준 scan 구현 대비 **최대 40 배** 빠른 구현을 얻었다 [21] [a].

동일한 수학, 40 배의 wall-clock 차이. 이것이 결론을 못박는다. inner-loop 를 가진 fast-weight/TTT 모델의 학습 스케일링에서진짜 병목은 “어떤 갱신 규칙이 더 표현력 있는가” 라는 알고리즘 질문이아니라, “그 갱신을 tensor core 용 matmul 로 얼마나 배치할 수 있는가” 라는**하드웨어 활용률** 질문이다. LaCT 의 청크 확대 (5%→70%대), DeltaNet 의 chunkwise 재구성, Mamba 의 fused scan 은 모두 알고리즘을 바꾸지않고 메모리 · 연산 스케줄만 바꿔 한 자릿수 배의 이득을 빼앗았다. 반대로 MesaNet 은 단 기간 알고리즘 (한정 갱신) 이 결국은 수렴배율 (솔버 반복) 으로

MODEL	PARAMS	NLL ↓	FID ↓	IS ↑	mIS ↑	AM ↓
SampleRNN	35.0M	2.042	8.96	1.71	3.02	1.76
WaveNet	4.2M	1.925	5.08	2.27	5.80	1.47
SaShiMi	5.8M	1.873	1.99	5.13	42.57	0.74
WaveGAN	19.1M	-	2.03	4.90	36.10	0.80
DiffWave	24.1M	-	1.92	5.26	51.21	0.68
+ SaShiMi	23.0M	-	1.42	5.94	69.17	0.59
Mamba	6.1M	1.852	0.94	6.26	88.54	0.52
Mamba	24.3M	1.860	0.67	7.33	144.9	0.36
Train	-	-	0.00	8.56	292.5	0.16
Test	-	-	0.02	8.33	257.6	0.19

blocks are ablated independently of the rest. Note that Transformer (MHA+MLP) were not tested in the more important outer block because of efficiency constraints.

OUTER	CENTER	NLL ↓	FID ↓	IS ↑	mIS ↑	AM ↓
S4+MLP	MHA+MLP	1.859	1.45	5.06	47.03	0.70
S4+MLP	S4+MLP	1.867	1.43	5.42	53.54	0.65
S4+MLP	Mamba	1.859	1.42	5.71	56.51	0.64
Mamba	MHA+MLP	1.850	1.37	5.63	58.23	0.62
Mamba	S4+MLP	1.853	1.07	6.05	73.34	0.55
Mamba	Mamba	1.852	0.94	6.26	88.54	0.52

5 Speed and Memory Benchmarks

To benchmark the speed of the SSM scan operation (state expansion $N = 16$), as well as the end-to-end inference throughput of Mamba, in Figure 8. Our efficient SSM scan is faster than the best attention implementation that we know (FlashAttention-2 (Dao 2024)) beyond sequence length 2K, and up to 20-40× faster than a standard scan implementation in PyTorch. Mamba achieves 4-5× higher inference throughput than a Transformer of similar size, since without the V cache it can use much higher batch sizes. For example, a Mamba-6.9B (untrained) would have higher inference throughput than a 5× smaller Transformer-1.3B. Details in Appendix E.5, which additionally includes a benchmark of memory consumption.

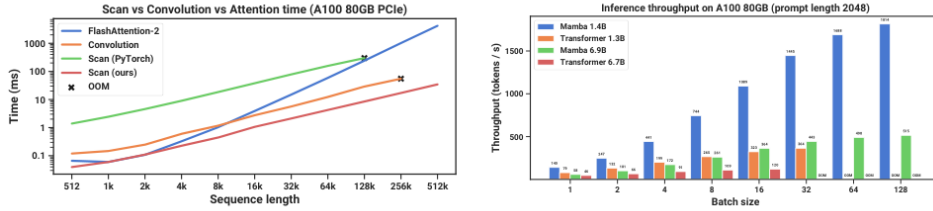


그림 4.6. Mamba 의 hardware-aware 병렬 scan 이 표준 구현 대비 40 배 속도 향상 — 동일 알고리즘, 다른 메모리 스케줄. (출처: Mamba, Figure 8, [21])

5. 병목 ③ — serving batch inference + Prefill/Decode(P/D)

3.1 batching 이라는 암묵적 계약: “요청이 공유하는 static weight”

현대 LLM serving 의 처리량은 거의 전적으로 **batching** 위에 서있다. vLLM 의 continuous batching, ORCA 의 iteration-level scheduling 이 공유하는 전제는 하나다 — 한 스텝에 묶인 모든 요청이 **동일한 weight 텐서 W** 를 읽는다는 것. 그래야 여러 요청의 activation 을 하나의 큰 GEMM 으로 합쳐 $[B, d] \times [d, d]$ 형태로 계산하고, DRAM 에서 W 를 한번 읽어 B 개 요청에 분할상환 (amortize) 할 수 있다. decode 는 요청당 토큰 1 개씩만 처리하므로 그 자체로는 GEMV (메모리 bound) 지만, B 개를 쌓으면 W -read 가 공유되어 arithmetic intensity 가 B 배로 올라간다. 즉 **batching 의 이득은 “static weight 를 공유한다” 는 계약에서 나온다.**

일반 Transformer 는 이 계약을 완벽히 지킨다. 요청별로 다른 것은 KV cache (요청 고유 상태) 뿐이고, 이건 attention 연산의 피연산자일 뿐 weight 가 아니다. weight 는 불변이므로 몇 명이 묶이든 공유된다.

3.2 TTT 가 계약을 깨는 지점: request-owned mutable fast-weight

Test-Time Training (TTT) 과 Titans 계열의 핵심은 정확히 이 불변성을 버리는데 있다. TTT layer 는 시퀀스를 흘리면서 fast-weight W_t 를 **매토큰 갱신**한다 — inner-loop gradient step 혹은 delta-rule 갱신으로. In-Place TTT 는 MLP 블록의 **최종 projection 행렬**을 그 fast-weight 로 삼아 기존 LLM 에 “drop-in” 으로 TTT 를 부여한다 [22]. 문제는 여기서 발생한다: **이제 각 요청이 자기만의 W_t 를 소유한다.** 두 요청 A, B 를 한 배치에 묶어도 A 는 W_t^A 를, B 는 W_t^B 를 읽어야 한다. 공유되는 static W 가 없으므로 하나의 GEMM 으로 합칠 수 없고, batch matmul (bmm) $[B, d, d]$ 형태의 요청별 독립행렬곱으로 퇴화한다. W -read 가 더 이상 분할상환되지 않으니 batching 의 근본 이득이 증발한다.

이 그림이 중요한 이유는, TTT serving 의 어려움을 “커널이 느리다” 가 아니라 **상태 소유권 (ownership) 문제**로 재정의하기 때문이다. RW-TTT 는 각 decode 스텝을 그 owner-version-READ/WRITE effect 로 태깅하고, 호환되는 phase 끼리만 그룹핑하며, 갱신된 상태 버전을 소유

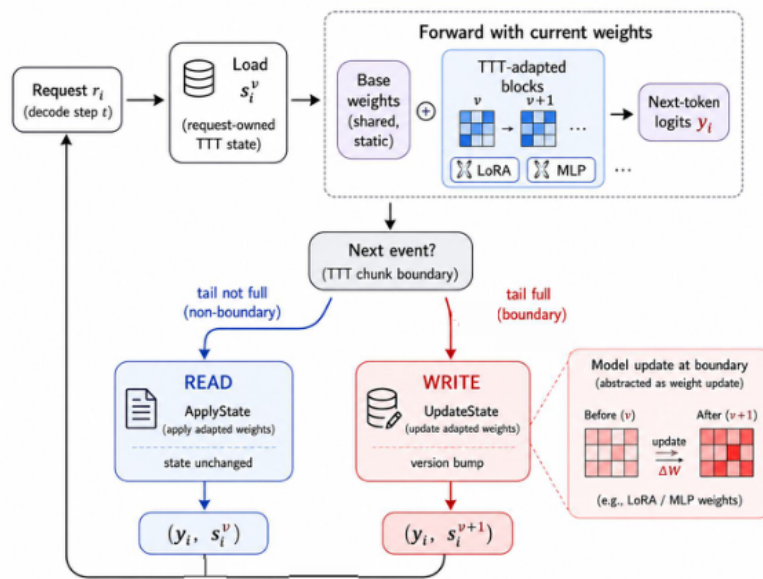


Figure 1: Per-request READ/WRITE semantics. Each decode step evaluates the shared base model with the request-owned TTT state and exposes either a version-preserving READ event or a version-creating WRITE event.

그림 5.1. 각 decode 스텝에 owner 와 READ/WRITE effect 를 태깅하는 per-request 상태 semantics — TTT 요청이 자기 fast-weight 를 읽고 (READ) 갱신본을 자기에게만 커밋 (WRITE) 하는 구조 (출처: RW-TTT, Figure 1, [23])

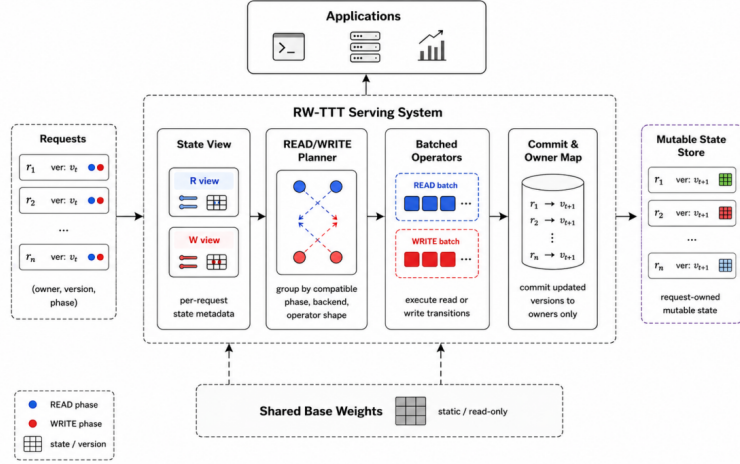


Figure 2: RW-TTT serving architecture. Requests carry owner-versioned mutable state alongside ordinary KV cache. The planner automatically groups compatible READ/WRITE transitions by backend type, operator shape, placement, and valid owner version; batched operators return outputs and updated state through the owner map. The figure shows serving semantics, not the backend update rule.

그림 5.2. owner-versioned state 를 중심에 둔 RW-TTT serving 아키텍처 — phase 호환 그룹핑과 요청별 상태 커밋 경로 (출처: RW-TTT, Figure 2, [23])

요청에게만 커밋한다 [23]. 즉 batching 을 되살리는 것은 새 커널이 아니라 **상태 버저닝을 얻은 serving contract** 다. 실측치를 보면 이재구성의 값어치가 드러난다.

표: RW-TTT — 단일 GPU, 8 개 fast-weight In-Place-TTT 스트림 [23] [a]

서빙 경로	집계 처리량 (tok/s)	상대 배수
Sequential TTT (요청 순차 처리)	29.5 (환산)	$1.00 \times$ (기준)
동일 메모리 예산 per-stream replica	79.8 (환산)	$3.44 \times$ 의미비교 기준
RW-TTT (owner-versioned batching)	274.61	sequential 대비 $9.31 \times$ / replica 대비 $3.44 \times$

집계 처리량 274.61 tok/s와배수 ($9.31 \times$ over sequential, $3.44 \times$ over per-stream replicas under the same memory budget) 는 논문 명시 [a]. sequential-replica 의 tok/s(29.5·79.8) 는 274.61 을배수로 나눈 역산 근사값 [b]. RULER(장문맥 벤치마크) 에서 동작이보존되고 owner/version 검사를 통과 [a].

주목할 점은 baseline 이 두 개라는 것이다. 하나는 “sequential” (요청을 순차처리 — 안전하지만 batching 을 포기) 이고, 다른 하나는 “동일 메모리 예산하의 per-stream replica” (같은 메모리 budget 안에서 스트림마다 모델사본을 띄우는 구성) 다. 논문이 강조하는 정직한 비교는 후자다 — **같은 메모리 예산에서** RW-TTT 가 replica 구성 대비 $3.44 \times$ 를 얻는다 [23]. 이 “memory-matched $3.44 \times$ ” 가 “TTT batching 이 실제로 복원 가능하다” 는 핵심증거다. 반면 $9.31 \times$ 는 batching 자체를 포기한 sequential 하한선과의 비교이므로 상대적으로 관대한 baseline 임을 감안해야 한다. 또한이는 8-stream 이라는 소규모 설정임을 유의해야 하며 (대규모동시성 · 이질적 시퀀스 길이에서의 스케일은 미검증), 단일논문의 수치라는 점에서 재현 유보가 필요하다.

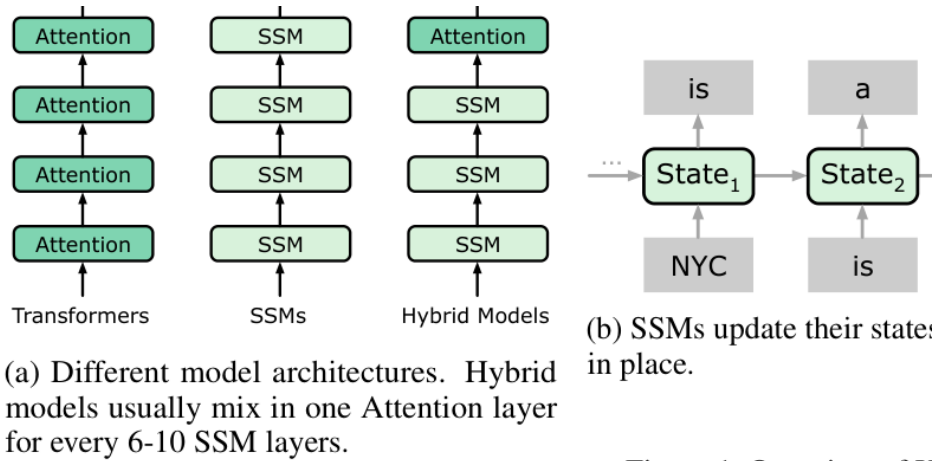


Figure 1. Overview of H

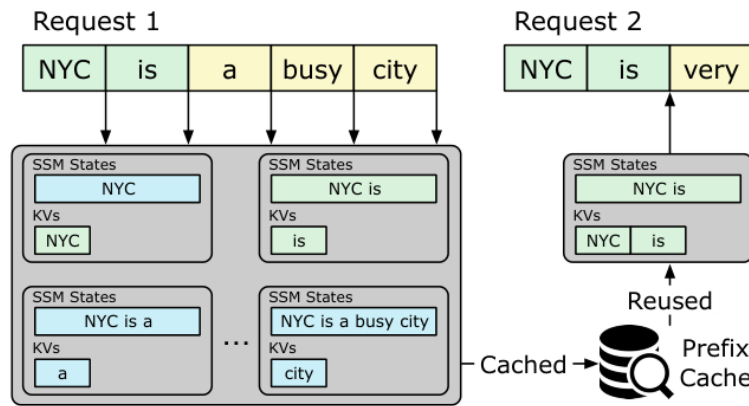


Figure 2. Prefix caching reuses model states of common prefixes (green) across requests, accelerating inference. Fine-grained checkpointing results in many sparsely-hit entries (blue).

그림 5.3. 공통 prefix 의 model state 를 재사용하는 prefix caching 개념도 — attention 의 per-token KV 는 잘라 붙지만 recurrent state 는 exact-match 만 가능 (출처: Marconi, Figure 2, [24])

3.3 recurrent state 는 KV cache 처럼 prefix-cache 되지않는다

두 번째 병목은 **재사용 (reuse)** 이다. 일반 LLM serving 에서 처리량을 크게 밀어올리는 무기가 prefix caching 이다: 여러 요청이 공유하는 system prompt·few-shot 예시의 KV 를 한 번 계산해 재사용한다. KV cache 는 **append-only per-token** 구조라서, 공통 prefix 까지의 KV 를 잘라 붙이는 부분 재사용이 자연스럽다.

recurrent/TTT 상태는 여기서 무너진다. Marconi 가 정확히 이 지점을 짚는다: hybrid model 의 recurrent layer 는 **in-place state update** 를 쓰기 때문에, 부분 시퀀스 겹침에 대해 캐시 엔트리를 **롤백할 수 없고**, 오직 exact-match cache hit 만 허용된다 [24]. per-token KV 는 “prefix 지점까지 잘라내기” 가 되지만, recurrent state S 는 시퀀스를 흡수하며 자기 자신을 덮어써온 하나의 압축된 요약이라, “앞의 N 토큰까지만의 S ” 를 사후에 복원할 방법이 없다. 그 결과 조금이라도 다른 prefix 는 재사용 불가 — 재사용 기회는 희박한데 엔트리는 거대한, 시퀀스당 대용량 저재사용 캐시의 “홍수 (deluge)” 가 발생한다 [24].

이 그림의 함의는 이렇다. Marconi 는 recency 뿐 아니라 **재사용가능성 예측 (forecast of reuse likelihood)** 과 **FLOP-aware eviction** (엔트리의 메모리 대비 잠재 compute 절감을 함께 고려) 을 반영한 admission/eviction 정책으로 이 저재사용 문제를 완화한다 [24]. 즉 recurrent

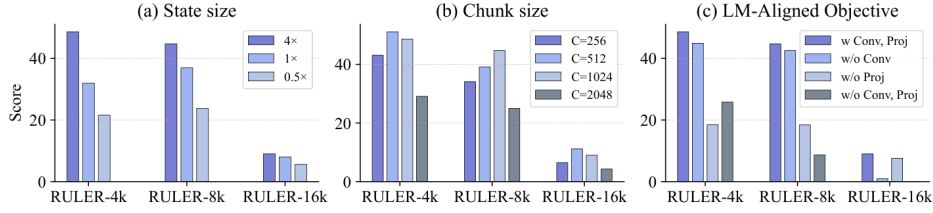


Figure 3 Ablation studies on the key design choices of the In-Place TTT framework, evaluated on the RULER benchmark with a 1.7B parameter model. The plots illustrate the impact of: (a) State size, showing that performance improves as the state size scales; (b) Chunk size, demonstrating a performance trade-off where intermediate sizes (e.g., 512, 1024) are optimal; and (c) The LM-Aligned Value objective, confirming that both the convolution (w Conv) and the projection (w Proj) are crucial.

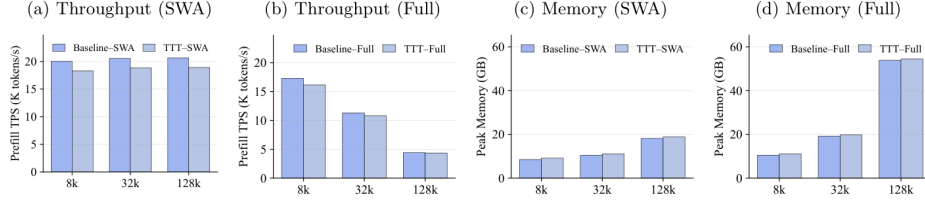


Figure 4 Efficiency analysis of In-Place TTT. Both prefill throughput (a, b) and peak memory (c, d) metrics are presented for 4B models with Sliding-Window Attention (SWA) and Full Attention at various context lengths. Our In-Place TTT introduces negligible overhead in practical scenarios.

그림 5.4. In-Place TTT 의 prefill throughput(a,b)·peak memory(c,d) — 4B 모델, Sliding-Window Attention/Full Attention 에서 컨텍스트 길이별로 base 대비 오버헤드가 미미함 (출처: In-Place TTT, Figure 4, [22])

state 의 근본적 non-rollback 성질은 못 바꾸니, “어떤 큰 상태를 캐시에 넣고 언제 버릴지” 를 영리하게 골라 hit rate 를 방어하는 우회로다. HYPIC 은 한 걸음 더 나가, 비연속 세그먼트 재사용 (position-independent caching) 을 위해 linear-attention layer 의 누락된 대수적 원소로 **segment-cumulative transition operator** 를 식별하고, 이를 각 세그먼트의 zero-start end-state 와 함께 캐싱해 독립 캐싱된 세그먼트의 near-exact · 상수시간 상태 합성을 가능케 한다 (세그먼트 경계의 작은 seam window 만 재계산) [25]. 이걸 “recurrent state 는 합성 불가” 라는 통념에 대한 부분적 반례지만, transition operator 를 별도로 저장 · 합성하는 추가 비용과 near-exact (완전 무손실이 아닌) 근사를 수반한다 [b].

3.4 prefill vs decode 의 비대칭: 대량 흡수 vs 순차 갱신

세 번째 축은 P/D 의 연산 성격 비대칭이다. 일반 LLM 에서도 prefill 은 compute-bound (긴 프롬프트 병렬 처리), decode 는 memory-bound (토큰 1 개씩, KV read 가 지배) 로 갈린다. 이 비대칭이 DistServe·Splitwise 의 P/D disaggregation 을 정당화했다 [26].

recurrent/TTT 는 이 비대칭이 더 극단적이고 성격이다르다.

- **prefill**: 시퀀스를 chunkwise parallel scan 으로 대량 흡수해 상태를 만든다 — compute-bound 이며 잘 최적화된다. In-Place TTT 는 이 prefill 오버헤드가 실사용 시 무시할 수준임을 보인다 [22].

이 그림이 논지에서 중요한 이유: TTT 의 병목이 prefill 이 아니라 **decode 에 몰려 있음**을 시각적으로 확정하기 때문이다. prefill 은 병렬화로 흡수되지만, decode 는 다르다.

- **decode**: 매 스텝 큰 상태 S 를 읽고 → 갱신하고 → 다시 써야 한다. 문제는 **상태 크기가 per-token key/value 보다 훨씬 크다**는 것이다. KVBuffer 가 이를 정면으로 다룬다: 기존 시스템은 매 decode 스텝마다 거대한 linear-attention state 를 recurrent 하게 재계산 · 갱신하는데, 상태가 토큰 KV 보다 크므로 막대한 메모리 접근이 발생해 비효율적이다 [27]. 즉 recurrent decode 는 **state-bandwidth bound** 다 — 일반 LLM decode 의 “KV-read bound” 보다 무거운, 요청당 고정 대형 상태의 read-modify-write 왕복이 매 토큰 발생한다.

표: recurrent/TTT decode 병목 완화 — 대표 수치 [a]

시스템	대상	기법	보고 효과
KVBuffer [27]	linear attention (SGLang / Qwen3-Next)	최근 K/V 버퍼링 + chunkwise 지연 · 일 괄 갱신	decode latency 최대 45.17%↓ ; spec-decode(4 draft 검증) 시 최대 요 청 수 5×
Kimi Linear [28]	KDA+MLA hybrid (3:1)	Gated DeltaNet 확장 finite-state RNN memory 미세 gating	KV cache 최대 75% ↓, 1M 컨텍스트 decode 처리량 최대 6×
In-Place TTT [22]	MLP 최종 projection fast-weight TTT	NTP-정렬목적함수 + chunk-wise 갱신	prefill 오버헤드 무시 수준

KVBuffer의 통찰이 특히 중요하다 — 매 토큰 상태를 갱신하는 대신 최근 K/V를 버퍼에 모아 두고 **갱신을 지연 · 일괄 (batch)** 적용하면, 상태 왕복 횟수 자체를 줄일 수 있다. 이는 recurrent decode의 state-bandwidth 병목이 “알고리즘적으로 강제된 것”이 아니라 “IO 스케줄링으로 완화 가능한 것”임을 시사한다. 다만 45.17%·5×는 특정 모델(Qwen3-Next)·특정 프레임워크(SGLang)·특정 설정(4 draft token 병렬 검증)에서의 상한치이므로, 일반화에는 주의가 필요하다.

3.5 hybrid의 P/D disaggregation이 왜 더 까다로운가

이제 세 병목이 만나는 지점이 hybrid(attention+recurrent) 모델의 **P/D disaggregation**이다. 일반 LLM에서 disaggregation의 메커니즘은 명료하다: prefill 노드에서 KV cache를 만들고, 그 KV를 **decode 노드로 전송**한 뒤 이어서 생성한다. 전송 대상이 per-token append 구조라 부분전송 · 오버랩 · layer-wise pipelining이 자연스럽다.

hybrid에서는 전송해야 할 상태가 이질적으로 둘이다:

1. **attention layer의 KV cache** — 컨텍스트 길이에 비례해 커지고, per-token으로 잘 정의됨.
2. **recurrent/TTT layer의 fixed-size state S** — 크기는 상수지만, in-place 갱신이라 부분전송 · 롤백 불가.

이 조합이 disaggregation을 어렵게 만드는 이유를 논리적으로 짚으면:

- **재사용 비대칭:** KV는 prefix-cache이지만 recurrent state는 exact-match만 된다 (§ 3.3). 같은 요청 안에서도 두 상태의 캐싱 정책이 갈려 통합 스케줄링이 깨진다 [24].
- **전송 프로파일 비대칭:** KV는 길이에 따라 커져 “긴 컨텍스트일수록 전송 비용 ↑”인데, recurrent state는 상수 크기라 “짧은 요청에선 상대적으로 무겁고 긴 요청에선 가벼운” 반대 곡선을 그린다. 하나의 $P \rightarrow D$ 전송을 두 프로파일에 동시에 맞추기 어렵다 [b].
- **decode 자원 성격 비대칭:** decode 노드는 attention 쪽은 KV-read bound, recurrent 쪽은 state-bandwidth bound (§ 3.4)로 서로 다른 병목을 동시에 감당해야 해, DistServe식 “decode 노드는 memory-bound에 맞춰 최적화”라는 단순 처방이 부분적으로만 통한다.
- **TTT 특유의 write 문제:** 일반 disaggregation에서 decode 노드는 KV를 읽기만 한다. 그러나 TTT decode는 매 스텝 상태를 쓴다. 갱신본은 그 요청에게만, 순서대로 커밋되어야 하므로 (§ 3.2), disaggregated 세팅에서 상태의 owner·version 일관성을 노드 경계를 넘어 유지해야 한다. RW-TTT의 owner-versioned contract가 필요해지는 지점이 정확히 여기다 [23].

정리하면, 일반 LLM 의 P/D 분리 (DistServe/Splitwise/Sarathi) 가 다루는대상은 “요청 고유 · 읽기전용 · per-token · 부분전송 가능” 한 KV cache 다. recurrent/TTT 가 추가하는 것은 “요청고유 · 읽고쓰는 · 압축 · 비가역 · 부분전송 · 롤백불가” 한 상태다. Sarathi 의 chunked-prefill+piggyback decode 같은 스케줄링 트릭도, 상태가 read-only 일 때 성립하는 자유도 (prefill 조각과 decode 를 자유롭게 섞기) 를 recurrent write-order 제약이 잠식한다 [b]. Nemotron-H [18]·Kimi Linear [28] 같은 프로덕션급 hybrid 가 KV 감축 · decode 처리량에서 큰 이득을 실증하면서도, 그 이득을 **disaggregated serving** 에서 온전히 실현하는 것은 별개의 의미해결 시스템 문제로 남아 있다. Marconi(prefix caching)·HYPIC(position-independent caching)·KVBuffer(IO-aware decode)·RW-TTT(owner-versioned batching) 는 각자 병목의 한 조각만 공략하며, 이들을 하나의 disaggregated 파이프라인으로 통합한 end-to-end 시스템은 2026 년 중반 기준 아직 부재하다 [c].

이 장의 결론은 단순하다. **mutable per-request state** 는 **serving** 스택의 세 가지 암묵적 가정 — **weight 공유 (batching)·per-token 재사용 (prefix cache)·read-only 전송 (disaggregation)** — 을 동시에 위반한다. 각 가정을 되살리려는 우회로가 존재하고 국소적으로는 3~9× 수준의 회복을 보이지만, 세 병목이 곱해지는 hybrid P/D disaggregation 은 여전히 이 study book 이 지목하는 가장 미성숙한 프런티어다.

Sources: Marconi 2411.19379, RW-TTT 2605.28053, HYPIC 2607.01299, KVBuffer 2605.19049, In-Place TTT 2604.06169, Kimi Linear 2510.26692, Nemotron-H 2504.03624, DistServe 2401.09670

Part II 정리. 세 병목은 한 뿌리의 세 그림자다 — **파라미터로 안 커지고** (유한 메모리 용량), **학습이 병렬화가 안 되고** (순차 inner-loop · 낮은 util), **서빙이 배치가 안 된다** (request-owned mutable state).

Part III — 해법 3 종 (무엇을 · 어떻게 · 왜 working)

6. 해법 ① — model-size scaling 해결

규모 병목을 다시정의하기

앞 장들에서 본 TTT·Titans 계열의 선형 순환 모델은 계산 · 메모리 측면에서 매력적이지만, 실전 배치의 관문에서 반복적으로 같은 벽에 부딪힌다. **model-size scaling** 문제다. 이벽은 세 층위로 나뉜다. 첫째, 프런티어급 성능은 여전히 수십~수백 B 파라미터의 거대 Transformer 에서만 나오는데, 선형 모델을 그 규모로 처음부터 학습하는 것은 검증되지 않았고 값비싸다. 둘째, 고정 크기 순환 상태 (fixed-size state) 는 규모를 키워도 용량 (capacity) 이 선형적으로 늘지 않아 recall·retrieval 에서 병목이 생긴다. 셋째, 일단 학습을 마친 모델은 파라미터가 동결되어 새 지식을 흡수하며 스스로 커질 방법이 없다.

이 장은 이 세 층위에 각각 대응하는 세 갈래의 해법 — (A) **규모상속** (큰 Transformer 를 값싸게 distill/linearize), (B) **용량 확장** (feature·state 를 키움), (C) **파라미터성장** (모델이 스스로 커짐) — 을 메커니즘 수준에서 따라간다. 세 갈래는 서로 목표가 다르며 역지로 하나로 봉합되지 않는다는 점을 미리 밝혀둔다. 각 갈래의 재현성 등급이 크게 다르다는 사실 — 상속은 이미 여러 프런티어 체급에서 재현되었고, 성장은 아직 소·중형 실험단계에 머문다 — 도 이 장을 관통하는 축이다.

갈래 A — 규모 상속: distill 과 linearize

가장 실용적인 답은 “큰 Transformer 를 처음부터 다시 학습하지 말고, 이미 존재하는 것에서 규모를 물려받자” 이다. 여기엔 두 하위 전략이 있다.

(1) **구조 이식형 distillation** — MOHAWK / Phi-Mamba, Llama. MOHAWK[29] 의 통찰은 Transformer 의 attention 과 SSM 의 순환이 모두 “시퀀스를 하나의 행렬 (sequence transformation matrix) 로 섞는 연산” 이라는 공통 언어를 갖는다는 것이다. 그렇다면 학습된

attention 행렬을 목표로 삼아 SSM 행렬을 맞춰 나가면 된다. MOHAWK 는 이를 3 단계로 분해한다. ① **Matrix Orientation** — 각 층의 두 시퀀스 변환행렬 (mixing matrix) 자체를 정렬, ② **Hidden-State Distillation** — 각 블록의 은닉 표현을 정렬, ③ **Weight Transfer + end-to-end** — MLP 등 나머지 가중치를 그대로 넘겨받고 소량 데이터로 최종 예측까지 미세마감. 이 점진적 정렬 (정렬의 범위를 행렬 → 은닉상태 → 전체 모델로 단계적으로 넓힘) 덕분에 Phi-1.5 를 Mamba-2 로 옮긴 Phi-Mamba 는 단 **3B 토큰** (하이브리드판 5B), 즉 처음부터 학습할 때 통상 쓰는데이터의 1% 미만만으로, 그때까지의 모든 오픈소스비-Transformer(subquadratic) 모델을 큰 폭으로 앞섰다 [29][a]. 왜이렇게 값싼가? 처음부터 언어를 배우는 게 아니라 이미 배운 계산을 근사하는 문제로 바꿨기 때문이다.

Llamba[30] 는 같은 MOHAWK 레시피를 실제 프런티어 체급으로 밀어 올린 사례다. Llama-3.1-8B 를 순수 Mamba-2 블록 구조로 옮기며 (Llamba-1B/3B/8B 계열) 원본 학습데이터의 **0.1% 미만**으로 벤치마크 성능을 동급으로 유지했고, H100 에서 생성 길이 8192 기준 원본 Llama-3.1-8B 대비 최대 **12 배** 높은 throughput 을 냈다 [30][a]. 이는 “distillation 은 소형실험에서만 통한다” 는 의심에 대한 반례다.

(2) **선형화형 linearize — LoLCATs, Liger.** distillation 이새 구조를 세운다면, linearize 는 원본 Transformer 의 몸통을 최대한 그대로 두고 attention 연산자만 선형 순환으로 갈아끼운다. LoLCATs[31] 는 두 단계로 압축한다. 먼저 **attention transfer** — 선형 attention 이 softmax attention 의 출력을 MSE 로 모방하도록 학습 (가중치 대부분 동결), 다음 **LoRA** — 저랭크 어댑터로 근사 오차만 보정. 핵심은 전체 재학습이 아니라 두 개의 값싼 국소 문제로 쪼갠 것이다. 그 결과 처음으로 **Llama-3.1 70B·405B** 의 선형화판 (이전 최대 대비 약 50 배 큰 규모) 을 만들었고, 단 **40M 토큰** (이전 방법이 쓰던 20B~100B 대비 약 500~2500 배 적은 예산) 으로 5-shot MMLU 를 70B 에서 **+39.0 점**, 405B 에서 **+38.3 점** 끌어올려 원본과의 격차를 각각 **77.8%·78.1%** 메웠다 [31][a]. 7B·8B 급 선형화는 40GB A100 한 장에서 약 5 시간, 파라미터의 0.2%만 학습하면 되는 규모다.

Liger[32] 는 여기서 한 발 더 나아가 “추가 모듈 없이” 선형화한다. 최신 선형 순환 모델의 성능을 좌우하는 것은 게이트 (gating) 인데, Liger 는 새 feature-map 모듈을 붙이는 대신 이미 학습된 key 가중치를 재활용해 다양한 게이트를 구성한다. LoRA 로만 마감하고 나머지는 동결, 그리고 층 내부에 소량의 attention 을 남기는 **Liger Attention**(intra-layer hybrid) 을 두어 **0.02% 토큰** 으로 **원본의 93%**를 회복한다 (1B~8B 에서 검증)[32][a].

표: 규모 상속 4 법의 정량 비교

방법	대상	목표 구조	토큰 예산	핵심 수치
MOHAWK / Phi-Mamba [29]	Phi-1.5	Mamba-2	3B (hybrid 5B)	scratch 데이터 <1%로 기존 모든 OSS 비- Transformer 상회
Llamba [30]	Llama-3.1-8B	Mamba-2	<0.1% (원본 학습데이터)	벤치마크 동급 유지, throughput 최대 12×
LoLCATs [31]	Llama-3.1 70B/ 405B	linear attn + LoRA	40M (이전비 500 – 2500× 적음)	5-shot MMLU +39.0/+38.3 점, 격차 77.8%/ 78.1% 회복
Liger [32]	LLM 1B – 8B	gated recurrent	0.02%	원본 성능 93% 회복

이 네 방법이 규모 병목을 완화하는 공통 메커니즘은 명확하다. “규모에상응하는 능력” 은 학습이 재현할 대상으로 이미 존재하므로, 문제를 능력 획득에서 능력 근사로 격하시켜 데이터·연산 비용을 2~4 자릿수 줄인다. 다만 [b] 이 격차 회복률 (77.8% 등) 은 여전히 원본을 완전히 따라잡지 못했음을 뒤집어 말하며, MMLU 같은 지식 벤치마크보다 long-context recall 에서 손실이 더 클 수 있다는 점은 여러 후속 연구가 조심스럽게 지적한다. skeptic 관점에서, “몇 점 회복” 이라는 헤드

라인은 어떤 baseline·few-shot 설정을 기준으로 삼았는지에 민감하므로 절대 점수보다 동일 조건 상대 비교로 읽어야 한다.

갈래 B — 용량 확장: feature 차수와 sparse state

상속만으로는 두 번째 층위, 즉 고정 상태의 용량 한계를 못 넘는다. 순환 모델의 메모리는 본질적으로 고정 크기 행렬이고, 여기에 얼마나 많은 과거를 손실 없이 불러 담느냐가 곧 recall 성능이다.

Atlas[10]는 이 용량을 세 축으로 동시에 키운다. ① **Omega Rule** — 기존 순환은 “마지막 입력에만 최적인” 온라인 갱신을하는데, Atlas는 슬라이딩 윈도우 내 모든 과거 토큰에 대해 메모리를 최적화한다 (개별 토큰이 아니라 문맥 패턴을 기억). ② **다항 feature mapping** — key·query에 polynomial kernel 같은 고차 특징을 씌운다. 선형 attention의 용량은 feature 차원에 묶이는데, 다항 차수를 올리면 유효 용량이 급증한다는 이론적 정당화가 붙는다. ③ **Muon optimizer** 기반 내부 메모리 갱신 — 2차 정보를 근사하면서도 행렬곱 위주라 시퀀스 방향으로 병렬화된다. 요컨대 파라미터 수를 늘리지 않고 상태의 표현 밀도를 높여 용량을 확장하는 노선이다. 논문은 이 조합 (polynomial feature를 쓰는 OmegaNet 변형)이 언어모델링·commonsense·recall 집약·long-context 과제에서 Transformer와 최신 linear RNN을 모두 앞선다고 보고한다 [10][a].

Sparse State Expansion (SSE)[33]는 다른 각도에서 접근한다. 상태 갱신을 “정보 분류 (information classification)” 문제로 재해석해, softmax 기반 top-k 하드 분류로 행·희소 (row-sparse) 갱신만 수행한다. 그리고 상태를 여러 파티션으로 확장 (SSE) 하되 각 갱신은 희소하게 유지한다. 이 설계의 핵심은 **파라미터 크기와 상태 용량을 분리 (decouple)** 하는 것 — 파라미터를 늘리지 않고도 유효 상태 용량을 키우고, 클래스 (파티션) 간 간섭을 줄여 in-context retrieval·reasoning 저하를 완화한다 [33][a].

두 방법이 규모 병목을 푸는 방식은 “파라미터를 키우는 대신 상태의 유효 랭크/밀도를 키운다”로 요약된다. 이는 갈래 A(상속)와 직교하며, 실제로 상속으로 얻은 몸통에 덧씌울 수 있다. DeltaNet 계열이 delta-rule로 상태 갱신에 오차 정정 (예측 오차만큼만 갱신해 catastrophic overwriting을 피함)을 도입해 1.3B급 표준 언어 모델링에서 Mamba·GLA 같은 선형 baseline을 perplexity·zero-shot 모두에서 앞선 결과 [20][a]는 이 “상태 관리 규칙의 개선이 곧 규모 효율”이라는 주장에 실증적 무게를 더한다.

갈래 C — 파라미터 성장: 모델이 스스로 커진다

세 번째 층위는 가장 근본적이다. 지금까지의 모든 방법은 동결된 최종 크기를 전제한다. 하지만 인간의 학습은 새 지식을 만나면 시냅스를 늘린다. 두 최신 연구가 이 “성장”을 아키텍처로 끌어온다.

Sleep[13]는 모델에게 수면 주기를 준다. 낮 동안 in-context로 쌓인 취약한 단기 기억을, 밤에 안정적 장기 파라미터로 응고 (consolidate) 시킨다. 그 핵심 단계가 **Knowledge Seeding** — 작은 자기 (smaller-self)의 기억을 replay와 함께 더 큰 네트워크로 상향 증류 (upward distillation)해 용량을 키우는 **parameter expansion**이다. 즉 용량이 부족해지면 네트워크를 실제로 키운 뒤 기존 기억을 심는다. 이어 **Dreaming** 단계에서 모델이 RL로 합성 데이터 커리큘럼을 스스로 생성해, 사람 감독 없이 새 지식을 리허설하고 기존 능력을 다듬는다. 이 구조가 규모 병목을 완화하는 논리는 명료하다 — 처음부터 거대 모델을 학습할 필요 없이, 필요해질 때 용량을 늘리고 그 순간의 기억으로 채우면 되기 때문이다 [13][a].

HOPE / Nested Learning[12]는 성장의 축을 폭이 아니라 깊이로 옮긴다. Nested Learning은 아키텍처와 옵티마이저를 “서로 다른 갱신 주기를 가진 다층 중첩 최적화 문제들”로 재정식화한다. 여기서 옵티마이저 (Adam·Momentum)조차 gradient를 압축하는 associative memory로 재해석된다. HOPE는 Titans의 변형으로 두 요소를 결합한다 — 자신의 갱신 알고리즘까지 학습하는 **self-modifying** 순환 모듈과, 서로 다른 빈도로 갱신되는 뉴런들의 스펙트럼 (고빈도=빠른 적응·단기, 저빈도=지속 지식)인 **Continuum Memory System**. 그 결과 HOPE는 원리상 여러 겹의 in-context learning 레벨을 쌓을 수 있고, 이 nested depth가 파라미터를 물리적으로

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	Avg.	SWDE acc ↑	SQuAD acc ↑	FDA acc ↑	State exp.
340M params / 15B tokens													
Transformer++	28.39	42.69	31.0	63.3	34.0	50.4	44.5	24.2	41.2	42.2	22.1	21.4	N/A
RetNet (w/o. conv)	32.33	49.19	28.6	63.5	33.5	52.5	44.5	23.4	41.0	13.3	27.6	2.9	512x
Mamba (w. conv)	28.39	39.66	30.6	65.0	35.4	50.1	46.3	23.6	41.8	12.4	23.0	2.1	64x
GLA (w/o. conv)	28.65	43.35	30.3	64.8	34.5	51.4	45.1	22.7	41.5	18.6	27.2	8.1	128x
(w. conv)	29.47	45.53	31.3	65.1	33.8	51.6	44.4	24.6	41.8	24.0	24.7	7.3	128x
DeltaNet (w/o. conv)	29.08	50.87	30.0	63.6	33.6	51.7	46.0	23.0	41.3	24.6	26.9	4.5	128x
DeltaNet (w. conv)	28.24	37.37	32.1	64.8	34.3	52.2	45.8	23.5	42.1	26.4	28.9	12.8	128x
+ Sliding Attn	27.06	38.17	33.4	64.0	35.3	50.9	45.9	23.2	42.1	39.3	32.5	18.8	N/A
+ Global Attn (2 layers)	27.51	35.04	33.5	64.0	34.5	51.7	46.0	23.3	42.1	42.9	32.1	23.1	N/A
1.3B params / 100B tokens													
Transformer++	16.85	13.44	48.9	70.8	49.6	53.6	56.0	26.5	50.9	66.6	31.5	27.4	N/A
RetNet (w/o. conv)	18.64	17.27	43.3	70.0	47.3	52.5	54.8	25.6	48.9	42.8	34.7	14.3	512x
Mamba (w. conv)	17.06	13.89	46.2	72.2	40.1	54.1	59.0	28.2	50.0	41.4	35.2	6.2	64x
GLA (w/o. conv)	17.22	14.47	46.9	71.8	49.8	53.9	57.2	26.6	51.0	50.6	42.6	19.9	256x
(w. conv)	17.25	14.92	46.2	70.6	49.9	53.0	55.3	27.0	50.4	52.4	37.4	22.3	256x
DeltaNet (w. conv)	16.87	12.21	48.9	71.2	50.2	53.6	57.2	28.3	51.6	49.5	37.4	17.2	128x
+ Sliding Attn	16.56	11.74	49.2	71.8	51.1	52.8	58.9	28.8	52.1	53.3	43.3	22.3	N/A
+ Global Attn (2 layers)	16.55	12.40	48.8	70.8	50.7	54.2	58.4	28.1	51.8	71.0	43.0	29.8	N/A
DeltaNet Ablations (340M)													
w. L_1 -norm & 1+ELU	31.12	55.96	26.3	63.9	33.0	50.9	44.3	21.8	40.1	14.5	23.9	6.2	128x
w. L_2 -norm & 1+ELU	28.03	37.62	32.2	65.7	34.7	51.8	45.4	22.5	42.1	23.8	28.6	13.1	128x
w. L_2 -norm & ReLU	28.75	43.53	30.2	64.0	33.9	48.9	45.6	22.8	40.9	27.2	26.7	9.0	128x

Table 1: Main language modeling results against Transformer++, RetNet [108], Mamba [31], and GLA [124]. All models are trained on the same subset of the SlimPajama dataset with the Mistral tokenizer. The Transformer++, RetNet, Mamba, GLA (w/o. conv) results are taking from Yang et al. [124]. For hybrid models, “Sliding Attn” interleaves a sliding window attention every other layer, and “Global Attn” uses full global attention on two layers. The 340M/1.3B models are trained for 15B/100B tokens respectively. All results are obtained through `lm-evaluation-harness` [27]. The last column denotes the expansion ratio of the recurrent state size relative to the product of the number of layers and model dimension (see Zhang et al. [131, App. C]).

Model	ARC	HellaSwag	OBQA	PIQA	WinoGrande	MMLU	Average
Llama-3.2-3B [111]	59.1	73.6	43.4	77.5	69.2	54.1	62.8
PowerLM-3B [104]	60.5	74.6	43.6	79.9	70.0	45.0	62.3
DeltaNet-3B	60.4	72.8	41.0	78.5	65.7	40.7	59.8
RecurrentGemma-2B [30]	57.0	71.1	42.0	78.2	67.6	31.8	57.9
RWKV-6-3B [79]	49.5	68.6	40.6	76.8	65.4	28.4	54.9
Mamba-2.7B [31]	50.3	65.3	39.4	75.8	63.1	26.1	53.3

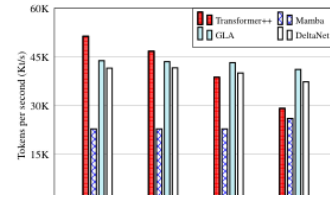


그림 6.1. DeltaNet의 zero-shot downstream 벤치마크: 단순 additive 갱신 대신 delta-rule(오차정정) 갱신을 쓰면 고정 상태 순환모델이 Mamba·GLA 등 선형 baseline을 앞서, 상태 관리 규칙의 개선이 규모 확장의 실질적 지렛대임을 뒷받침한다. (출처: DeltaNet, Figure 5, [20])

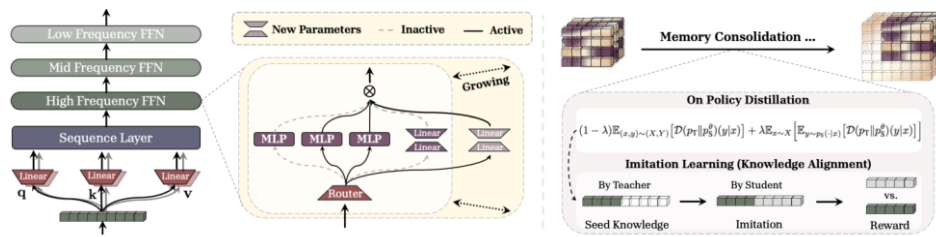


그림 6.2. Sleep가 continual learning 도중 모델 크기를 스스로 키우는 과정: Knowledge Seeding으로 용량이 포화되면 네트워크가 커지고, 작은 자기의 기억이 큰 네트워크로 상향 증류된다 — “미리 크게”가 아니라 “필요할 때 크게”의 실증. (출처: Sleep, Figure 2, [13])

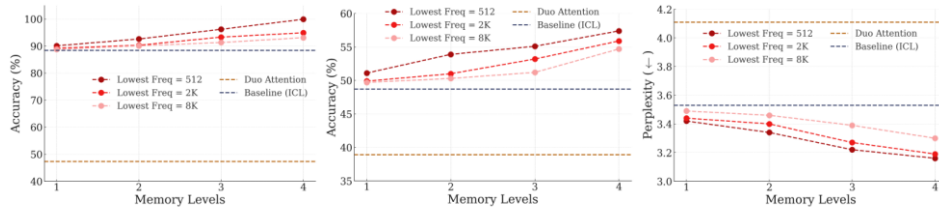


그림 6.3. HOPE의 메모리 레벨 구조: 서로 다른 갱신 주기의 메모리모듈들이 continuum을 이루며, 이 중첩 깊이 (nested depth)가 규모 확장의 새로운 축 — 폭이 아닌 깊이로 용량 성장을 제공함을 보여준다. (출처: HOPE, Figure 7, [12])

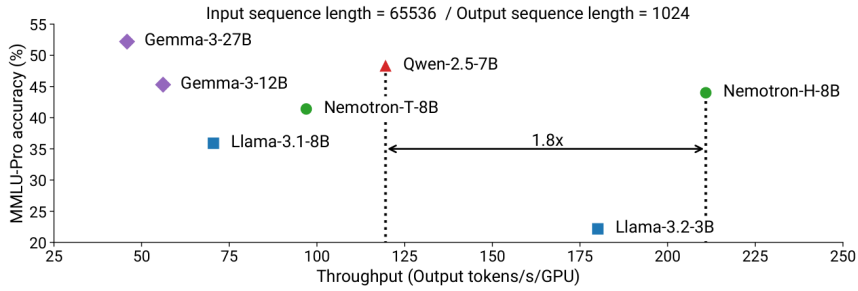


Figure 7 | MMLU-Pro accuracy versus inference throughput (normalized by number of GPUs used) for Nemotron-H-8B-Base compared to existing similarly-sized Transformer models.

그림 6.4. Nemotron-H의 MMLU-Pro 정확도 대 추론 throughput: 하이브리드 모델들이 Pareto 프런티어를 오른쪽 (더 높은 처리량)으로 밀어내며, 동일 정확도에서 순수 Transformer 보다 뚜렷이 높은 throughput을 달성함을 보여준다 — “정확도를 지키며 규모 비용을 낮춘다”는 이 장 전체 주장의 시각적 요약. (출처: Nemotron-H, Figure 7, [18])

늘리지않고도 “깊이 방향의 용량”을 제공한다. 논문은 HOPE가 언어모델링·commonsense에서 최신 순환 모델과 표준 Transformer 대비 더 낮은 perplexity와 높은 정확도를 보였다고 보고한다 [12][a].

Sleep와 HOPE는 목표 (continual/self-modification)는 겹치나성장의 방향(폭 vs. 깊이)이 달라 상보적이다. 다만 [c] 둘 다 아직 소~중형 규모의 검증 단계이며, 상향증류·self-modification의 안정성이 프런티어 체급에서 유지되는지는 미명이다. skeptic 관점에서, “스스로 커지는 모델”의 재현성이 이 장에서 다른 주장군 가운데 가장 취약하다 — 두 논문 모두 매우 최근 (2025~2026)이고 독립 재현·대규모 검증이 아직 부족하다.

가로지르는 실용 해법: hybrid로 attention 일부유지

세 갈래 어디에도 완전히 속하지 않지만 규모 병목을 실전에서 가장 확실히 완화하는 것은 **hybrid** — attention 층 일부만 남기고 나머지를 순환으로 바꾸는 절충이다. 순수 순환의 약점 (정확한 recall)은 소수의 attention이 매우고, 비용의 대부분은 순환이 절감한다. Nemotron-H[18]는 8B·56B/47B 규모에서 self-attention 대다수를 Mamba(토큰당 상수 계산·상수메모리)로 교체해, Qwen-2.5·Llama-3.1 동급 대비 정확도를 동급 이상 유지하면서 추론을 **최대 3배** 빠르게 했다. 나아가 pruning+distillation 기법 **MiniPuzzle**로 56B를 47B-Base로 압축해 정확도를 유지한 채 추론을 약 20% 더 빠르게 했다 [18][a].

이 hybrid의 성립 근거는 앞선 Mamba-2의 scaling law 관측에 있다 — SSM 코어가 대략 1.3B 급까지 Transformer++와 대등한 loss 곡선을 그린다는 사실 [34]이 “순환으로 대부분을 대체해도 된다”는 확신을 준다. 다만 원논문 자체가 1.3B 부근 이후로는 곡선의 교차 가능성을 시사하므로, hybrid에서 소수의 attention을 남기는 선택은 이 불확실성에 대한 실용적 보험이기도 하다 [34][b].

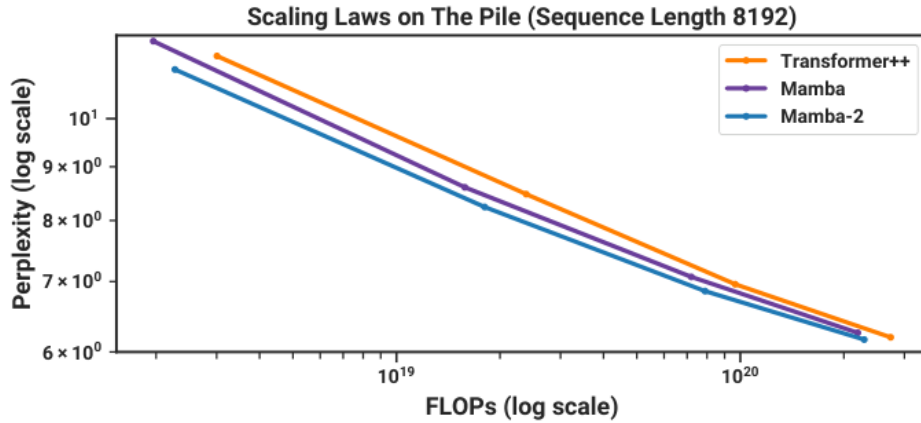


그림 6.5. Mamba-2의 scaling law: 파라미터 규모에 따른 perplexity 곡선이 1.3B 급까지 Transformer++와 겹쳐, SSM 코어가 규모에따라 attention 만큼 잘 확장됨을 보여준다 — hybrid 설계가 순환 층에 몸통대부분을 맡길 수 있는 실증적 근거. (출처: Mamba-2, Figure 9, [34])

표: 세 갈래 + hybrid 의 규모 병목 완화 메커니즘

갈래	대표 방법	완화하는 층위	핵심 메커니즘	재현성
A 상속	MOHAWK· Llama· LoLCATs·Liger	프런티어 규모 학습 비용	능력 획득 → 능력 근사로 격하	높음 (405B 까지 실증)
B 확장	Atlas·SSE	고정 상태 용량 한계	feature 차수 ↑ · sparse 파티션으로 유효 용량 ↑	중간
C 성장	Sleep·HOPE	동결된 최종 크기	파라미터확장 · nested depth로 스스로 성장	낮음 (소~중형)
hybrid	Nemotron-H	정확도-비용 트레이드오프	소수 attention 유지 + 다수 순환 대체	높음 (56B 실증)

종합하면 model-size scaling 병목은 단일 해법이 아니라 서로 다른층위를 겨냥한 상보적 전략들로 풀린다. 상속은 오늘 당장 값싸게 규모확보하고 (재현성 가장 높음), 확장은 상태 용량의 근본 한계를 밀며, 성장은동결이라는 전제 자체를 해체한다 (재현성 가장 낮음). hybrid는 이 셋을 실천배치로 접합하는 접착제다. 이들을 억지로 하나의 이론으로 봉합하기보다, 각층위의 병목에 맞는 도구를 고르는 것이 현재로서는 가장 정직한 처방이다.

7. 해법 ② — training parallel/batch scaling 해결

이 장이 겨냥하는 병목: “재귀는 왜 GPU 를 못채우는가”

앞 장에서 test-time training(TTT) 계열이 왜 표현력에서 유리한지를보았다면, 이 장은 그 표현력이 실전 학습에서 부딪히는 두 번째 벽 —**training 병렬성과 GPU utilization**— 을 어떻게 깨는지를 다룬다. 병목의 뿌리는 하나다. Titans·TTT·DeltaNet 류는매 스텝 fast weight W_t 를 online gradient step 으로 갱신한다. $W_t = f(W_{t-1}, x_t)$ 라는 재귀는 시퀀스 길이 L 만큼직렬로 풀어야 하고, 각 스텝의 연산량은 작다 (state 하나에 대한 rank-1 update 수준). 이것이 GPU 에서 재앙인 이유는 두 가지다. 첫째, 직렬의존성이 시퀀스 병렬화를 막는다. 둘째, 스텝당 행렬-벡터곱은 arithmetic intensity가 낮아 tensor core 를 굶긴다. LaCT 저자들의 계측에 따르면 소형 online

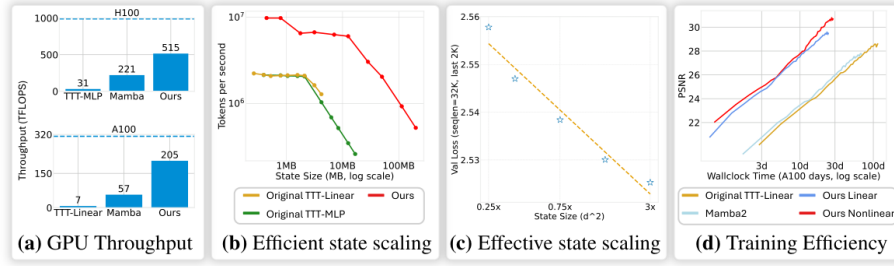


Figure 1: Using larger chunk sizes significantly improves GPU utilization compared to the original test-time training (TTT) method that even uses customized kernels (a). This enhanced utilization enables efficient and effective scaling to larger state sizes (b), (c), leading to better overall performance in less wall-clock time (d). The dotted line in (a) is the theoretical peak BF16 throughput of the GPU. Panel (c) measure average validation loss of the last 2K tokens in sequences processed by a LaCT language model across varying state sizes, demonstrating benefits of larger state size. Panel (d) compares performance versus training time across different baselines on the novel view synthesis benchmark. Further experimental details can be found in Sec. C.4.

그림 7.1. chunk 크기를 키울수록 FLOPs utilization 이 한자릿수%에서 70% 수준으로 도약함을 보이는 그래프 — LaCT 의 핵심 논거 (출처: LaCT, Figure 1, [6])

minibatch(16~64 토큰마다 갱신) 를 쓰는 TTT 구현의 FLOPs utilization 은 종종 5% 미만이다 [6]. 즉 하드웨어가 90% 이상 놓고 있다.

해법은 세 층위로 나뉜다. (1) **chunk** 를 키워 재귀 단위를 성기게만들고 그 안을 병렬 행렬곱으로 처리한다 (LaCT, TFLA). (2) 재귀 자체를 **대수적으로 재작성**해 직렬 스텝을 행렬 연산으로 접는다 (DeltaNet 의 WY/Householder, GLA 의 secondary chunking, Comba 의 SPLR). (3) 비선형 재귀를 아예 **수치 병렬 solver** 로 푼다 (DEER → ParaRNN, MesaNet 의 CG, Predictability). 이 장은 각 갈래가 직렬성 ·util 을 어떻게 깨는지, 그리고 **왜 정확도를 지키는지**를 논리 전개를 따라 설명한다.

LaCT — “chunk 를 orders-of-magnitude 키운다”

LaCT(Large Chunk Test-Time Training) 의 착상은 반직관적일 만큼 단순하다. 기존 TTT 는 “최대한 자주 갱신해야 memory 가 잘 적응한다” 는 직관에서 16~64 토큰마다 fast weight 를 갱신했다. LaCT 는 이 직관을 뒤집어 **2K 에서 1M 토큰**을 하나의 chunk 로 묶어 그 chunk 전체에 대해 한 번의 큰 갱신을 수행한다 [6]. chunk 내부의 처리는 fast weight 를 상수로 고정한 채 대형 행렬곱으로 이루어지므로 tensor core 가 포화되고, chunk 경계에서만 재귀가 발생하므로 직렬 스텝 수가 L/chunk 로 orders-of-magnitude 줄어든다.

위 그림은 이 장 전체의 핵심 논거를 한 장에 담는다. 갱신 단위 (chunk) 를 키울수록 utilization 이 상승해, 순수 PyTorch 수집 출만으로 A100 에서 **최대 70%**에 도달한다 [6]. 이는 커널 튜닝이 아니라 **알고리즘 재구조화**로 얻은 이득이라는 점이 중요하다 — 즉 하드웨어 전문성 없이도 재현 가능하다.

큰 chunk 가 열어준 두 번째 문은 **state 용량 확대**다. 스텝당 갱신이 성기므로, fast weight 를 훨씬 크게 잡아도 직렬 비용이 감당된다. LaCT 는 nonlinear(MLP) state 를 모델 파라미터의 **최대 40%**까지 키운다 [6]. 이는 “재귀 상태를 크게 = 기억 용량 크게” 라는 방향을 처음으로 저렴하게 만든 것으로, 긴 문맥 · 고해상도 비디오에서 성능의 원천이 된다.

표: LaCT 핵심 결과 (2505.23884)

항목	기존 TTT baseline	LaCT	비고
갱신 chunk 크기	16~64 토큰	2K~1M 토큰	3~4 orders-of-mag ↑

항목	기존 TTT baseline	LaCT	비고
FLOPs utilization (A100)	<5%	최대 ~70%	순수 PyTorch
nonlinear state 용량	소형	파라미터의 최대 40%	대형 MLP state
Novel view synthesis	full-attn 과 동급	PSNR 37.9 dB @48 views	prefill 16s → 1.4s
지원 문맥	—	최대 1M 토큰	sparse-view 에서 3DGS 능가

정확도를 지키는 이유는 세 가지다. (a) chunk 내부는 여전히 정확한 병렬 attention/regression 으로 처리되므로 근사 손실이 없다. (b) chunk 경계 갱신이 생겨진 대신 state 가 커져 chunk 당 정보량을 흡수한다. (c) 언어 모델링에서는 sliding-window attention 과 하이브리드로 결합해, 큰 chunk 가 놓치는 국소 정밀도를 window 가 메운다 — 저자들은 이하이브리드 (window $M = 2048, 4096$) 가 긴 위치에서 GLA/DeltaNet 대비 retrieval 정확도를 최대 +20pp 끌어올린다고 보고한다 [6]. 다만 이 수치들은 단일 그룹 보고이며 modality 별 세팅이 상이하므로, “util 70%” 는 특정커널/배치에서의 상한으로 읽어야 한다. [b]

chunkwise-parallel 의 대수학 — 재귀를 행렬 연산으로접기

LaCT가 “chunk 를 키우는” 접근이라면, DeltaNet 계열은 **재귀 자체를단한 행렬식으로 다시 쓴다**. DeltaNet 의 스텝은 delta rule $W_t = W_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$ 로, 각 스텝이 W_{t-1} 에 **generalized Householder 변환**을 곱하는형태다 [20]. Householder 곱의 누적은 WY representation 으로 압축된다 — n 개의 rank-1 갱신의 곱을 두 개의 얇은 행렬 W, Y 로 표현해, 매스텝 크기 $d \times d$ 의 state 를 materialize 하지 않고도 chunk 전체를한 번의 행렬곱 블록으로 계산할 수 있다 [20].

위 그림은 이 대수적 재작성의 실질을 보여준다. 순차 형식은 시퀀스가길어질수록 직선적으로 느려지지만, chunkwise 형식은 chunk 내부를 half-precision matmul 로 처리해 GPU 를 채운다. 이 재작성이 정확도를**정확히 보존**한다는 점이 핵심이다 — WY 는 근사가 아니라 Householder 곱의 대수적 항등식이므로, 병렬 형식과 순차 형식은 수치 오차범위에서 동일한 출력을 낸다. 이것이 “속도는 언되 정확도는 공짜” 인 이유다.

이 계보 위에 여러 확장이 놓인다. GLA 는 data-dependent gate 를 더하면서 secondary-level chunking 으로 non-half-precision(비-matmul) FLOPs 를 줄여, FlashAttention-2 보다 짧은 시퀀스 (1K) 에서도 빠른 FlashLinearAttention 을구현했다 [4]. Comba 는 여기서 한 걸음 더 나아가 **closed-loop control** 관점을 도입한다: scalar-plus-low-rank(SPLR) state transition 에 state feedback 과 output feedback 보정을 결합하되, chunk 연산자를**단일 행렬 역변환**만 요구하도록 재설계했다 [35]. Gated-DeltaNet 이 역행렬을 두 번 계산하는 데 반해 Comba 는 한 번만 계산해 $O(d^3)$ 역행렬 병목을 완화하며, 그 결과 Comba 의 Triton chunk 커널은 Gated-DeltaNet 대비 forward 에서 약 +40% 빠르다 [35].

표: chunkwise-parallel 계열의 병렬화 기법

방법	재귀 재작성 핵심	병렬화 이득	정확도 근거
DeltaNet [20]	Householder 곱 → WY representation	순차 대비 대폭 speedup, state materialize 회피	대수적 항등식 (무 손실)
GLA [4]	gate + secondary chunking	FlashAttn-2 보다 빠름 (1K 부터)	하드웨어-효율 chunkwise form
Comba [35]	SPLR + 단일 역행렬 WY	Gated-DeltaNet 대비 forward +40%	RWKV-7·GDN 능가 ppl(340M/1.3B)

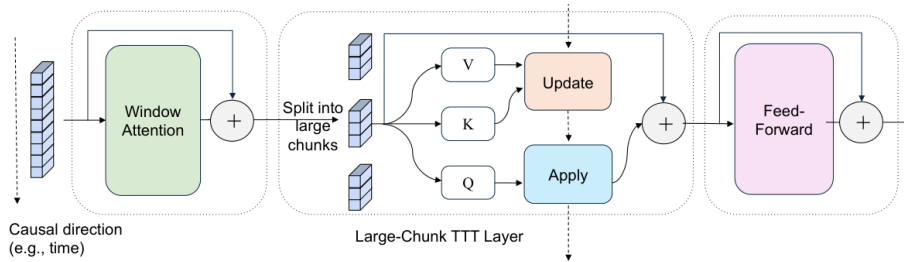


Figure 2: The basic diagram for a LaCT block. The large-chunk TTT layer updates the fast weight W to store historical context information, while the window attention handles the locality and internal structures within the chunk. The solid line denotes the information flow over model depth and the dashed line denotes the information flow over time (i.e., the fast weight W passing through chunks). Various instantiations in Sec. 4 use different chunk sizes and window attention types according to the specific data structure. Additionally, window attention and large-chunk TTT layers can be combined within the same layer by sharing the QKV and summing their outputs; this in-layer mixing is used in our language modeling and video generation experiments (see Appendix 2 for such pseudocode).

그림 7.2. 한 chunk 를 대형 병렬 연산으로 처리하고 chunk 경계에서만 fast weight 를 갱신하는 LaCT block 구조도 (출처: LaCT, Figure 2, [6])

TNT — chunk-size 딜레마와 계층적 memory

큰 chunk 는 공짜가 아니다. TNT(2511.07343) 는 이 장의 skeptic 역할을 한다: 기존 병렬화가 **chunksize** 라는 **hyperparameter** 가 강제하는 근본적 상충에 갇혀 있음을 정면으로 짚는다 — chunk 를 키우면속도는 오르지만 성능이 떨어지고, 따라서 고정된 차선의 타협을강요당한다 [11]. 한 chunk 내부에서 fast weight 가 고정되어 있으므로 chunk 안의 미세한 기억 갱신이 사라지는 것이 성능 저하의 원인이다. LaCT 가 “state 를 키워” 이를 상쇄했다면, TNT 는 **training 효율과 inference 성능을 분리 (decouple)** 하는 2 단계 학습으로 상쇄한다.

1 단계 (pre-training) 는 효율 중심으로, hierarchical memory 를둔다. **global module** 은 크고 하드웨어 친화적인 chunk 로 장거리문맥을 처리하고, 여러 개의 **parallel local module** 이 그 안에서 fine-grained detail 을 병렬로 담당한다 [11]. 이때 local memory state 를주기적으로 reset 해 직렬 의존성을 끊음으로써 대규모 context parallelization 을 가능케 한다. **2 단계** 는 짧은 fine-tuning 으로, local memory module 만 더 작은 고해상도 chunksize 로 적응시켜 최소오버헤드로 정확도를 회복한다 [11]. 저자들은 이 구성이 가장 정확한 baseline 대비 학습을 **최대 17 배** 가속한다고 보고한다 [11].

이 그림이 중요한 이유는 chunk-size 딜레마를 “선택” 이 아니라 “구성” 으로 푸는 것을 시각화 하기 때문이다. 단일 chunk 축에서 속도-성능을 고르는데는, global(큰)·local(작은) 두 축을 동시에 두어 runtime 은대형-chunk 쪽에, 성능은 소형-chunk 쪽에 붙인다. [b] TNT 는 비교적최신 (2025-11) 결과이므로 대규모 재현은 아직 제한적이며, “decouple” 의이득이 scale 에서 유지되는지는 추가 검증 대상이다.

비선형 recurrence 자체의 병렬화 — DEER → ParaRNN, Predictability

WY 대수는 delta rule 처럼 **선형에 가까운** 재귀에서만 닫힌 형태를준다. 재귀가 본질적으로 비선형 (예: MLP fast weight, gated nonlinear state) 이면 이 트릭이 안 통한다. 여기서 세 번째 갈래 — **재귀를수치 solver 로 병렬화** — 가 등장한다.

출발점은 DEER(2309.12252) 다. 비선형 재귀 $s_t = f(s_{t-1}, x_t)$ 의 전체 궤적을 하나의**고정점 (fixed-point) 문제**로 보고, 병렬 형태의 Newton 방법으로푼다 [36]. 각 Newton iteration 은 재귀를 국소 선형화하고, 그 선형재귀는 **associative scan**(예: `jax.lax.associative_scan`) 으로 $O(\log L)$ 깊이 에 병렬 해결된다. 저자들은 이 방법이 아키텍처에 특별한구조를 요구하지 않으면서 출력 정확도를 해치지 않고 GPU 평가를 최대 3 orders-of-magnitude, 실제 순차 모델 학습을 10 배 이상 가속한다고보고한다 [36]. 문제는 (a) Newton 스텝이 state 크기에 대해 cubic 이고, (b) 수치 불안정이 발생한다는 것.

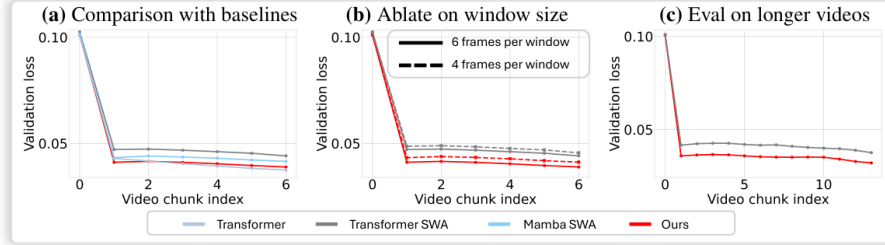


Figure 6: (a) We achieve comparable validation loss to the full-attention baseline and outperform both Mamba with sliding window and sliding window attention baselines. This improvement over SWA is consistent across different window sizes (b) and when evaluating on longer videos (c).

chunk-wise denoising loss across evaluated video frames. We only measure validation loss up to training sequence length. See project website for our autoregressively generated videos ².

5.4 Analysis on Design Choices

In this section, we analyze several key design choices in our model, focusing on both the novel view synthesis and language modeling tasks, where good metrics exist. Specifically, we evaluate the impact of state size (Fig. 7a), test-time optimizers (Fig. 7b), linear versus nonlinear fast weights (Fig. 8a), and per-token recurrence versus chunk-wise recurrence (Fig. 8b). Overall, we find that a large state size, advanced optimizers such as Muon, and nonlinear fast weights significantly improve our model’s performance. For comparing chunk recurrence with per-token recurrence, in a controlled NVS experiment, our linear large-chunk recurrence strategy outperforms linear per-token recurrence with the same state size. For language modeling, where chunk structures are not inherent, our linear large-chunk recurrence variant—while initially underperforming per-token methods like GLA and DeltaNet—surpasses them when combined with a large nonlinear state and the Muon optimizer. We refer the readers to each figure and its caption for more detailed analysis.

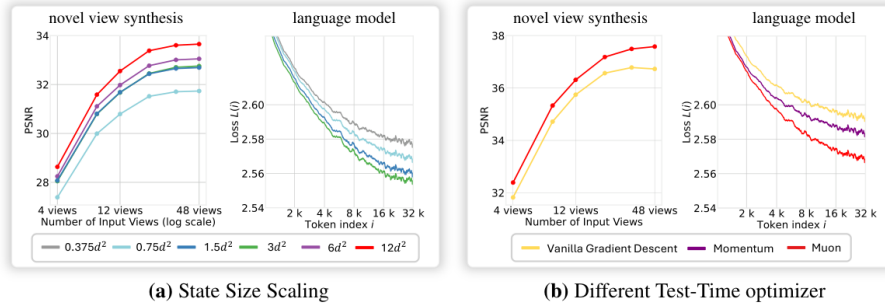


Figure 7: (a) Scaling up the state size consistently improves performance in both novel view synthesis and language modeling tasks. Note, the largest version has state size of $12d^2$ per block, totaling 40% of model weights as fast weights. (b) Comparison of test-time optimizers demonstrates Muon’s surprising effectiveness over Vanilla Gradient Descent and Momentum.

그림 7.3. nonlinear state 크기를 키울수록 성능이 개선되는 scaling 곡선 — 40% 지점까지의 이득 (출처: LaCT, Figure 7, [6])

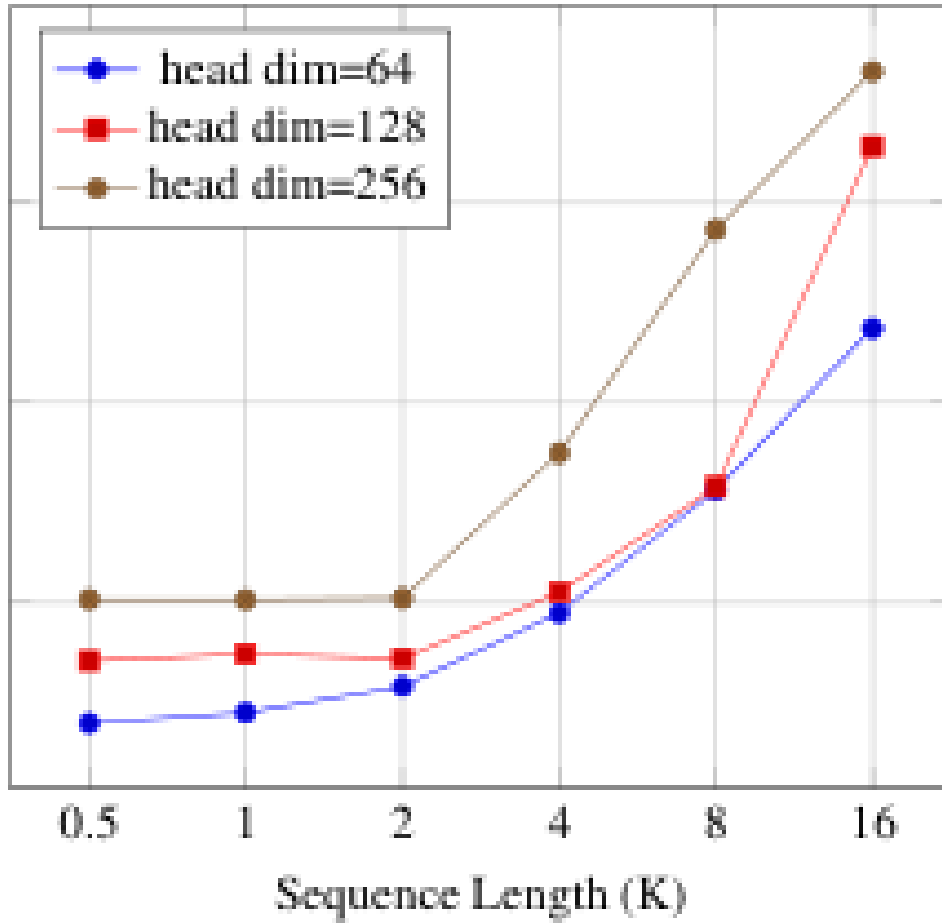


그림 7.4. DeltaNet의 chunkwise-parallel 형식이 순차 (recurrent) 형식 대비 얻는 wall-clock speedup 곡선 (출처: DeltaNet, Figure 1, [20])

TNT	JAX	{8}	2.54	7.68×
TNT	JAX	{16}	1.65	11.78×
TNT	JAX	{32}	1.22	15.92×
TNT	JAX	{64}	1.12	17.37×
TNT	JAX	{128}	1.16	16.75×

As sequence length increases, TNT's superior scalability creates a crossover point where it becomes faster. This efficiency is most pronounced at very long contexts; for instance, at a sequence length of 32K, a implementation of TNT ($C_L = 128$) outperforms even the highly optimized FlashAttention kernel, confirming TNT's efficiency for demanding long-context training scenarios.

Quality Comparison. We next translate step runtime gains into a practical time-to-train. As shown in Table 1, our TNT framework accelerates the total training time required to reach target model quality. Our best configuration is up to **17.4× faster** than the original Titans. This efficiency gain is fundamental to the performance of TNT; for instance, using an identical local memory budget of 8, TNT is already **7.7× faster** than its baseline. While competitive with standard transformers in JAX, our implementation does not match highly optimized baselines like the former with FlashAttention (Dao, Fu, et al., 2022). As an expected result, as TNT currently lacks a self-attention kernel, which we leave for future work. Nonetheless, these results establish TNT as an efficient foundation.

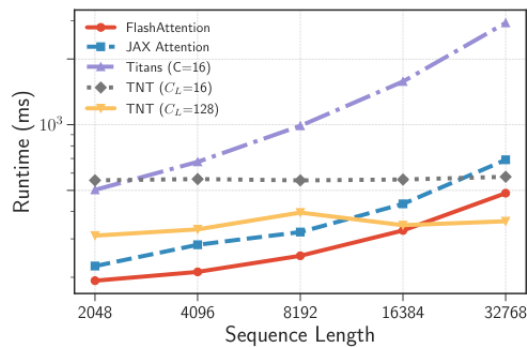


그림 7.5. TNT의 계층적 chunk 구성이 순수대형-chunk/소형-chunk 대비 runtime에서 갖는 위치를 보여주는 비교 그래프 (출처: TNT, Figure 4, [11])

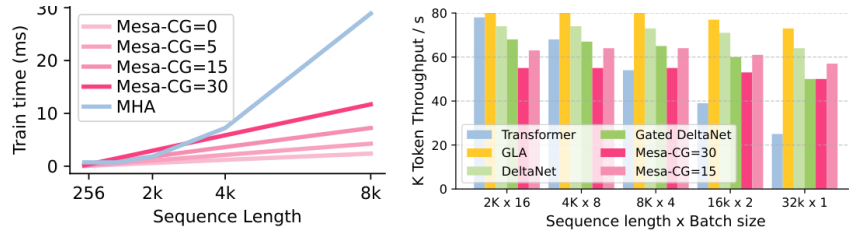


Figure 2: Train and inference time of a Mesa layer using different number of CG steps. *Left:* Train time of a single Mesa layer on a TPUv5: output the entire sequence, compute the cross entropy loss, and gradients w.r.t. layer parameters. We use batch size of 4, key size of 128 and 8 heads. *Right:* Token throughput (in thousands) when training 1B parameter models on a H100 GPU. We compare a Flash-Attention-2 (Dao, 2023) transformer implementation with a triton-based chunkwise parallel implementation of RNN models, including the MesaNet which uses 30 or 15 CG steps across all layers. All models use a key size of 128 and share the same backbone, see Appendix G. We observe competitive token throughput on H100s of the MesaNet despite using substantially more flops.

그림 7.6. MesaNet 의 CG solver step 수와 wall-clock time 의 관계 — step 을 늘릴수록 test-time 최적화 정확도는 오르지만 그만큼 추가시간 ·compute 를 소모하는 trade-off (출처: MesaNet, Figure 2, [19])

ParaRNN(2510.21450) 은 이 계보를 LLM 규모로 밀어붙인다. 시퀀스 전체의 비선형 재귀를 하나의 방정식계로 캐스팅하고 Newton iteration 을 custom parallel reduction 과 결합해, naive 순차 대비 최대 $665\times$ speedup 을 보고하며 7B 파라미터 LSTM/GRU 변형을 Transformer·Mamba2 에 필적하는 perplexity 로 학습한다 [37]. 다만 $665\times$ 는 커널 마이크로벤치의 상한이며, 실제 end-to-end 학습에서의 이득은 이보다 보수적으로 읽어야 한다. [b]

Predictability(2508.16817) 는 이 갈래에 이론적 경계를 준다. 병렬 solver 의 성패는 시스템 dynamics 의 예측 가능성에 달려있으며, 이는 merit function 의 조건수 — Polyak-Łojasiewicz(PL) 상수 — 로 정량화되고, 이 PL 상수는 최대 Lyapunov 지수 (LLE) 로 측정되는 dynamics 의 예측 가능성에 의해 지배된다 [38]. 결론이 날카롭다: **DEER 류 병렬화는 예측 가능한 dynamics 에서 순차 대비 order-of-magnitude 빠르지만, 예측 불가능한 dynamics 에서는 order-of-magnitude 느리다** [38]. 즉 “비선형 재귀를 병렬로 풀 수 있는가” 는 무조건이 아니라 시스템의 수축성에 조건부다 — 이 장에서 유일하게 **원리적 한계**를 명시하는 결과다.

MesaNet(2506.05233) 은 같은 solver 관점을 표현력 쪽으로 쓴다. 매 시점에서 지금까지 본 토 큰에 대한 in-context regression 을 **conjugate gradient(CG)** 로 최적점까지 푼다 — “locally optimal test-time training” [19]. 기존 하드웨어-효율 chunkwise 구현 위에서 CG 의 forward/backward 를 chunk 단위로 병렬 실행하는, 수치적으로 안정한 Mesa layer 를 만들되, CG step 수를 조절해 test-time compute 를 배분할 수 있게 한다.

이 그림은 solver 갈래의 본질적 trade-off 를 드러낸다. CG step 을 늘리면 각 토큰의 test-time 최적화가 최적점에 가까워져 긴 문맥 perplexity 가 낮아지지만, 그만큼 wall-clock time(즉 inference compute) 이 늘어난다 [19]. 즉 병렬 solver 는 “직렬성을 깨는 대가로 compute 를 쓰는” 축이며, LaCT 의 “chunk 를 키워 compute 를 아끼는” 축과 정확히 대칭이다.

커널 레벨 — Comba 와 Tiled Flash Linear Attention

마지막 층위는 알고리즘이 아니라 **커널**이다. 아무리 chunkwise 형식을 유도해도, 기존 FLA 류 커널은 chunk 크기가 제한적이라 많은 중간 state 를 HBM 에 materialize 해야 하고, 이는 arithmetic intensity 를 떨어뜨리고 IO 비용을 키운다 [39]. Tiled Flash Linear Attention(TFLA) 은 chunk 내부에 두 번째 레벨의 시퀀스 병렬화를 도입해 임의로 큰 chunk 를 허용한다 [39]. 두 층의 병렬성 (chunk 간 + chunk 내 tile) 이 materialize 할 state 수를 줄여, 같은 chunkwise 수식을 IO-효율적으로 실행한다. TFLA 기반 mLSTM 커널은 고도로 최적화된 FlashAttention·Linear Attention·Mamba 커널을 능가하며, sigmoid input gate 를 쓴 mLSTMsig 가 1.4B 규모 (160M/400M/1.4B, DCLM) 까지 성능 손실 없이 더 빠르다고 보고한다 [39].

표: solver·커널 갈래 요약

방법	직렬성 파괴 메커니즘	보고 이득	조건/대가
ParaRNN [37]	Newton + parallel reduction	순차 대비 최대 665×	Newton 안정화 필요
DEER [36] / Predictability [38] MesaNet [19]	fixed-point + associative scan 토큰별 CG 최적화	예측 가능 시 OoM 빠름 낮은 ppl· 긴 문맥 우위	예측 불가 시 OoM 느림 (PL 조건) inference compute ↑
TFLA [39]	chunk 내 2-level tiling	FlashAttn/Mamba 커널 능가	커널 구현 복잡도
Comba [35]	단일 역행렬 WY(Triton)	GDN 대비 forward +40%	bilinear 구조 한정

이 커널 갈래가 정확도를 지키는 방식은 가장 명확하다 — TFLA 와 Comba 커널은 수학적으로 동일한 chunkwise 형식을 더 빠르게 실행할 뿐, 근사를 도입하지 않는다. 정확도는 이미 대수(WY) 나 solver(CG) 가 보장했고, 커널은 그 형식을 하드웨어에 최적 사상한다.

정리: 직렬성을 깨는 세 축, 그리고 서로 다른대가

이 장의 해법들은 하나의 만능 열쇠가 아니라, 같은 병목을 서로 다른층위에서 공격하는 세 축이다. (축 1) **chunk 확대** (LaCT·TFLA) 는 재귀 단위를 성기게 만들어 util 을 끌어올리고, 남은 여유를 state 용량 (LaCT 의 40%) 이나 계층 memory(TNT) 로 되사서 정확도를 복원한다. (축 2) **대수적재작성** (DeltaNet·GLA·Comba) 은 재귀를 Householder/WY 항등식으로 접어 병렬 행렬곱으로 바꾸되출력을 무손실로 보존한다. (축 3) **수치 solver** (DEER → ParaRNN·MesaNet) 는 비선형 재귀조차 병렬로 풀지만, Predictability 가 보인 대로 그 성패는 시스템의 PL 조건에 달려있고 compute 를 추가로 요구한다.

skeptic 관점의 결론: “util 70%”, “665×”, “17×”, “+40%” 같은 헤드라인수치는 각각 특정 modality·커널·마이크로벤치의상한이며 end-to-end 학습 이득과 동일시하면 안 된다. 그럼에도 방향성은건고하다 — TTT 의 표현력을 살리려면 갱신 단위를 키우거나 (축 1), 재귀를대수로 접거나 (축 2), solver 로 병렬화 (축 3) 해야 하며, 세 축은 상호배타적이지 않고 (LaCT 가 하이브리드 window 와, TNT 가 계층 memory 와 결합하듯) 조합될 때 가장 강력하다.

Sources: - LaCT / Test-Time Training Done Right (2505.23884) - Parallelizing Linear Transformers with the Delta Rule (2406.06484) - Gated Linear Attention Transformers with Hardware-Efficient Training (2312.06635) - TNT: Improving Chunkwise Training for Test-Time Memorization (2511.07343) - Comba: Improving Bilinear RNNs with Closed-loop Control (2506.02475) - DEER / Parallelizing non-linear sequential models over the sequence length (2309.12252) - ParaRNN (2510.21450) - Predictability Enables Parallelization of Nonlinear SSMs (2508.16817) - MesaNet: Locally Optimal Test-Time Training (2506.05233) - Tiled Flash Linear Attention (2503.14376)

8. 해법 ③ —serving/P/D batch inference 해결

왜 serving 이 TTT·recurrent 계열의 마지막관문인가

앞선 두 장에서 학습·아키텍처 차원의 효율을 다뤘다면, 이장은 **배포 (serving)** 시점에서 발생하는 병목을 정면으로 본다. TTT(test-time training) 계열과 그 사촌인 linear-attention·SSM(state

space model) 계열은 공통적으로하나의 구조적 성질을 공유한다. **decode 시점에 상태 (state) 를 제자리 (in-place) 로 갱신한다**는 것이다. Transformer의 KV cache가 “append-only” 로그라면, recurrent 상태는 매 토큰마다덮어쓰기 (overwrite) 되는 단일 슬롯이다. 이 차이가 serving 스택 전체를재설계하게 만든다.

문제의 뿌리는 세 가지다. 첫째, **batching 붕괴**: prefill 배치는여러 요청의 attention 을 나란히 세워 GEMM 으로 처리하지만, 각 요청이 서로다른 시점의 상태를 in-place 로 쓰려 하면 write 들이 충돌한다. 순차 (sequential) 실행은 정확하지만 느리고, naive batching 은 요청 상태를오염 (corrupt) 시킨다 [23]. 둘째, **prefix caching 무력화**: Transformer 는 공유 prefix 의 KV 를 부분적으로 잘라 재사용 (rolling back) 할수 있지만, recurrent 층의 in-place 갱신은 중간 지점을 복원할 수 없어 **exact-match hit 만** 허용된다 [24]. 셋째, **P/D(prefill/decode) 분리의 재정 의**: KV cache 를 전송하던 disaggregation 파이프라인이, 이제는 크기가 훨씬 작지만의미가 다른 recurrent 상태를 전송해야 한다.

이 장은 이 세 병목을 각각 겨냥한 네 갈래의 해법 — (1) write 인지 batching(RW-TTT), (2) hybrid prefix caching(Marconi·Sparse Prefix·HYPIC), (3) IO-aware 상태 이동 (KVBuffer), (4) 최소상태·hybrid 스택 (In-Place TTT·Nemotron-H·Kimi Linear) — 을 논리적으로따라간다. 핵심 질문은 매번 동일하다. **왜 이 설계가정합성 (correctness) 을 깨지 않으면서 처리량을 올리는가?**

RW-TTT: read/write 를분리하면 batch 가살아난다

RW-TTT 의 출발점은 단순한 관찰이다. TTT decode 에서 각 요청이 수행하는연산은 두 종류다. 현재 상태를 **읽어 (READ)** 출력을 내는 부분과, gradient step 으로 상태를 **갱신 (WRITE)** 하는 부분. READ 는 side-effect 가 없어 서로 다른 요청을 얼마든지 함께 batch 해도 안전하다. 충돌은 오직 WRITE 에서, 그것도 **같은 상태 (owner) 에** 서로 다른요청이 쓸 때만 발생한다. 여기서 상태란 fast weight, low-rank delta, 또는 streaming learner state 처럼 요청이 소유 (request-owned) 하는 갱신 대상을가리킨다 [23].

RW-TTT 는 이를 **owner/version/READ-WRITE 태깅**으로 형식화한다. 모든 decode step 에 owner 식별자, version 카운터, READ/WRITE 효과 태그를 붙이고, 배치 스케줄러는 연산을 phase 단위로 분해한다. **호환되는 (compatible) phase 만 함께 batch** 하되 — 즉 READ 끼리, 또는 서로 다른 owner 에 대한 WRITE 끼리 — WRITE 는**자신의 owner 상태에만 commit** 한다. version 카운터는 한 요청이읽은 상태가 그 사이 다른 write 로 오염되지 않았음을 보장하는 낙관적동시성 (optimistic concurrency) 의 역할을 한다. 이렇게 하면 순차 처리에서낭비되던 GPU 점유율을, correctness 를 한 치도 양보하지 않고 되찾는다.

위 그림은 RW-TTT 의 데이터 경로를 보여준다. 요청 스트림이 들어오면스케줄러가 각 step 을 READ/WRITE 로 분류하고, READ 는 넓게합쳐 GEMM 효율을 끌어올린 뒤, WRITE 만 owner 단위로 좁게 직렬화한다. 이구조가 중요한 이유는, **batch 이득의 대부분이 READ 에서 나오는 반면 correctness 위험은 WRITE 에 집중되어** 있기 때문이다. 둘을 분리하는 순간, 위험을 격리한 채 이득만 취할 수 있다.

성능은 **한 GPU 에서 8 개의 fast-weight In-Place-TTT 스트림**을 돌린조건에서 보고된다. 집계 (aggregate) 처리량 **274.61 tok/s**, 순차 (sequential) 대비 **9.31×**, 그리고 같은 메모리 예산에서 스트림별 복제 (per-stream replicas) 대비 **3.44×**이며, 장문 벤치마크 RULER 에서행동 (behavior) 을 보존하고 owner/version 검사를 통과한다고명시된다 [23][a]. 스케일링 특성은 아래 그림이 두 축으로 나눠보여준다.

이 스케일링 그림이 논지에 결정적인 이유는, RW-TTT 의 이득이 **동시스트림 수 (concurrency) 가 늘수록 커진다**는 점을 드러내기 때문이다. 스트림이 하나뿐이면 READ 를 합칠 대상이 없어 순차와 다를 바 없지만, 스트림이 쌓일수록 READ phase 의 batch 폭이 넓어져 GPU 가 포화에가까워진다. 즉 RW-TTT 는 고부하 멀티테넌트 서버에서 정확히 필요할 때 힘을쓴다.

WRITE 경로 자체도 커널 수준에서 최적화된다.

이 커널 그림은 WRITE 가 직렬 병목임에도 불구하고, fused Triton 연산자로 owner 별 갱신을 좁고 빠르게 처리해 직렬 구간의 절대 길이를 줄였음을보여준다. batching 이 “폭” 을 넓히는 전략이라면, 이 커널 최적화는 남은직렬 “길이” 를 줄이는 보완책이다. 두 축이 곱해져 9.31× 가나

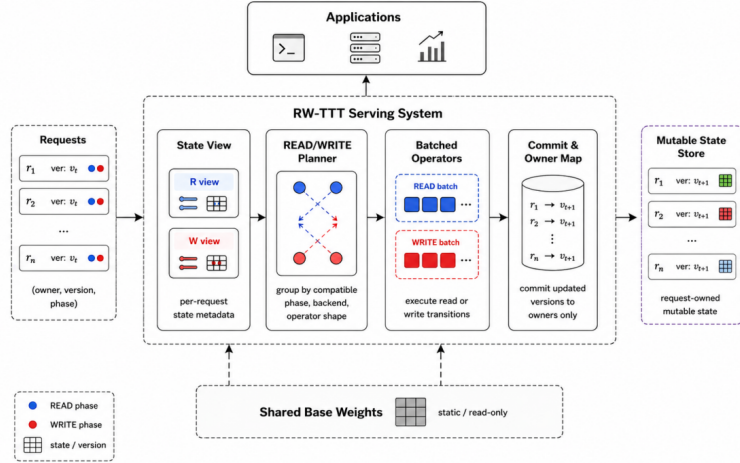


Figure 2: RW-TTT serving architecture. Requests carry owner-versioned mutable state alongside ordinary KV cache. The planner automatically groups compatible READ/WRITE transitions by backend type, operator shape, placement, and valid owner version; batched operators return outputs and updated state through the owner map. The figure shows serving semantics, not the backend update rule.

그림 8.1. RW-TTT serving 아키텍처: owner/version 태깅으로 READ phase 는 대량 batch, WRITE phase 는 owner 별로 분리 commit 하는 스케줄 경로 (출처: RW-TTT, Figure 2, [23])

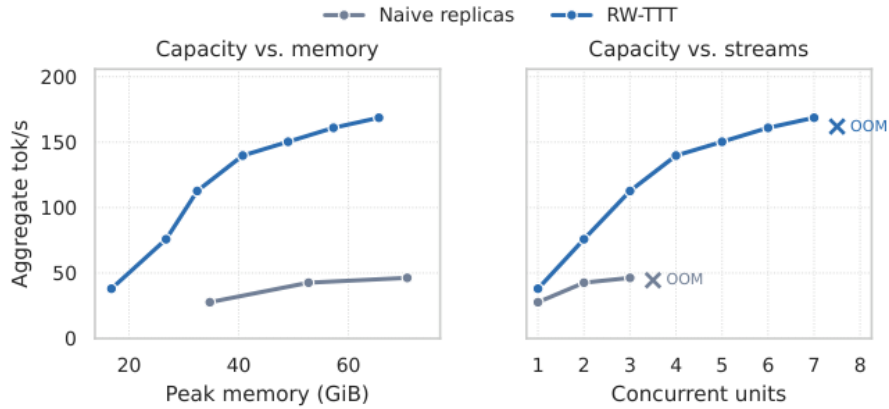


Figure 3: Scaling under a 16K prompt. Left: feasible runs ordered by peak memory. Right: same runs ordered by concurrent units. RW-TTT keeps request-private streams inside one read-write runtime.

그림 8.2. 용량 (capacity) 대 메모리, 그리고 동시 스트림 수에 따른 처리량 스케일링 곡선 (출처: RW-TTT, Figure 3, [23])

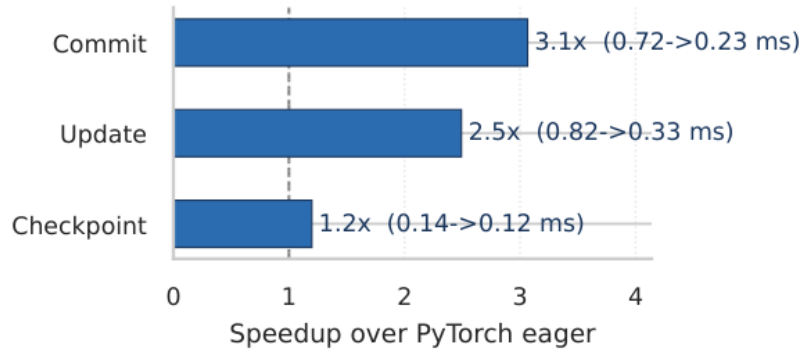


Figure 4: WRITE-side operator speedups from Triton kernels over PyTorch eager. Parentheses show latency before and after kernelization in milliseconds.

그림 8.3. Triton 으로 구현한 WRITE-side 연산자의 speedup 막대그래프 (출처: RW-TTT, Figure 4, [23])

온다.

표: RW-TTT 핵심 지표 (1 GPU, 8 fast-weight In-Place-TTT 스트림)

지표	값	baseline / 비교
처리량 speedup	9.31×	순차 (sequential) 대비
처리량 speedup	3.44×	per-stream replicas 대비 (동일메모리 예)
절대 처리량	274.61 tok/s	8 스트림 집계 (aggregate)
correctness 검증	RULER 행동 보존 + owner/version 검사통과	—
batch 대상	READ phase / 서로 다른 owner 의 WRITE	호환 phase 만

skeptic 관점에서, $9.31\times$ 라는 배수는 **순차 baseline 의약함**에 크게 의존한다. 순차 실행은 GPU 를 극도로 underutilize 하므로 어떤 batching 이든 큰 배수가 쉽게 나온다. 이 점을 논문 자체도 의식해, 더공정한 비교인 **per-stream replicas 대비 $3.44\times$** 를함께 보고한다 — 즉 “메모리를 똑같이 쓰되 요청마다 모델을 복제해병렬화한” 강한 baseline 에 대해서도 이득이 남는다는 뜻이다. 다만 절대치 274.61 tok/s 는 특정 fast-weight 구성 · 8 스트림 · 단일 GPU 라는 좁은 운영점의값이므로, 다른 모델 크기 · 시퀀스길이 · 하드웨어로 일반화하려면 별도 확인이 필요하다 [c].

hybrid prefix caching: exact-match 만 되는 세계의 캐시경제학

Marconi 는 문제를 정면으로 규정한다. hybrid LLM 은 attention 층과 recurrent(SSM) 층을 섞는데, recurrent 층의 in-place 갱신 때문에 부분 prefix 에 대한 롤백이 불가능하고 오직 **exact-match hit 만** 가능하다 [24]. Transformer 라면 “공유 prefix 가 90% 겹치니 그 부분 KV 를재사용” 할 수 있지만, hybrid 는 recurrent 상태가 단일 슬롯이라 한 토큰만달라도 그 층을 다시 계산해야 한다. 이 제약 아래서 캐시 슬롯은 훨씬귀하고, **무엇을 admit 하고 무엇을 evict 할지**가 처리량을 좌우한다.

Marconi 의 두 정책이 핵심이다. **admission** 은 요청 overlap 을 radix tree 로 추적하며, 여러 hit 시나리오의 분류 (taxonomy) 위에서 각 후보의재사용 가능성을 예측해 재사용 가능성이 높은 항목만 캐시에 들인다 (예: 순수 입력인 system prompt 는 높고, 입력-출력이 섞인 대화이력은 낮음). **eviction** 은 recency 만 보는 LRU 를 넘어, 각 항목이 hit 시 절약해주는 연산량 (compute savings) 을 그 메모리 발자국 (memory footprint) 대비로평가하는 **FLOP-aware** 로

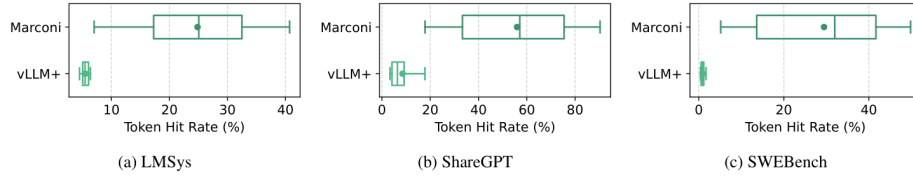


Figure 7. Comparison with vLLM+. With judicious cache admission, Marconi utilizes the limited cache space to retain states with higher

그림 8.4. Marconi 대 vLLM+ 의 token hit rate 비교 (캐시 크기별) (출처: Marconi, Figure 7, [24])

간다. recurrent 상태는 크기가 작지만 재계산 비용은 시퀀스 길이에 비례해 크므로, **작지만 비싼 항목을 지키는 FLOP-aware 관점**이 정확히 이 아키텍처에 들어맞는다. 논문은 특히 긴 컨텍스트, 높은 SSM 층 비율, 큰 SSM 상태 차원일수록 Marconi 의이득이 커진다고 밝히는데 [24], 이는 최근 모델 추세와 정확히 겹친다.

이 그림이 논지에 중요한 이유는, hit rate 이득이 **운영점 (캐시크기 · workload) 에 따라 크게 달라진다는** 점을 보여주기때문이다. 캐시가 지나치게 작거나 크면 어떤 정책이든 차이가 줄지만, 메모리 압박이 실재하는 중간 영역에서 FLOP-aware admission/eviction 이 가장 큰 격차를 만든다. 이는 정책이 “현실적 운영점” 에서 값어치를 한다는 뜻이다.

TTFT 분포 그림이 결정적인 까닭은, 평균이 아니라 **꼬리 (P95)** 를본다는 데 있다. hit rate 가 오르면 재계산이 줄고, 재계산이 줄면 큐잉이줄어 최악 지연이 압축된다. Marconi 는 최신 prefix caching 시스템 대비 최대 $34.4\times$ 높은 token hit rate 를, 이를 통해 최대 71.1%(617ms) 낮은 TTFT 를 달성한다고 보고한다 [24][a].

표: hybrid·recurrent prefix caching 계열요약 (arXiv id 교정 반영)

시스템	핵심 메커니즘	보고 수치	baseline
Marconi [24]	radix-tree admission + FLOP-aware eviction	token hit rate 최대 $34.4\times$, TTFT 최대 - 71.1%(- 617ms)	최신 prefix caching(vLLM+/SGLang+ 계열)
Sparse Prefix Caching [40]	희소 (sparse) checkpoint 위치에 exact recurrent state 저장 → hit 시 가장 깊은 checkpoint 에서 재개, 나머지 suffix 는 정확히 재계산	재계산량 절감 (정확도 무손실)	dense per-token 재사용 가정 시스템
HYPIC [25]	position-independent caching(PIC): linear-attention 층의 segment-cumulative transition operator + zero-start end-state 를캐시	TTFT $2.45\times \downarrow$, peak throughput 최대 $2.0\times \uparrow$	기존 hybrid serving

이 표는 초안의 arXiv id 오배정을 바로잡은 것이다. Sparse Prefix Caching(2605.05219, Shirokikh & Nikolenko) 의 핵심은 초안이 적은 “근사 (approximate) 매칭” 이 아니라, **sparse checkpoint 에저장된 정확한 recurrent state 에서 재개하고 남은 suffix 를 정확히재계산하는**

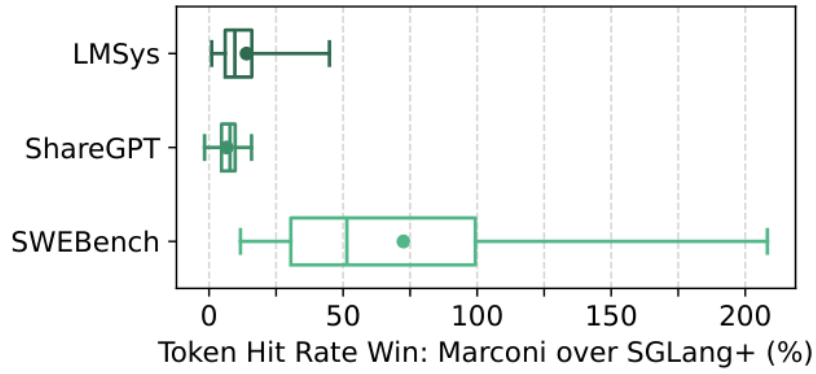


Figure 8. Comparison with SGLang+, which uses LRU as its eviction policy. Marconi balances recency and FLOPs efficiency, improving the token hit rate significantly, especially for workloads with longer context.

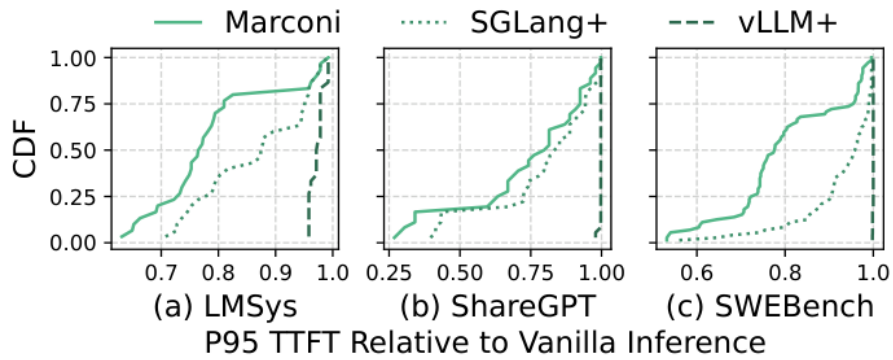


Figure 9. Distribution of P95 TTFT relative to no prefix caching.

그림 8.5. P95 TTFT 분포: Marconi 가 꼬리 지연 (tail latency) 을 어떻게 압축하는지 (출처: Marconi, Figure 9, [24])

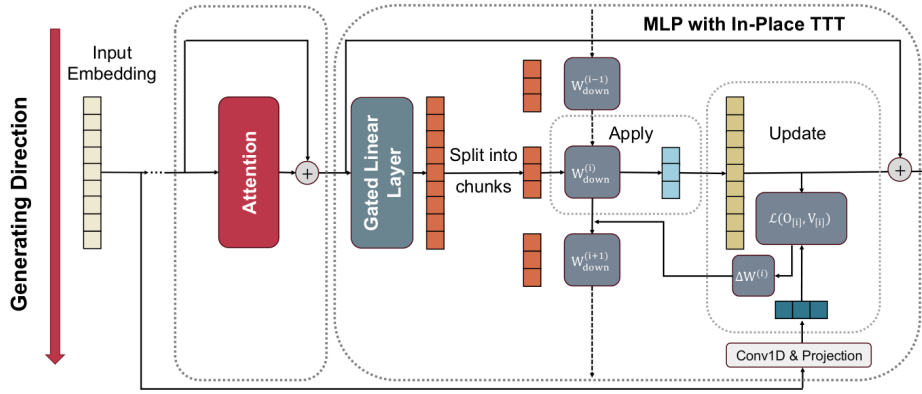


Figure 1 The overall framework of our In-Place Test-Time Training. The module operates sequentially on input chunks. For each chunk, the current fast weights are first *applied* to the intermediate activations $\mathbf{Z}$ to produce the output. Then, these weights are *updated* using the activations $\mathbf{Z}$ and a value $\mathbf{V}$ derived from the token embeddings. This "apply-then-update" cycle allows the model to dynamically adapt to incoming context in a strictly causal manner.

그림 8.6. In-Place TTT 프레임워크: 새 모듈 없이 기존 MLP의 최종 projection을 fast weight로 삼아 제자리 갱신 (출처: In-Place TTT, Figure 1, [22])

것이다 — 즉 정확도 손실 없이 재계산량만 줄인다. HYPIC(2607.01299)는 계층 인지 인덱싱이 아니라 **position-independent caching(PIC)**을 hybrid-attention에 이식한것으로, per-token KV 재사용 원시연산이 per-request recurrent state 로전이되지 않는 문제를 linear-attention 층의 “segment-cumulative transition operator”라는 대수적 원시연산을 캐시함으로써 우회한다 [25][a]. 세 시스템 모두 매우 최신 (2024 말~2026) 연구이므로 workload·컨텍스트 길이·하드웨어의존성이 크고, 단일 headline 배수를 일반화해선 안 된다.

IO-aware와 최소 상태: 이동 비용과 상태 크기를 함께 줄인다

hit rate를 올려도, 상태를 GPU 메모리 계층 사이로 옮기고 계산 형태를 바꾸는 **IO 비용**이 남는다. KVBuffer(2605.19049)는 linear attention을 위한 IO-aware serving으로, chunkwise decoding·speculative decoding 검증·short-context decoding 등 서로 다른 decoding 시나리오에 맞춰 **유연한 계산 형태**를 취해 불필요한 메모리 접근을 줄인다 [27][b]. recurrent 상태는 KV cache보다 작지만, 시나리오마다 최적의 계산 형태 (재계산 vs 상태 읽기)가 다르므로, 그 선택을 IO 비용 기준으로 스케줄하는 것이 요점이다. (초안이 이 항목에 붙인 id 2607.01299는 HYPIC의 것이므로 교정했다.)

또 하나의 방향은 **상태 자체를 최소화**하는 것이다. In-Place TTT(2604.06169)는 TTT를 별도 모듈 없이 기존 구조에 넣는다 — 흔한 MLP 블록의 **최종 projection 행렬을 fast weight로 재 활용**하고, chunk-wise 갱신 메커니즘으로 계산 효율을 확보하며, 일반적 reconstruction 목표 대신 **Next-Token-Prediction에 정렬된 목표**를 쓴다 [22]. (ICLR 2026 Oral.)

이 프레임워크 그림이 중요한 이유는, “상태를 복제해 롤백 가능하게 만드는” 전통적 안전장치를 두지 않고 **기존 가중치를 제자리에서 갱신**함으로써 별도 상태 버퍼와 그 관리 오버헤드를 없앤다는 설계 철학을 드러내기 때문이다. 이는 Marconi가 지적한 “in-place라서 부분 재사용 불가”라는 제약을 **비용이 아니라 자원**으로 재해석한 것으로 읽을 수 있다 [b] — 어차피 롤백하지 않을 상태라면, 사본을 없애 메모리와 대역폭을 절약한다.

효율 그림은 상태 최소화가 처리량으로 환산되는 경로를 시사한다. 상태·모듈이 작아지면 같은 메모리로 더 큰 batch를 담을 수 있고, batch가 커지면 GPU 점유율이 올라 tok/s가 오른다 [22][b]. 실제로 앞의 RW-TTT 실험이 fast-weight In-Place-TTT 스트림을 대상으로 삼았다는 점은 우연이 아니다 — In-Place TTT가 “상태 크기”를 줄여 놓으면 RW-TTT가 그 위에서 “batch 폭”을, KVBuffer가 “이동·계산 형태”를 최적화한다. 세 레버는 상충하지 않고 곱해진다.

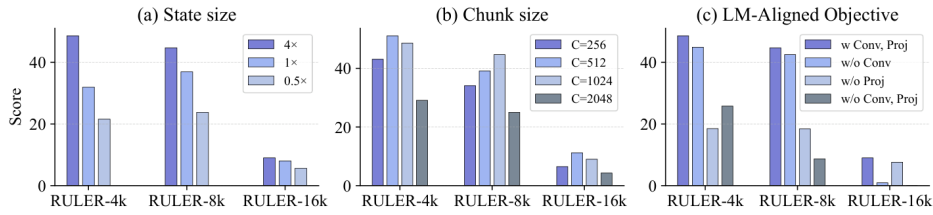


Figure 3 Ablation studies on the key design choices of the In-Place TTT framework, evaluated on the RULER benchmark with a 1.7B parameter model. The plots illustrate the impact of: (a) State size, showing that performance improves as the state size scales; (b) Chunk size, demonstrating a performance trade-off where intermediate sizes (e.g., 512, 1024) are optimal; and (c) The LM-Aligned Value objective, confirming that both the convolution (w Conv) and the projection (w Proj) are crucial.

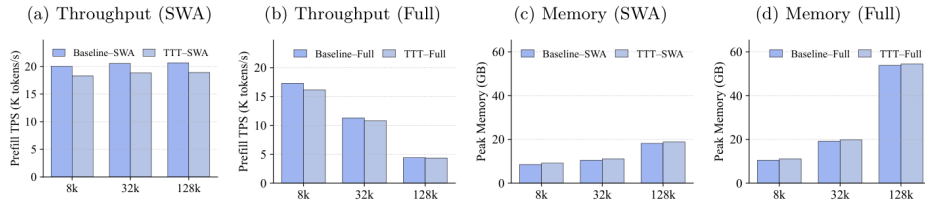


Figure 4 Efficiency analysis of In-Place TTT. Both prefill throughput (a, b) and peak memory (c, d) metrics are presented for 4B models with Sliding-Window Attention (SWA) and Full Attention at various context lengths. Our In-Place TTT introduces negligible overhead in practical scenarios.

그림 8.7. In-Place TTT 의 효율: 메모리 · 계산 관점의 개선 (출처: In-Place TTT, Figure 4, [22])

hybrid 스택과 recurrent-aware P/D 분리

마지막 갈래는 아키텍처 선택으로 serving 부담을 애초에 줄이는 것이다. **Nemotron-H** 는 self-attention 층 대부분을 Mamba-2 로 대체해, decode 시 상수 메모리 · 상수 연산에 가까워진다. 56B-Base 는 **Mamba-2 54 · MLP 54 · self-attention 10**(총 118 층) 으로 구성된다 [18][a]. NVIDIA 는 이 계열이 유사규모의 순수 Transformer 대비 **최대 3× 높은 추론 처리량**을 낸다고 보고하는데, 이는 abstract 수준의 **패밀리 전체** 주장이다 [a]. 세부 벤치마크에서는 긴 컨텍스트 (입력 65,536 · 출력 1,024, H100) 에서 **47B 변형이 Qwen-2.5-72B·Llama-3.1-70B 대비 약 2.9×** 로 보고된다 — 즉 “3×” 를 56B 한 모델에 못박기보다 패밀리헤드라인으로 읽는 것이 정확하다 [b]. 소수의 attention 층만 남기므로 KV cache 가 급감하고, 그 여유가 곧 더 큰 batch 로 환산된다.

Kimi Linear 는 같은 철학을 더 밀어붙인다. KDA(Kimi Delta Attention, per-channel gating 으로 Gated DeltaNet 을 확장) 를 full attention 과 **3:1** 비율로 교차 배치해, 1M 컨텍스트 장문 생성에서 **KV cache** 를 **최대 75% 절감**하고 **최대 6× decode 처리량**을 얻는다 [28][a]. 남긴 1/4 의 full attention 이 전역정보 흐름을 보존하는 동안, 3/4 의 KDA 가 KV 를 지운다. (초안이던 붙였던 “2~3× prefill 가속” 은 abstract 에 명시되지 않아 제거했다.)

표: hybrid 스택의 KV · 처리량 이득

모델	구성	KV/메모리	처리량	조건
Nemotron-H-56B [18]	Mamba-2 54 / MLP 54 / attn 10 (118 층)	attn 층만 KV 보유	패밀리 최대 3× (47B 는 세부 벤치서 ~2.9×)	65,536 in / 1,024 out, H100, vs Qwen-2.5-72B·Llama-3.1-70B
Kimi Linear-48B-A3B [28]	KDA:full = 3:1	최대 - 75% KV	decode 최대 6×	1M 컨텍스트

이 hybrid 스택은 **P/D 분리 (disaggregation) 의 의미를 바꾼다**. 순수 Transformer 의 disaggregation 은 prefill 노드가 만든 대용량 KV cache 를 decode 노드로 전송하는 것이 핵심이었다. hybrid-recurrent 에서는 전송 대상이 **작은 recurrent 상태 + 소수 attention 층의 KV** 로 바뀐다. 전송량이 줄어드는 것은 이득이지만, exact-match 특성 때문에 prefill 결과를 **부분적으로** 이

어붙이기 어렵다는 새로운 제약이 생긴다. 그래서 recurrent-aware disaggregation 은 KV 전송 최소화보다 **상태 owner 의 이관 (handoff) 과 version 일관성**을 핵심 과제로 삼게 되며, 이 지점에서 RW-TTT 의 owner/version 태깅이 disaggregation 프로토콜과 자연스럽게 맞물린다 [b].

정합성은 어떻게 지켜지는가 — 네 레버의 합

이 장의 네 갈래는 서로 다른 레버를 당기지만, 정합성을 지키는 논리는 하나로 수렴한다. 위험은 **in-place WRITE 에 집중되어 있으므로, 그것만 격리하고 나머지는 자유롭게 최적화한다**. RW-TTT 는 WRITE 를 owner 별로 직렬화해 batch 를 열고, Marconi·Sparse Prefix·HYPIC 은 exact-match(또는 정확한 suffix 재계산·transition operator 캐시) 로 캐시의 correctness 를 보존하며, In-Place TTT 는 상태를 기존 MLP 에 흡수해 최소화하고, hybrid 스택은 WRITE 대상 자체를 소수 층으로 줄인다. 배수 ($9.31 \times 3.44 \times 34.4 \times 6 \times 3 \times 2.45 \times$) 들은 각기 다른 baseline·조건에서 나온 값이라 곱하거나 나란히 비교해선 안 되지만, **레버들이 직교 (orthogonal)**하다는 점은 실무적으로 중요하다 — batch 폭, 캐시 hit, 이동·계산 형태, 상태 크기를 동시에 당길 수 있다.

skeptic 정리: (1) 이 계열의 상당수 (RW-TTT·In-Place TTT·KVBuffer·Sparse Prefix·HYPIC) 는 매우 최신 (2024 말~2026) 연구로, 독립 재현이 아직 폭넓게 축적되지 않았다 — 다만 RW-TTT 는 순차뿐 아니라 per-stream replicas 대비 $3.44 \times$ 를 함께 보고해 baseline 약함 논란을 일부 방어하고, In-Place TTT 는 ICLR 2026 Oral 로 검증 신뢰도가 상대적으로 높다. (2) Marconi·Nemotron-H·Kimi Linear 는 공개 아티팩트가 있으나 수치는 **workload·컨텍스트 길이·하드웨어 의존성**이 크다 (Marconi 의 $34.4 \times$ 는 긴 컨텍스트·높은 SSM 비율에서, Kimi 의 $6 \times$ 는 1M 컨텍스트 한정, Nemotron 의 $3 \times$ 는 패밀리 헤드라인). 단일 headline 배수를 일반화하는 순간과장이 된다. 정직한 요약은 이렇다 — **recurrent/TTT serving 의 병목은 in-place WRITE 에서 나오며, 이를 서로 다른 층위에서 격리하는 설계들이 각자의 조건에서 유의미한 이득을 보고한다**.

Sources: - RW-TTT (arXiv:2605.28053) - Marconi (arXiv:2411.19379) - Sparse Prefix Caching (arXiv:2605.05219) - KVBuffer (arXiv:2605.19049) - HYPIC (arXiv:2607.01299) - In-Place TTT (arXiv:2604.06169) - Nemotron-H (arXiv:2504.03624) - Kimi Linear (arXiv:2510.26692)

Part III 정리. 해법도 세 방향 — (규모) 큰 Transformer 를 **distill** 로 물려받기, (학습) 순차성을 큰 **chunk·병렬화**로 접기, (서빙) 배치·캐시 프리미티브 자체를 재설계. 공통 조건은 ‘표현력을 지키며 순차성·용량 비용을 줄인다’.

Part IV — memory device 집중조명

9. memory device 집중 조명 ① — 한계

memory device 라는 렌즈 — 두 개의 서로 다른 “메모리” 가 겹치는 곳

이 장은 하나의 단어가 서로 다른 두 개의 물리·수학적 대상을 동시에 가리킨다는 사실에서 출발한다. TTT/Titans 계열의 논의에서 “memory” 는 (i) fast-weight 또는 recurrent state 를 하나의 **저장 장치**로 보는 추상적 대상이면서, 동시에 (ii) 그 state 가 실제로 올라앉는 HBM 이라는 **물리적 장치**이기도 하다. 이 두 얼굴은 독립적으로 진화한 것이 아니라, model-size·training·serving 세 병목이 정확하게 지점에서 서로를 조인다. 신경 메모리의 용량을 키우면 physical memory 의 점유가 커지고, physical memory 의 대역폭이 decode 를 묶으면 다시 신경 메모리를 얼마나 크게 쓸 수 있는지가 제약된다. 이 장의 목표는 두 얼굴이 각각 어디서 무너지는지를 메커니즘 수준에서 따라가는 것이다.

(i) 신경 메모리 — associative memory 로서의 유한용량

신경 메모리를 이해하는 가장 생산적인 렌즈는 associative memory 다. Linear attention 과 TTT 계열의 state 는 본질적으로 key-value 쌍을 outer product 로 누적인 행렬 $S = \sum_i v_i k_i^T$ 이

고, 이는 $d \times d$ 크기의 저장소다. 이 구조에서 “읽기”는 query q 에 대해 $Sq = \sum_i v_i(k_i^\top q)$ 를 계산하는 일인데, 여기서 즉시 문제가 드러난다. 우리가 원하는 것은 q 가 특정 key k_j 와 정렬될 때 대응하는 v_j 만 깨끗하게 나오는 것이지만, 실제로는 나머지 모든 $k_i^\top q$ 항이 **crosstalk(간섭)**로 더해진다. key들이 서로 직교할 때만 간섭이 0이 되고, d 차원 공간에서 완전 직교하는 벡터는 최대 d 개뿐이다. 따라서 이 형태의 신경 메모리 용량은 근본적으로 $\mathcal{O}(d)$ 로 상한된다 — state가 d^2 개의 실수를 담고 있어도, 무손실로 구분 저장할 수 있는 연상 쌍의 수는 차원 d 에 선형이다.

이 상한을 넘어 $N > d$ 개를 밀어 넣으면 어떤 일이 벌어지는가. key들이 더이상 직교할 수 없으므로 각 retrieval은 나머지 $N - 1$ 개로부터 누적간섭을 받고, 이 간섭항의 크기는 저장량 N 이 커질수록 함께 자란다. 즉 **retrieval error가 저장량의 함수로 성장한다**. 이것이 fixed-size state를 가진 모든 모델이 문맥이 길어질수록 초반 정보를 흐리게 기억하는 구조적 이유다: 새 토큰이 들어올 때마다 같은 $d \times d$ 캔버스에 덧칠을 하고, 캔버스의 표현력은 고정되어 있다.

표: 신경 메모리 유형별 용량 스케일링 (개념 정리)

메모리 유형	state 크기	무손실 저장 용량 (대략)	병목
Classical Hopfield (Hebbian)	d neurons, d^2 weights	$\approx 0.14 d$ 패턴	간섭으로 인한 spurious minima
Modern Hopfield [16]	d 차원 연상공간	차원에 지수적 (well-separated 가정)	잘 분리되지 않은 패턴에서 붕괴
Linear attention / outer-product state	$d \times d$	$\mathcal{O}(d)$ (직교 key 수)	crosstalk, 온라인 업데이트
Deep memory (MLP state) [8]	파라미터 $\gg d^2$	depth에 따라 증가	학습 · 연산 비용

여기서 modern Hopfield network [16]는 흥미로운 대조군이다. 이 논문은 continuous state에 대해 softmax 형태의 업데이트를 쓰면 **차원 (연상공간의 차원)에 지수적으로 많은** 패턴을 저장하고, 한번의 업데이트로 지수적으로 작은 retrieval error를 내며 검색할 수 있음을 보인다 [a]. 이는 고전 Hopfield의 선형 용량 (d 차원에서 무손실로는 약 d 개, 랜덤 패턴에서는 $Cd/\ln d$)과 극적으로 대비된다. 다만 skeptic 관점에서 이 “지수 용량”은 패턴들이 서로 충분히 잘 분리되어 있다는 가정 위에서만 성립한다. 논문 자신이 에너지 지형의 극소점 (업데이트의 fixed point)을 세 종류 — 모든 패턴을 평균한 global fixed point, 부분집합을 평균한 metastable state, 단일 패턴을 저장한 fixed point — 으로 분류하는데, 실제 언어 데이터처럼 key들이 조밀하게 뭉친 경우 metastable state로 collapse하면서 여러 패턴이 하나로 뭉개진다. 즉 softmax를 붙여도 간섭 문제가 사라지는 게 아니라, “잘 분리된 영역”으로 밀어낼 뿐이다.

ATLAS [10]는 이 유한 용량 문제를 정면으로 진단한 좋은 예다. 이 논문은 기존 memory-based 접근의 한계를 세 가지로 정리한다: (1) 용량이 메모리 아키텍처와 입력 feature map에 의해 제한되고, (2) 업데이트가 **온라인**이라 메모리가 오직 마지막 입력에 대해서만 최적화되며, (3) fixed-size 메모리를 관리하는 방식이 덜 표현력 있다는 것이다. 위 그림의 recall 실험은 이 진단의 첫 번째 항을 정량적으로 뒷받침한다 — 저장 쌍수가 용량을 넘어서면 정확도가 열화하는 유효 용량의 벽이 존재한다는 것이 핵심이다 [b]. ATLAS는 이를 완화하려고 마지막 토큰이 아니라 **문맥의 슬라이딩 윈도우 전체**에 대해 메모리를 최적화하는 Omega rule과 고차 (higher-order) feature map, 그리고 Muon optimizer 기반 메모리 갱신을 도입하지만, 이는 “용량 상한을 없앤다”기보다 “같은 state를 더 잘 쓴다”는 개선임을 유의해야 한다.

용량이 성능을 얼마나 직접적으로 좌우하는지는 최근 스케일링 연구에서 더 선명하게 드러난다. Test-Time Training Done Right (LaCT) [6]는 nonlinear state 크기를 모델 파라미터의 **최대 40%까지** 키워 state capacity를 실질적으로 늘렸고, 이를 통해 14B auto-regressive video diffusion 모델을 최대 56K 토큰까지, novel view synthesis를 100만 토큰이 넘는 문맥까지 확장했다 [a]. 이 논문의 메시지는 단순하다: **state size가 곧 성능의 상한**으로 작동하며, hardware utilization을 수 배 끌어올린 large-chunk 갱신 (LaCT)이 그 확장을 실용가능하게 만든다는 것.

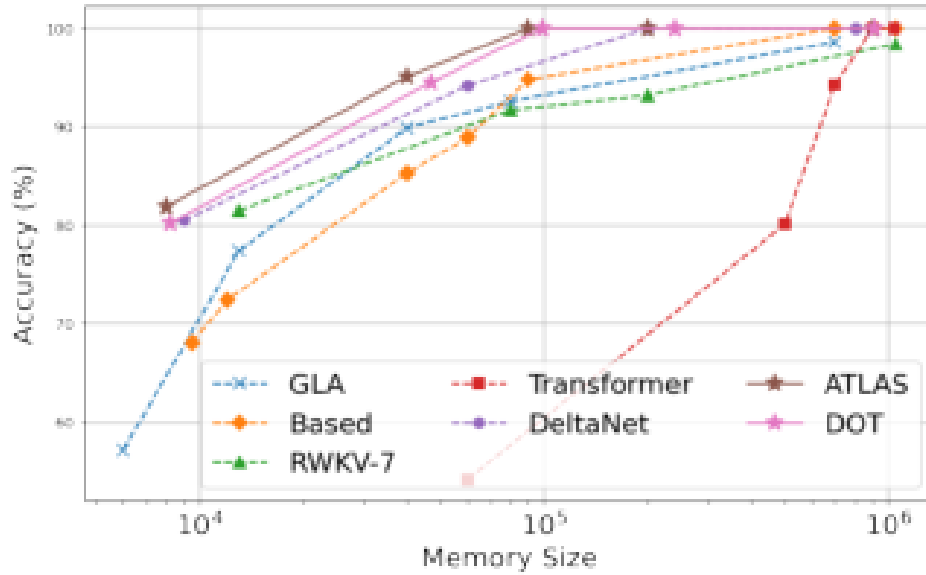


그림 9.1. ATLAS의 associative-memory recall 실험: 저장한 연상쌍의 수가 state 용량을 넘어 서면 recall 정확도가 무너지기 시작하는 지점을 보여주며, 신경 메모리의 유효 용량이 유한함을 실험적으로 확인한다. (출처: Atlas, Figure 7, [10])

용량을 늘리는 또 다른 축은 **깊이**다. Titans [8]는 state를 단일 행렬이 아니라 깊은 비선형 MLP로 두는 neural long-term memory(LMM)를 제안한다. 행렬 state의 용량이 $\mathcal{O}(d)$ 에 묶이는 반면, 깊은메모리는 파라미터 수를 늘려 더 많은 연상을 겹침 없이 저장할 여지를 준다. 논문은 실제로 $L_M \geq 1$ 층 MLP를 메모리로 두되, 2층 이상의 깊은메모리가 실전에서 더 효과적임을 보고한다.

이 그림이 던지는 skeptic 포인트는, 깊이로 산 용량은 공짜가 아니라는 것이다. 깊은 메모리는 test-time에 자기 가중치를 최적화하는 inner-loop를 요구하므로 training과 serving 양쪽의 연산 비용을 함께 밀어 올린다 — 세병목이 얹히는 첫 신호다.

고정 용량이 긴 문맥에서 열화하는 방식

유한 용량의 가장 직접적인 관측 가능한 증상은 문맥이 학습 길이를 넘어설 때의 성능 붕괴다. Conformal-sympow transformer [41]는 symmetric power transformer의 recurrent state가 유한 용량이라 training/evaluation 문맥을 늘리면 정보 보존에 실패하고 성능이 열화한다고 명시한다 [a]. 이들의 처방 — data-dependent multiplicative gating으로 용량을 동적으로 비워 주고 (free up) data-dependent rotary embedding으로 저장 위치를 적응시키는 것 — 은 곧 “고정 캔버스를 어떻게 덜 낭비할까”의 문제다. 같은 증상을 문맥 길이축에서 직접 그리면 다음과 같다.

Mamba-Transformer 하이브리드 분석 [42]은 이 열화가 왜 실무에서 하이브리드로 귀결되는지를 설명한다: SSM 계열의 고정 state는 짧은 문맥에서 효율적이지만 정확한 recall에서는 attention에 밀리고, 그래서 순차 (sequential) 하이브리드는 짧은 문맥에, 병렬 (parallel) 하이브리드는 긴 문맥에 각각 강점을 보인다는 것이다. 나아가 local pattern 위주 작업은 SSM-heavy 구성이, in-context learning-retrieval 위주 작업은 attention 층이 더 많은 구성이 유리하다고 정리한다. 즉 유한 용량 메모리는 attention의 크기만 (길이에 비례해 자라지만) 비싼 KV cache와 역할분담을 하도록 떠밀린다.

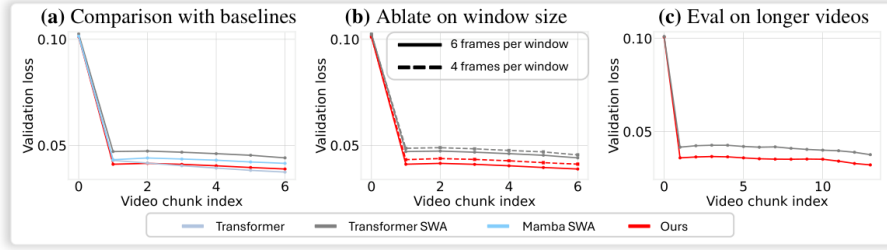


Figure 6: (a) We achieve comparable validation loss to the full-attention baseline and outperform both Mamba with sliding window and sliding window attention baselines. This improvement over SWA is consistent across different window sizes (b) and when evaluating on longer videos (c).

chunk-wise denoising loss across evaluated video frames. We only measure validation loss up to training sequence length. See project website for our autoregressively generated videos ².

5.4 Analysis on Design Choices

In this section, we analyze several key design choices in our model, focusing on both the novel view synthesis and language modeling tasks, where good metrics exist. Specifically, we evaluate the impact of state size (Fig. 7a), test-time optimizers (Fig. 7b), linear versus nonlinear fast weights (Fig. 8a), and per-token recurrence versus chunk-wise recurrence (Fig. 8b). Overall, we find that a large state size, advanced optimizers such as Muon, and nonlinear fast weights significantly improve our model’s performance. For comparing chunk recurrence with per-token recurrence, in a controlled NVS experiment, our linear large-chunk recurrence strategy outperforms linear per-token recurrence with the same state size. For language modeling, where chunk structures are not inherent, our linear large-chunk recurrence variant—while initially underperforming per-token methods like GLA and DeltaNet—surpasses them when combined with a large nonlinear state and the Muon optimizer. We refer the readers to each figure and its caption for more detailed analysis.

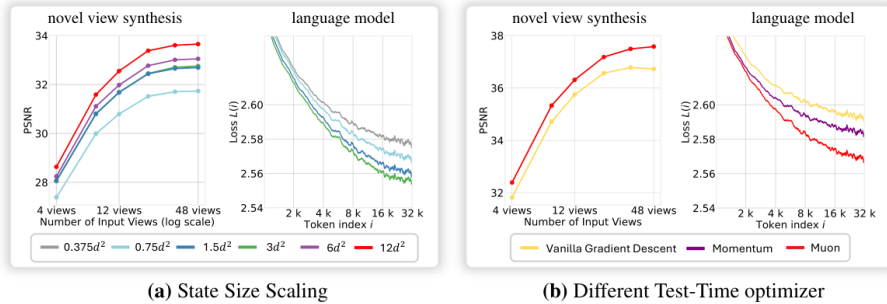


Figure 7: (a) Scaling up the state size consistently improves performance in both novel view synthesis and language modeling tasks. Note, the largest version has state size of $12d^2$ per block, totaling 40% of model weights as fast weights. (b) Comparison of test-time optimizers demonstrates Muon’s surprising effectiveness over Vanilla Gradient Descent and Momentum.

그림 9.2. state 크기를 키울수록 성능이 단조적으로 향상되는곡선. 신경 메모리의 유효 용량이 모델 품질을 직접 규정한다는 명제를보여주며, '용량을 늘리려면 결국 물리 메모리를 더 써야 한다'는 이 장의다음 절로 자연스럽게 연결된다. (출처: LaCT, Figure 7, [6])

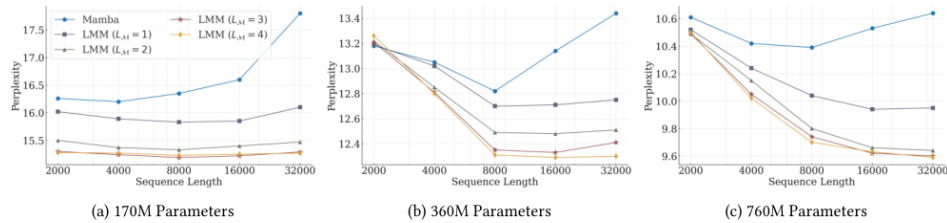


그림 9.3. 메모리 깊이 (memory depth) 에 따른 성능 변화: 메모리를깊게 할수록 특히 긴 시퀀스에서 perplexity가 개선됨을 보여준다. 용량을 '너비 (d)' 대신 '깊이' 로 사는 대안이 존재하지만, 그 대가는학습 · 연산 비용의 증가다. (출처: Titans, Figure 7, [8])

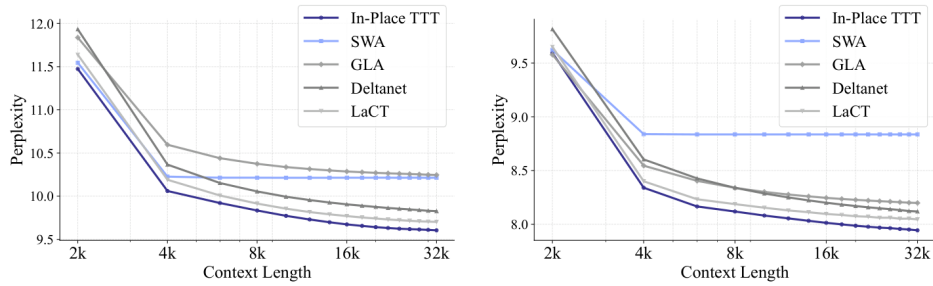


Figure 2 Sliding Window Perplexity at varying context lengths on the Pile dataset for 500M (left) and 1.5B (right) parameter models. Our In-Place TTT consistently achieves lower perplexity than all competitive baselines.

그림 9.4. 문맥 길이에 따른 sliding-window 방식의 perplexity 곡선. 윈도우 (=유효 메모리) 밖으로 밀려난 정보가 회수되지 않으면서 특정길이 이후 perplexity 개선이 멈추는 구간을 드러낸다. 유한 용량이 '길이일반화 실패'로 관측된다는 이 장의 핵심 증상. (출처: In-Place TTT, Figure 2, [22])

(ii) physical memory — state 가 HBM 에 올라왔을때

이제 두 번째 얼굴이다. 신경 메모리의 state 는 추상적 저장소인 동시에 HBM 을 물리적으로 점유한다. Linear/SSM 계열이 attention 대비자랑하는 것은 KV cache 가 문맥 길이에 따라 자라지 않고 고정 크기 state 로 상수 저장을 유지한다는 점이다. 그러나 이 장점은 serving 에서 곧바로 새로운 병목으로 뒤집힌다.

핵심은 decode 가 **memory-bandwidth bound** 라는 사실이다. 자기회귀 decode 의 매 스텝은 토큰 하나를 내는데 필요한 연산량이 작은 반면, 그 연산을 위해 전체 recurrent state 를 HBM 에서 읽고 다시 써야 한다. Gated DeltaNet 같은 linear attention 에서 batch-1 decode 를 하면, 매 토큰마다 full recurrent state 가 HBM 을 왕복하므로 실행 시간이 연산이 아니라 state 의 대역폭 왕복에 지배된다. 실제로 KVBuffer [27] 는 linear attention 의 state 가 per-token key-value 보다 훨씬 커서 매 스텝 재귀적으로 state 를 갱신하는 방식이 상당한 메모리 접근 비용을 유발한다고 지적한다. 여기서 앞 절의 결론과 물리 병목이 정면충돌한다 — 성능을 위해 state 를 키우면 (예: 파라미터의 40%), 바로 그 state 가 매 스텝 왕복해야 하는 대역폭 부담이 되어 decode 를 더 느리게 만든다.

표: physical memory 측에서 본 두 메모리 형태 (개념 정리)

항목	Attention (KV cache)	Linear/SSM (recurrent state)
footprint 의 문맥 길이 의존성	선형 증가 $\mathcal{O}(L)$	상수 $\mathcal{O}(1)$
decode 스텝당 HBM 트랙픽	과거 전체 KV 읽기	전체 state 읽기 · 쓰기 왕복
지배 병목	용량 (길이 $\uparrow \rightarrow$ OOM)	대역폭 (state $\uparrow \rightarrow$ decode 느려짐)
동시성 상한	batch $\times$ KV(길이)	batch $\times$ state

세 번째 행이 마지막 병목, 즉 **동시성 상한**이다. 서버가 HBM 한장에 담을 수 있는 총량은 유한하므로, 동시에 처리 가능한 요청 수는 대략 batch $\times$ (요청당 state 크기)로 묶인다. Attention 에서는 이 항이 길이에 비례해 커지고, linear 계열에서는 길이엔 무관하지만 성능을 위해 state 를 키운 만큼 커진다. 어느 쪽이든 “얼마나 많은 사용자를 동시에 태울 수 있는가”가 물리 메모리 예산에 직접 매인다.

이 대역폭 병목을 겨누는 시스템 연구들이 이미 등장했다. KVBuffer [27] 는 최근 key-value 를 버퍼링해 chunkwise 로 계산하고 state 갱신을 지연 · 일괄 적용하는 IO-aware serving 을 제안해, linear attention decode 지연을 최대 45% 줄이고 speculative decoding 에서 동시처리 요청 수를 크게 늘린다. 한편 별도의 하드웨어 분석 연구는 batch-1 LLM decode 가 “memory-bound 이지만 대역폭을 다 쓰지도 못하는” 물리적 비효율 구간에 있음을 44-cell cross-GPU 측정으로 지적

한다 — 예컨대 같은 workload 에서 L4 는 분석적 메모리 상한의 약 81%까지 도달하는 반면 H100 은 27%에 그친다 [43]. 요지는, state 기반 모델이 이론적 효율 (상수저장) 을 실측 처리량으로 옮기려면 알고리즘이 아니라 **메모리시스템** 층위의 설계가 필요하다는 것이다.

세 병목이 만나는지점

정리하면 memory device 는 다음처럼 세 병목을 한 점에 모은다. **Model-size** 병목은 신경 메모리의 유효 용량이 $\mathcal{O}(d)$ (또는 깊이 · feature map 으로 완화된 상한) 에 묶여 있어, 품질을 올리려면 state 를 키우는 것 외엔 근본 해법이 없다는데서 온다 [6]. **Training** 병목은 그 큰 state, 특히 깊은 메모리를 test-time 에 최적화하는 inner-loop 가 연산을 배가시키고 하드웨어활용률을 떨어뜨린다는 데서 온다 (TTT 계열의 낮은 활용률을 large-chunk 로 끌어올린 LaCT 가 이 증상의 학습 시점 대응책이다) [8][6]. **Serving** 병목은 바로 그 state 가 HBM 을 점유하고 매 decode 스텝대역폭을 왕복하며 동시성 상한을 규정한다는 데서 온다 [27]. 세 병목은 독립 변수가 아니라 하나의 자유도 — state 크기 — 를 서로 반대 방향으로 잡아당긴다: 용량을 위해 키우고 싶은 state 를, 학습 · serving 비용이 작게 유지하라고 민다.

이 장의 결론은 처방이 아니라 진단이다. “memory 를 신경망에 넣자” 는 아이디어는 유한 용량 · 간섭 · retrieval error 성장이라는 수학적 한계와, HBM 점유 · 대역폭 bound · 동시성 상한이라는 물리적 한계를 **분리해서 다룰 수 없게** 만든다. 다음 장에서 볼 개선들 (Omega rule, 깊은 메모리, gating, IO-aware serving) 은 모두 이 한 점에서 어느 병목을 조금 풀고 다른 병목에 얼마를 지불하는지의 교환으로 읽어야 한다.

10. memory device 집중 조명 ② — 개선으로 해결되는해법

들어가며 — 장치를 고치는 두갈래

앞 장이 memory device 의 병목을 “진단” 했다면 — 고정 크기 state 가 만드는 용량 천장, 압축이 유발하는 간섭 (interference), 그리고 그 위에서 벌어지는 retrieval · reasoning 붕괴 — 이 장은 그 device 자체를 **개선해서** 병목을 푸는 해법들을 다룬다. 핵심 긴장은 셋의 삼각형이다. 용량 (capacity) 을 키우면 FLOPs 가 커지고, FLOPs 를 묶으면 정확도가 흔들리며, 정확도를 지키려 정보를 다 담으면 footprint 가 폭발한다. 개선 해법들은 이 삼각형의 서로 다른 변을 끊는다. 알고리즘 · 구조 갈래 (i) 는 “용량과 FLOPs 를 분리” 하거나 “간섭을 구조적으로 줄이는” 방향이고, 물리 · 시스템 갈래 (ii) 는 같은 state 를 **더 싸게 저장 · 이동**시킨다. 두 갈래는 상보적이며, 억지로 하나의 정답으로 통합되지 않는다.

(i-a) expandable/sparse state — 용량은 키우고 FLOPs 는 묶는다

가장 직접적인 해법은 state 를 키우는 것이지만, dense 한 outer-product 갱신에서 state 를 키우면 갱신 FLOPs 가 그대로 따라 커진다. 세 논문이 서로다른 방식으로 이 둘을 떼어낸다.

LaCT (Test-Time Training Done Right) 는 아주 큰 chunk (2K~1M 토큰) 로 갱신을 묶어 GPU utilization 을 끌어올리고, 그 여력으로 **nonlinear state 를 모델 파라미터의 최대 40%까지** 키운다 [6]. 핵심 관찰은 기존 TTT 가 작은 minibatch 로 갱신해병렬성이 죽고, 그래서 state 를 키우지 못했다는 것이다. large-chunk 로 병렬성을 회복하니 A100 에서 최대 70% 이용률을 순수 PyTorch 수십 줄로 달성하고, 최대 56K 토큰 시퀀스의 14B AR video diffusion 까지 확장된다 [a].

이 그림이 중요한 이유는 “용량을 키우면 성능이 오른다” 는 가정이 **경험적으로 성립**함을 보여주기 때문이다. 병목이 진짜 용량이라면, 용량을 열어주는 순간 곡선이 우상향해야 한다. LaCT 는 그 곡선을 그려보이며 (원 논문 Figure 7 의 state-size scaling 패널), 이후 모든 expandable-state 논문의 전제를 정당화한다. 같은 그림의 다른 패널은 test-time optimizer 로 Muon 이 vanilla GD · momentum GD 를 앞선다는 점도 함께 보인다.

SSE (Sparse State Expansion) 는 갱신을 “정보분류 (classification)” 로 재해석한다 [33]. state 를 여러 partition 으로 확장하되, 각 토큰이 softmax top-k hard classification 으로 **일부 행 (row) 만** 갱신하도록 row-sparse 갱신을 쓴다. 그러면 파라미터 크기와 state 용량이 분리되고 (decoupling), 서로 다른 class 로 정보를 라우팅하니 inter-class 간섭이 준다. 하이브리드 SSE-H

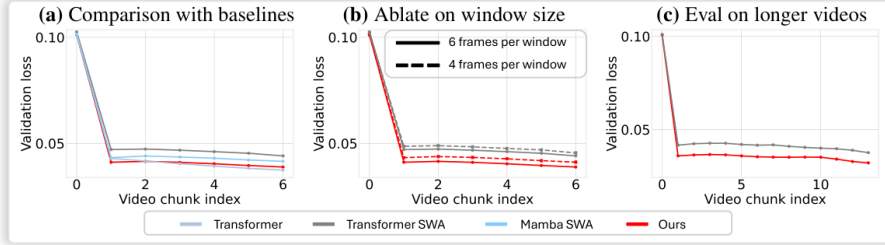


Figure 6: (a) We achieve comparable validation loss to the full-attention baseline and outperform both Mamba with sliding window and sliding window attention baselines. This improvement over SWA is consistent across different window sizes (b) and when evaluating on longer videos (c).

chunk-wise denoising loss across evaluated video frames. We only measure validation loss up to training sequence length. See project website for our autoregressively generated videos ².

5.4 Analysis on Design Choices

In this section, we analyze several key design choices in our model, focusing on both the novel view synthesis and language modeling tasks, where good metrics exist. Specifically, we evaluate the impact of state size (Fig. 7a), test-time optimizers (Fig. 7b), linear versus nonlinear fast weights (Fig. 8a), and per-token recurrence versus chunk-wise recurrence (Fig. 8b). Overall, we find that a large state size, advanced optimizers such as Muon, and nonlinear fast weights significantly improve our model’s performance. For comparing chunk recurrence with per-token recurrence, in a controlled NVS experiment, our linear large-chunk recurrence strategy outperforms linear per-token recurrence with the same state size. For language modeling, where chunk structures are not inherent, our linear large-chunk recurrence variant—while initially underperforming per-token methods like GLA and DeltaNet—surpasses them when combined with a large nonlinear state and the Muon optimizer. We refer the readers to each figure and its caption for more detailed analysis.

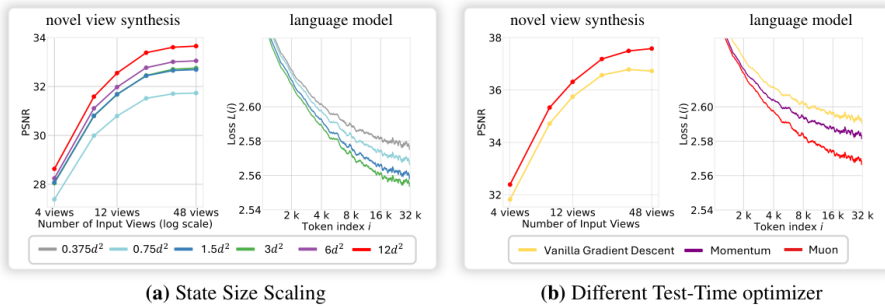


Figure 7: (a) Scaling up the state size consistently improves performance in both novel view synthesis and language modeling tasks. Note, the largest version has state size of $12d^2$ per block, totaling 40% of model weights as fast weights. (b) Comparison of test-time optimizers demonstrates Muon’s surprising effectiveness over Vanilla Gradient Descent and Momentum.

그림 10.1. nonlinear state size(fast weight) 를 키울수록 novel view synthesis·language modeling 모두에서 성능이 단조상승하는 scaling 곡선 — 용량 ↑ = 성능 ↑ 이 실제로 성립함을 보인다 (출처: LaCT, Figure 7, [6])

는 RULER 의 single/multi-needle 태스크 (최대 32k) 에서 GLA-H 계열을 넘고 Transformer 수준에 근접한다 [a].

Sparse Delta Memory(SDM) 는 Gated DeltaNet 의 dense KV outer-product 를 큰 **explicit memory** 에 대한 **sparse read/write** 로 바꾼다 [44]. isoFLOP· 동일파라미터 제약 아래에서 state 용량을 orders-of-magnitude 로 키우면 in-context learning 과 long-context retrieval 이 크게 개선된다. 세 논문의 공통 논리는 하나다 — 주소 (**addressing**) 를 **sparse** 하게 만들면, 큰용량을 유지하면서도 매 토큰이 건드리는 FLOPs 와 간섭을 작게 묶을 수있다.

(i-b) exact memory — 압축하지 말고 정확히 저장한다

sparse-state 가 “큰 압축 메모리” 라면, 다른 갈래는 압축을 아예 포기하고 필요한 것만 정확히 저장한다. **HOLA**(A Hippocampus for Linear Attention) 는 semiparametric 구조를 제안한다 [45]. 기존 delta-rule state 는 “선형적으로 압축 가능한 구조” 를 담는 compressive memory 로 두고, 그 옆에 **bounded exact KV cache** 를 붙여 “state 에 억지로 밀어 넣으면 안 되는 연상 (association)” 을 정확히 보관한다. 즉 recurrent state 가 “잊어버리는 것” 만 hippocampus 가 정확 저장한다. 340M 파라미터를 15B SlimPajama 토큰으로 학습한 설정에서의 결과는 다음과 같다.

표: HOLA(340M, 15B SlimPajama) 의 언어모델링 ·retrieval 결과 [a]

모델	Wikitext ppl	LAMBADA ppl	비고
delta-rule base	27.32	30.95	HOLA 의 출발점 (압축 state)
Transformer++ (full attn)	26.88	—	full-attention 기준선
HOLA (delta-rule + bounded exact KV)	22.92	30.26	base 대비 -16.1%

여기서 수치를 정확히 읽는 것이 중요하다. HOLA 는 자신의 delta-rule base(27.32) 의 Wikitext ppl 을 22.92 로 **16.1% 낮추고**, 이 값은 full-attention Transformer++(26.88) 까지도 밑돈다 [45]. 즉 -16.1%는 Transformer++ 대비가 아니라 **자신의 압축 base 대비** 수치이며, Transformer 자체와의 비교에서도 HOLA 가 앞선다 (약 14.7% 낮음)[a]. 여기에 linear in-context retrieval 최고 성능을 내고, RULER needle-in-a-haystack 에서 학습 길이의 16 배인 32k 토큰 까지 robust 하다는 점이 더해진다. 이는 “압축 대신 정확 저장” 이 단지 방어책이 아니라 순수이득일 수 있음을 시사하지만 [b], 단일 저자 preprint 이고 규모도 340M/15B 로 작아 독립 재현 검증은 아직 없다.

EFLA(Exact Flow / Error-Free Linear Attention) 는 다른 각도에서 “정확” 을 추구한다 [46]. delta-rule 갱신이 실은 연속시간동역학의 **explicit Euler 이산화**임을 보이고, dynamics 행렬의 rank-1 구조 덕분에 matrix exponential 과 입력 적분이 닫힌 형태로 붕괴한다는 점을 이용해 **1 차 Euler 오차를 제거한 exact closed-form flow** 갱신으로 바꾼다 (이 closed-form 은 무한차 Runge – Kutta 에 해당). 파라미터 · 선형복잡도 · chunkwise 병렬성을 그대로 두고 discretization error 만 없애 DeltaNet 대비 ppl 과 downstream 이 개선되며, 특히 입력 강도를 키운 조건에서 Euler 기반 DeltaNet 이 붕괴할 때 EFLA 는 안정적이다 — “free lunch” 라는 제목 그대로다 [a].

이 “정확” 에는 대가가 있다. **MesaNet** 은 매 토큰에서 지금까지 본 토큰의 누적 in-context 손실을 conjugate gradient(CG) 로 **국소최적까지 풀어** state 를 정한다 [19]. 정확한 해를 얻지만, 그만큼 토큰마다 solver 반복이 늘어 test-time compute 가 동적으로 증가한다.

이 그림은 exact 갈래 전체의 숨은 청구서를 드러낸다. CG step k 가 클수록 gated linear attention 대비 약 k 배 더 많은 flops 를 쓴다는 것 — 그래서 HOLA 의 “bounded” cache 나 EFLA 의 “닫힌 형태” 같은 장치가 중요하다. 무한정 정확은 무한정 비싸기 때문이다 a.

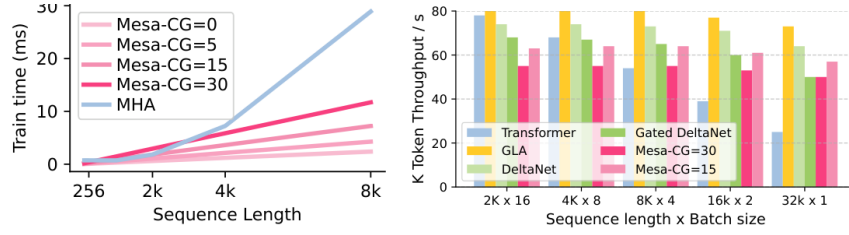


Figure 2: Train and inference time of a Mesa layer using different number of CG steps. *Left:* Train time of a single Mesa layer on a TPUv5: output the entire sequence, compute the cross entropy loss, and gradients w.r.t. layer parameters. We use batch size of 4, key size of 128 and 8 heads. *Right:* Token throughput (in thousands) when training 1B parameter models on a H100 GPU. We compare a Flash-Attention-2 (Dao, 2023) transformer implementation with a triton-based chunkwise parallel implementation of RNN models, including the MesaNet which uses 30 or 15 CG steps across all layers. All models use a key size of 128 and share the same backbone, see Appendix G. We observe competitive token throughput on H100s of the MesaNet despite using substantially more flops.

그림 10.2. MesaNet 의 비용 프로파일 — TPUv5 추론에서 CG step 수를 늘릴 때의 토큰당 시간과, H100 에서의 학습 throughput 비교를 보인다 (출처: MesaNet, Figure 2, [19])

(i-c) growing memory — state 가 문맥과 함께자란다

세 번째 갈래는 state 크기 자체를 문맥 길이에 따라 자라게 한다. **Memory Caching(MC)** 은 RNN 의 hidden state 를 가변 길이 세그먼트끝마다 checkpoint 로 캐싱하고, 각 토큰이 online memory 와 과거 cached memory 들을 함께 attend 한다 [47]. 복잡도는 $O(NL)$ 로 제어되어, RNN 의 $O(L)$ 과 Transformer 의 $O(L^2)$ 사이를 연속적으로 보간한다. recall-intensive 태스크에서 고정 크기 state 가 지는 이유가 용량이라면, 용량을 길이에 비례해 늘리되 이차 비용은 피하는 절충이다.

Key-Value Means(KVM) 는 fixed-size 와 growing state 를 하나의 block-recurrence 로 통합한다 [48]. growable KVM cache 를 쓰면 subquadratic prefill 과 sublinear state 성장으로 long-context 에서 경쟁력을 내고, 커스텀 커널 없이 chunk-wise 병렬학습·prefill 이 되며 prefill 복잡도를 $O(N) \sim O(N^2)$ 사이에서 연속적으로 고를 수 있다. 두 논문모두 “고정 크기가 병목이면 크기를 놓아주되, 이차로 터지지 않게 성장속도를 통제한다” 는 동일한 처방을 공유한다.

(i-d) erase-write 주소 분리 — 간섭의 뿌리를 끊는다

간섭은 종종 하나의 스칼라 게이트가 두 일을 겸하는 데서 온다. Gated DeltaNet·KDA 에서 active edit 는 스칼라 게이트 하나로 “key 쪽에서 옛 연상을 얼마나 지울지” 와 “value 쪽에서 새 내용을 얼마나 쓸지” 를 동시에 결정한다. **Gated DeltaNet-2** 는 KDA 의 channel-wise decay 는 물려받되 이 스칼라 결속을 끊어, key 축의 channel-wise **erase gate** 와 value 축의 channel-wise **write gate** 로 분리한다 [49]. 지우기와 쓰기를 독립적으로 조절하니, 특히 여러 key 가 경쟁하는 multi-key retrieval 에서 고정 크기 state 가 서로 다른 연상을 더 깔끔히 분리한다. 1.3B 파라미터를 100B FineWeb-Edu 토큰으로 학습한 설정에서 Mamba-2·Gated DeltaNet·KDA·Mamba-3 변형들 사이에서 가장 강한 종합 성능을 보고한다 [a]. 논리는 단순하고 강력하다: 주소공간을 분리하면 덮어쓰기 간섭이 준다.

(i-e) 고용량 feature map — 상태의 유효 차원을 키운다

state 행렬 크기를 직접 키우지 않고, **feature map $\phi(\cdot)$ 의 차수**를 올려 유효 용량을 늘리는 길도 있다. **Atlas** 는 다항 (polynomial) feature mapping 으로 key 의 유효차원 (따라서 메모리 용량) 을 키우고, Omega rule(슬라이딩 윈도우의 모든 과거토큰으로 갱신) 과 Muon optimizer 로 표현력·최적성을 높인다 [10]. 엄밀히는 다항 feature map+Omega rule 변형이 OmegaNet 이고, 여기에 2 차 정보 근사인 Muon 을 얹어 국소 최적 메모리를 만든 것이 Atlas 다. 이론적으로 메모리 용량은 다항 차수 p 에 대해 $O(d_k^p)$ 로 커진다.

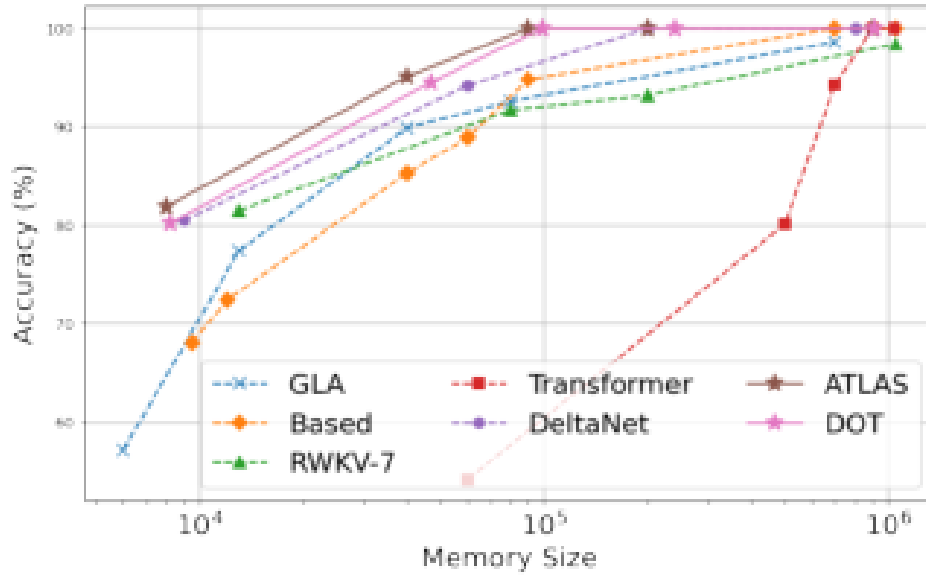


그림 10.3. Atlas 의 용량 — 다항 feature map 으로 associative recall 용량이 커짐을 보인다 (출처: Atlas, Figure 7, [10])

이 그림은 “feature map 차수 $\uparrow$ $\rightarrow$ 저장 가능한 연상 수 $\uparrow$ ” 라는 인과를 시각화한다. 선형 ϕ 의 유효 rank 가 용량 천장이라면, 고차 ϕ 는 그 천장을 끌어올린다 — 다만 ϕ 의 차원이 커지면 state와 FLOPs도 커지므로 이 역시 삼각형 안의 절충이다 [b].

sympow(Symmetric Power) transformer는 대칭 텐서 (symmetric tensor) 임베딩으로 ϕ 를 구성하고 차수 p 로 state 크기를 조절한다 [41]. p 를 키우면 작은 recurrent state로 softmax transformer에 필적하는 성능을 내지만, 유한 state 용량은 문맥이 길어지면 여전히 천장에 부딪힌다. 그래서 후속 conformal-sympow는 data-dependent multiplicative gating과 rotary 임베딩으로 용량을 동적으로 비워/재배치해 이 한계를 완화한다 — 고용량 feature map도 만능은 아니라는 skeptic 지점이다 [a].

(i-f) 압축 — 잘 버려서 정확도를 지킨다

exact 저장의 반대편에는 **똑똑한 압축**이 있다. **Lattice**는 K-V의 저 rank 구조를 이용해 cache를 고정 슬롯 수로 압축하되, 각 슬롯을 현재 상태에 직교 (orthogonal) 하는 정보로만 갱신한다 [50]. 새롭고 비중복인 정보만 받아들여 기존 상태와의 간섭을 최소화하는 것이 핵심이며, 갱신 규칙은 온라인 최적화의 단일 gradient step + 입력 의존 gating으로 유도된다. **Trellis**는 고정 크기 메모리에 대해 forget gate를 가진 **two-pass recurrent 압축**을 test-time에 online gradient descent로 학습해, 들어오는 토큰의 중요 문맥을 재귀적으로 저장한다 [51]. 둘 다 “무엇을 버릴지”를 학습해 정확도 손실을 억제한다.

LoLA는 training-free 증강으로, 과거 토큰을 셋으로 라우팅한다 — 최근 KV는 sliding window cache, 외우기 어려운 (interfering) KV는 sparse global cache, 나머지는 recurrent state [52]. 판별 기준이 영리하다: **self-recall check**로 현재 state의 선형 연상 map이 자기 value를 되불러오지 못하는 KV만 full-rank로 sparse 캐싱한다. 즉 state가 이미 잘 외운 것은 놔두고, 간섭을 일으키는 것만 정확히 따로 둔다.

표: 압축·캐싱 계열의 대표 정량 결과

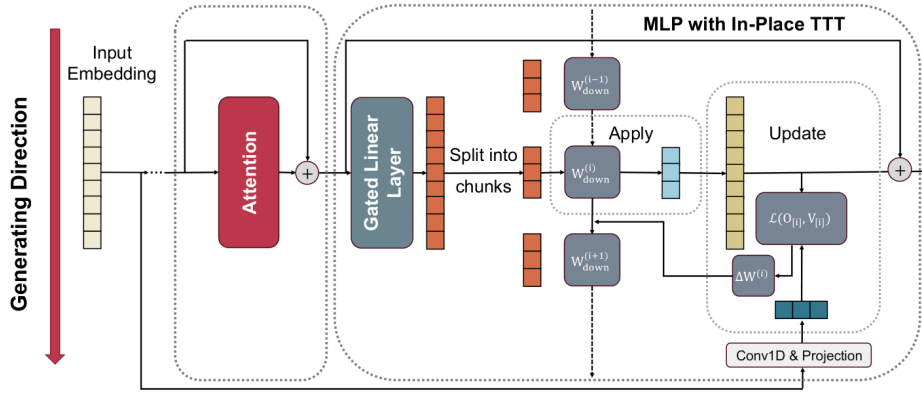


Figure 1 The overall framework of our In-Place Test-Time Training. The module operates sequentially on input chunks. For each chunk, the current fast weights are first *applied* to the intermediate activations $\mathbf{Z}$ to produce the output. Then, these weights are *updated* using the activations $\mathbf{Z}$ and a value $\mathbf{V}$ derived from the token embeddings. This "apply-then-update" cycle allows the model to dynamically adapt to incoming context in a strictly causal manner.

그림 10.4. In-Place TTT — 각 chunk 마다 현재 fast weight 를적용해 출력을 내고 (apply) 곧 바로 그 활성으로 가중치를 갱신하는 (update) 구조. 기존 MLP projection 가중치를 fast weight 로 재활용해 별도 state 저장도 없다 (출처: In-Place TTT, Figure 1, [22])

방법	핵심 메커니즘	대표 수치
LoLA [52]	self-recall 기반 sparse cache	passkey 0.6% → 97.4% , Llama-3.1 8B 대비 cache $4.6\times$ 작음 (4K)[a]
Lattice [50]	직교 갱신, 저 rank 압축	고정 슬롯으로 sub-quadratic, 강 baseline 상회 [a]
Trellis [51]	two-pass, forget gate	recall·시계열등에서 강 baseline 상회 [a]

LoLA 의 0.6%→97.4%는 같은 가중치·더 작은 **cache** 로얼은 값이라, “간접 KV 만 골라내는” 진단이 얼마나 결정적인지를보여준다 [a].

(ii) physical — 같은 state 를 더 싸게 저장하고옮긴다

구조가 아니라 물리적 비용을 치는 갈래도 있다. **TTQ**(Activation-Aware Test-Time Quantization) 는 오프라인 calibration 데이터에 의존하지 않고 추론 시점에 효율적인 online calibration 으로 activation-aware 양자화를 걸어 footprint 를 줄이고 추론을가속한다 [53]. AWQ/GPTQ 가 calibration 데이터에 의존해 겪는 도메인 시프트 위험을 피하면서 매 프롬프트에 즉석 적응하는 방식이다.

KVBuffer 는 대역폭 (bandwidth) 병목을 친다 [27]. linear attention 서빙은 매 디코딩 스텝마다 토큰당 KV 보다 훨씬 큰 state 를 읽고 쓰기 때문에 memory access 가 커진다. KVBuffer 는 최근 key/value 를 버퍼링해 state 갱신을 미뤘다가 chunk 단위로일괄 적용하고 (갱신 비용을 여러 스텝에 amortize), 짧은 문맥은 버퍼에서직접 출력을 계산해 state 를 아예 만들지 않는다. SGLang·Qwen3-Next 구현에서 decoding latency 를 최대 **45.17%** 줄이고, 4 개 draft 토큰을 검증하는 speculative decoding 에서 최대 서빙 요청 수를 $5\times$ 늘린다 [a].

In-Place TTT 는 가장 급진적이다 — **별도 state 를없앤다** [22]. MLP 블록의 최종 projection 행렬 (W_{down}) 을 그대로 fast weight 로 삼아 추론 중 갱신하니, TTT 를 위한 추가 메모리구조·모듈이 필요 없다.

이 그림이 중요한 이유는, 앞선 모든 해법이 “state 를 어떻게 키우고압축하고 옮길까” 를 고민한 반면 In-Place 는 “**state 라는 별도 객체를없앨 수 있다**” 는 정반대 답을 제시하기 때문이다.

reconstruction 대신 next-token-prediction에 정렬된 목적함수와 chunk-wise “apply-then-update” 갱신으로, 사전학습 모델에 drop-in으로 붙거나 scratch 학습에서 경쟁 TTT 기법을 상회한다고 보고된다 (ICLR 2026 Oral)[a]. footprint 관점에서 “가장 싼 state는 존재하지 않는 state”라는 논리다.

정리 — 하나의 정답이 아니라 절충의지도

이 장의 해법들은 서로 대체가 아니라 **삼각형 위의 서로 다른좌표**다. sparse-state(i-a)와 growing-memory(i-c)는 용량을 열되 FLOPs·성장속도를 통제하고, exact-memory(i-b)는 정확도를사되 bounded cache나 닫힌 형태로 비용을 막으며, erase-write 분리(i-d)와 압축(i-f)은 간섭이라는 정확도의 뿌리를 구조적으로 끊는다. 고용량 feature map(i-e)은 state를 키우지 않고 유효 차원을 올린다. 물리 갈래(ii)는 이모든 것의 **저장·이동·존재 비용**을 낮춘다. 공통 교훈은 앞 장의 진단과 정확히 맞물린다 — 병목이 “용량”이면 용량을 열고, “간섭”이면 주소를 분리하며, “footprint”면 양자화·재활용으로 친다. 다만 상당수가 2026년 preprint(일부는 단일 저자·소규모)라 재현·독립검증은 얇고, 표의 수치는 저자 보고값임을 기억해야 한다.

주요 출처: Sparse State Expansion [2507.16577]; Sparse Delta Memory [2607.07386]; HOLA [2607.02303]; EFLA [2512.12602]; Key-Value Means [2605.09877]; Memory Caching [2602.24281]; Lattice [2504.05646]; Trellis [2512.23852]; LoLA [2505.23666]; TTQ [2603.19296]; KVBuffer [2605.19049]; In-Place TTT [2604.06169]; LaCT [2505.23884]; Atlas [2505.23735]; MesaNet [2506.05233]; Gated DeltaNet-2 [2605.22791]; sym-pow/conformal-sympow [2503.03269].

Part IV 정리. memory device는 세 병목이 만나는 물리적 결절 — **용량·간섭·대역폭**. 개선은 ‘더 크게 (sparse/expandable)·더 정확히 (exact)·더 싸게 (quantize/in-place)’의 세 축이며 각 축이 특정 병목을 직접 겨룬다.

11. 종합 — 세 병목 × 해법 × memory device 지도

병목	근본 원인	대표 해법 (무엇을)	왜 working	memory device 연결
① model-size	문맥을 파라미터가 아니라 유한 메모리 용량으로 저장 (d^2 에 ~ d)	distill/linearize로 규모 상속, 용량 확장, 파라미터 성장	규모는 물려받고 용량은 feature/state로 키움	용량 상한 이 진원
② training	inner-loop 순차 의존 → 작은 chunk면 FLOPs util < 5%	큰 chunk(LaCT), chunkwise-parallel, 비선형 recurrence 병렬화	순차성을 chunk 경계로 밀거나 병렬 형태로 접음	state footprint 가 chunk·batch 제약
③ serving	request마다 mutable state 소유 → batching·prefix-cache 붕괴	RW-TTT, hybrid prefix caching, In-Place, IO-aware	재사용·배치 프리미티브를 recurrent에 맞게 재설계	대역폭·footprint 가 decode bound

관통하는 한 줄: 모든 효율 이득은 결국 **memory device** 의 유한한 용량 · 대역폭과 거래한다. 표현력을 위해 state 를 키우면 training FLOPs·serving footprint 가 오르고, 이를 줄이려 압축 · 양자화하면 용량 · 정확도가 깎인다. 프론티어는 세 꼭짓점을 ‘동시에’ 미는 것 — LaCT(큰 chunk+ 큰 state), In-Place TTT(state 없이 기존 가중치 재활용), RW-TTT(요청별 state 배치서빙) — 이 나 이들은 아직 서로 다른 갈래다.

12. 열린 질문

1. **큰 chunk vs 세밀한 적응.** LaCT 의 2K~1M chunk 는 util 을 살리지만 chunk 내부의 순차 의존을 지연시킨다. 정확도 손실 없이 둘을 동시에 얻는 chunk 스케줄의 이론적 상한은? [c]
2. **TTT $\approx$ linear attention 등가의 함의.** 복잡한 다층-MLP·momentum TTT 조차 학습된 linear attention operator 로 등가 재작성된다면, “test-time 학습” 의 표현력 우위는 실제로 어디서 오는가? [c]
3. **용량 벽.** d^2 에 $\sim d$ 라는 associative-memory 용량한계를 sparse/expandable state 가 상수배 개선인지, 스케일링 지수 자체를 바꾸는지. [c]
4. **request-owned state 서빙의 스케일.** RW-TTT 가 수천 동시요청에서도 성립하나 — 요청별 fast weight 의 메모리 폭발과 phase 호환배치의 한계. [b]
5. **hybrid recurrent state 의 표준 캐시 프리미티브.** Marconi·HYPIC·Sparse Prefix 가 제각각인데 KV cache 에 준하는 표준이 나올 수 있나? [c]
6. **scratch 대규모 학습의 길.** model-size 병목이 distillation 으로만 실용적으로 풀린다면, TTT/Titans 를 처음부터 초대형으로 pre-train 하는 경로는 닫힌 것인가? [c]
7. **재현성.** Titans Revisited 의 지적처럼, neural memory 의 실이득이 어느 규모 · 태스크에서 baseline 을 확실히 넘는지 아직 합의가 없다. [a]
8. **물리-알고리즘 co-design.** memory device 의 물리 (HBM 용량 · 대역폭) 와 알고리즘 (state 용량 · chunk) 을 함께 모델링하는 비용 프레임으로 무엇을 예측할 수 있나? [c]

References

1. Learning to (Learn at Test Time): RNNs with Expressive Hidden States (TTT-Linear/MLP). arXiv:2407.04620. <https://arxiv.org/abs/2407.04620>
2. Linear Transformers Are Secretly Fast Weight Programmers. arXiv:2102.11174. <https://arxiv.org/abs/2102.11174>
3. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. arXiv:2006.16236. <https://arxiv.org/abs/2006.16236>
4. Gated Linear Attention Transformers with Hardware-Efficient Training (FlashLinearAttention). arXiv:2312.06635. <https://arxiv.org/abs/2312.06635>
5. Test-time regression: a unifying framework for designing sequence models with associative memory. arXiv:2501.12352. <https://arxiv.org/abs/2501.12352>
6. Test-Time Training Done Right (LaCT). arXiv:2505.23884. <https://arxiv.org/abs/2505.23884>
7. Test-Time Training with KV Binding Is Secretly Linear Attention. arXiv:2602.21204. <https://arxiv.org/abs/2602.21204>
8. Titans: Learning to Memorize at Test Time. arXiv:2501.00663. <https://arxiv.org/abs/2501.00663>
9. It’s All Connected: A Journey Through Test-Time Memorization, Attentional Bias, Retention, and Online Optimization (Miras). arXiv:2504.13173. <https://arxiv.org/abs/2504.13173>
10. ATLAS: Learning to Optimally Memorize the Context at Test Time. arXiv:2505.23735. <https://arxiv.org/abs/2505.23735>

11. TNT: Improving Chunkwise Training for Test-Time Memorization. arXiv:2511.07343. <https://arxiv.org/abs/2511.07343>
12. Nested Learning: The Illusion of Deep Learning Architectures (HOPE). arXiv:2512.24695. <https://arxiv.org/abs/2512.24695>
13. Language Models Need Sleep: Learning to Self-Modify and Consolidate Memories. arXiv:2606.03979. <https://arxiv.org/abs/2606.03979>
14. Titans Revisited: A Lightweight Reimplementation and Critical Analysis of a Test-Time Memory Model. arXiv:2510.09551. <https://arxiv.org/abs/2510.09551>
15. (제목 확인 필요). arXiv:2605.05189. <https://arxiv.org/abs/2605.05189>
16. Hopfield Networks is All You Need. arXiv:2008.02217. <https://arxiv.org/abs/2008.02217>
17. Muon Outperforms Adam in Tail-End Associative Memory Learning. arXiv:2509.26030. <https://arxiv.org/abs/2509.26030>
18. Nemotron-H: A Family of Accurate and Efficient Hybrid Mamba-Transformer Models. arXiv:2504.03624. <https://arxiv.org/abs/2504.03624>
19. MesaNet: Sequence Modeling by Locally Optimal Test-Time Training. arXiv:2506.05233. <https://arxiv.org/abs/2506.05233>
20. Parallelizing Linear Transformers with the Delta Rule over Sequence Length. arXiv:2406.06484. <https://arxiv.org/abs/2406.06484>
21. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752. <https://arxiv.org/abs/2312.00752>
22. In-Place Test-Time Training. arXiv:2604.06169. <https://arxiv.org/abs/2604.06169>
23. RW-TTT: Batched Serving for Request-Owned Test-Time Training State. arXiv:2605.28053. <https://arxiv.org/abs/2605.28053>
24. Marconi: Prefix Caching for the Era of Hybrid LLMs. arXiv:2411.19379. <https://arxiv.org/abs/2411.19379>
25. HYPIC: Accelerating Hybrid-Attention LLM Serving with Position-Independent Caching. arXiv:2607.01299. <https://arxiv.org/abs/2607.01299>
26. (제목 확인 필요). arXiv:2401.09670. <https://arxiv.org/abs/2401.09670>
27. KVBuffer: IO-aware Serving for Linear Attention. arXiv:2605.19049. <https://arxiv.org/abs/2605.19049>
28. Kimi Linear: An Expressive, Efficient Attention Architecture. arXiv:2510.26692. <https://arxiv.org/abs/2510.26692>
29. Transformers to SSMs: Distilling Quadratic Knowledge to Subquadratic Models (MOHAWK/Phi-Mamba). arXiv:2408.10189. <https://arxiv.org/abs/2408.10189>
30. Llama: Scaling Distilled Recurrent Models for Efficient Language Processing. arXiv:2502.14458. <https://arxiv.org/abs/2502.14458>
31. LoLCATs: On Low-Rank Linearizing of Large Language Models. arXiv:2410.10254. <https://arxiv.org/abs/2410.10254>
32. Liger: Linearizing Large Language Models to Gated Recurrent Structures. arXiv:2503.01496. <https://arxiv.org/abs/2503.01496>
33. Scaling Linear Attention with Sparse State Expansion. arXiv:2507.16577. <https://arxiv.org/abs/2507.16577>
34. Transformers are SSMs: Structured State Space Duality (Mamba-2). arXiv:2405.21060. <https://arxiv.org/abs/2405.21060>
35. Comba: Improving Bilinear RNNs with Closed-loop Control. arXiv:2506.02475. <https://arxiv.org/abs/2506.02475>
36. DEER: Parallelizing non-linear sequential models over the sequence length. arXiv:2309.12252. <https://arxiv.org/abs/2309.12252>
37. ParaRNN: Unlocking Parallel Training of Nonlinear RNNs for Large Language Models. arXiv:2510.21450. <https://arxiv.org/abs/2510.21450>

38. Predictability Enables Parallelization of Nonlinear State Space Models. arXiv:2508.16817. <https://arxiv.org/abs/2508.16817>
39. Tiled Flash Linear Attention: More Efficient Linear RNN and xLSTM Kernels. arXiv:2503.14376. <https://arxiv.org/abs/2503.14376>
40. Sparse Prefix Caching for Hybrid and Recurrent LLM Serving. arXiv:2605.05219. <https://arxiv.org/abs/2605.05219>
41. Conformal Transformations for Symmetric Power Transformers. arXiv:2503.03269. <https://arxiv.org/abs/2503.03269>
42. Understanding and Enhancing Mamba-Transformer Hybrids for Memory Recall and Language Modeling. arXiv:2510.26912. <https://arxiv.org/abs/2510.26912>
43. (제목 확인 필요). arXiv:2605.30571. <https://arxiv.org/abs/2605.30571>
44. Sparse Delta Memory: Scaling the State of Linear RNNs through Sparsity. arXiv:2607.07386. <https://arxiv.org/abs/2607.07386>
45. A Hippocampus for Linear Attention: An Exact Memory for What the Recurrent State Forgets. arXiv:2607.02303. <https://arxiv.org/abs/2607.02303>
46. Error-Free Linear Attention (EFLA): Exact Solution from Continuous-Time Dynamics. arXiv:2512.12602. <https://arxiv.org/abs/2512.12602>
47. Memory Caching: RNNs with Growing Memory. arXiv:2602.24281. <https://arxiv.org/abs/2602.24281>
48. Key-Value Means: Transformers with Expandable Block-Recurrent Compressed Memory. arXiv:2605.09877. <https://arxiv.org/abs/2605.09877>
49. Gated DeltaNet-2: Decoupling Erase and Write in Linear Attention. arXiv:2605.22791. <https://arxiv.org/abs/2605.22791>
50. Lattice: Learning to Efficiently Compress the Memory. arXiv:2504.05646. <https://arxiv.org/abs/2504.05646>
51. Trellis: Learning to Compress Key-Value Memory in Attention Models. arXiv:2512.23852. <https://arxiv.org/abs/2512.23852>
52. LoLA: Low-Rank Linear Attention With Sparse Caching. arXiv:2505.23666. <https://arxiv.org/abs/2505.23666>
53. TTQ: Activation-Aware Test-Time Quantization to Accelerate LLM Inference On The Fly. arXiv:2603.19296. <https://arxiv.org/abs/2603.19296>